

# Overview of capabilities

- Overview of capabilities
  - Main features
  - Capabilities related to equilibria
  - (limited) Capabilities related to maps
  - Summary table for equilibria
  - Capabilities related to Periodic orbits (PO)
  - Capabilities related to Homoclinic orbits
  - List of detected bifurcations

## Main features

- Newton-Krylov solver with generic linear / eigen *preconditioned* solver. Idem for the arc-length continuation.
- Newton-Krylov solver with nonlinear deflation and preconditioner. It can be used for branch switching for example. It is used for deflated continuation.
- Continuation written as an *iterator*.
- Monitoring user functions along curves computed by continuation, see *events*.
- Continuation methods: PALC, Moore Penrose, Multiple, Polynomial, Deflated continuation, ANM, ...
- Bifurcation points / events located with bisection.
- Compatible with GPU

## Capabilities related to equilibria

- Detection of Branch, Fold, Hopf bifurcation points of stationary solutions and computation of their normal form. Other non-generic bifurcations based on spectrum are also detected.
- Automatic branch switching at branch points (**whatever the dimension of the kernel**) to equilibria
- Automatic computation of bifurcation diagrams of equilibria**
- Fold / Hopf continuation based on Minimally Augmented formulation, with Matrix Free / Sparse / Dense Jacobian.
- Detection of all codim 2 bifurcations of equilibria and computation of the normal forms of Bogdanov-Takens, Bautin, Cusp, Zero-Hopf. (Hopf-Hopf normal form not implemented)
- Branching from Bogdanov-Takens / Zero-Hopf / Hopf-Hopf points to Fold / Hopf curve

## (limited) Capabilities related to maps

- continuation of fixed points of maps
- computation of normal form of Period-doubling, Neimark-Sacker, Branch point bifurcations.

## Summary table for equilibria

Note that you can combine most solvers, like use Deflation for Periodic orbit computation or Fold of periodic orbits family.

Custom state means, you can use something else than AbstractArray, for example your own struct.

|          |             |              |          |     |
|----------|-------------|--------------|----------|-----|
| Features | Matrix Free | Custom state | Tutorial | GPU |
|----------|-------------|--------------|----------|-----|

| BifurcationKit                                                              | MultiParamContinuation                                                  | DDEBifurcationKit | HclinicBifurcationKit | Tutorials           | Citing                                                          |     |
|-----------------------------------------------------------------------------|-------------------------------------------------------------------------|-------------------|-----------------------|---------------------|-----------------------------------------------------------------|-----|
| GitHub                                                                      | Continuation PALC (Natural, Secant, Tangent, Polynomial)                |                   | Yes                   | Yes                 | All                                                             | Yes |
|                                                                             | Deflated Continuation                                                   |                   | Yes                   | Yes                 | <a href="#">link</a>                                            | Yes |
|                                                                             | Bifurcation / Fold / Hopf point detection                               |                   | Yes                   | Yes                 | All / All / <a href="#">link</a>                                | Yes |
|                                                                             | Fold Point continuation                                                 |                   | Yes                   | Yes                 | <a href="#">PDE</a> , <a href="#">PDE</a> , <a href="#">ODE</a> | Yes |
|                                                                             | Hopf Point continuation                                                 |                   | Yes                   | AbstractArray       | <a href="#">ODE</a>                                             |     |
|                                                                             | Branch point / Fold / Hopf normal form                                  |                   | Yes                   | Yes                 |                                                                 | Yes |
|                                                                             | Branch switching at Branch points                                       |                   | Yes                   | AbstractArray       | <a href="#">link</a>                                            | Yes |
|                                                                             | Automatic bifurcation diagram computation of equilibria                 |                   | Yes                   | AbstractArray       | <a href="#">link</a>                                            |     |
|                                                                             | Bogdanov-Takens / Bautin / Cusp / Zero-Hopf / Hopf-Hopf point detection |                   | Yes                   | Yes                 | <a href="#">ODE</a>                                             |     |
|                                                                             | Bogdanov-Takens / Bautin / Cusp normal forms                            |                   | Yes                   | AbstractArray       | <a href="#">ODE</a>                                             | Yes |
| Branching from Bogdanov-Takens / Zero-Hopf / Hopf-Hopf to Fold / Hopf curve |                                                                         | Yes               | AbstractArray         | <a href="#">ODE</a> |                                                                 |     |

## Capabilities related to Periodic orbits (PO)

- PO computation and continuation using **parallel** (Standard or Poincaré) Shooting, Finite Differences or Orthogonal Collocation (mesh adaptive).
- Automatic branch switching from simple Hopf points to PO
- Automatic branch switching from simple Period-Doubling points to PO
- Assisted branch switching from simple Branch points to PO
- Detection of Branch, Fold, Neimark-Sacker (NS), Period Doubling (PD) bifurcation points of PO.
- Fold / PD / NS continuation based on Minimally Augmented formulation (for shooting and collocation). Trapezoid method only allows continuing Fold of PO.
- Detection of all codim 2 bifurcations of PO (R1, R2, R3, R4, GPD, NS-NS, Chenciner, Fold-Flip, Fold-NS, PD-NS)
- Computation of the normal forms of PD, NS (for shooting and collocation) using the method based on Poincaré return map or the looss normal form.
- automatic branching from Bautin to curve of Fold of PO
- automatic branching from Zero-Hopf to curve of NS of PO
- automatic branching from Hopf-Hopf to curve of NS of PO

Legend for the table: Standard shooting (SS), Poincaré shooting (PS), Orthogonal collocation (OC), trapezoid (T).

| Features                        | Method     | Matrix Free | Custom state   | <a href="#">Tutorial</a>                  | GPU |
|---------------------------------|------------|-------------|----------------|-------------------------------------------|-----|
| Branch switching at Hopf points | SS/PS/OC/T | See each    |                | <a href="#">ODE</a>                       |     |
| Newton / continuation           | T          | Yes         | AbstractVector | <a href="#">PDE</a> , <a href="#">PDE</a> | Yes |
| Newton / continuation           | OC         |             | AbstractVector | <a href="#">ODE</a>                       |     |
| Newton / continuation           | SS         | Yes         | AbstractArray  | <a href="#">ODE</a>                       | Yes |
| Newton / continuation           | PS         | Yes         | AbstractArray  | <a href="#">PDE</a>                       | Yes |

| BifurcationKit | MultiParamContinuation                                               | DDEBifurcationKit | HclinicBifurcationKit | Tutorials      | Citing                                     |     |
|----------------|----------------------------------------------------------------------|-------------------|-----------------------|----------------|--------------------------------------------|-----|
| GitHub         | Branch switching at Branch point                                     | SS/PS/OC/T        | See each              |                | <a href="#">ODE</a>                        |     |
|                | Branch switching at PD point                                         | SS/PS/OC/T        | See each              |                | <a href="#">ODE</a>                        |     |
|                | Continuation of Fold points                                          | SS/PS/OC/T        | See each              | AbstractVector | <a href="#">ODE</a><br><a href="#">PDE</a> | Yes |
|                | Continuation of Period-doubling points                               | SS/OC             |                       | AbstractVector | <a href="#">ODE</a>                        |     |
|                | Continuation of Neimark-Sacker points                                | SS/OC             |                       | AbstractVector | <a href="#">ODE</a>                        |     |
|                | detection of codim 2 bifurcations of periodic orbits                 | SS/OC             |                       | AbstractVector | <a href="#">ODE</a>                        |     |
|                | Branch switching at Bautin point to curve of Fold of periodic orbits | SS/OC             |                       | AbstractVector | <a href="#">ODE</a>                        |     |
|                | Branch switching at ZH/HH point to curve of NS of periodic orbits    | SS/OC             |                       | AbstractVector | <a href="#">ODE</a>                        |     |

## Capabilities related to Homoclinic orbits

These are available through the plugin [HclinicBifurcationKit.jl](#). Please see the [specific docs](#) for more information.

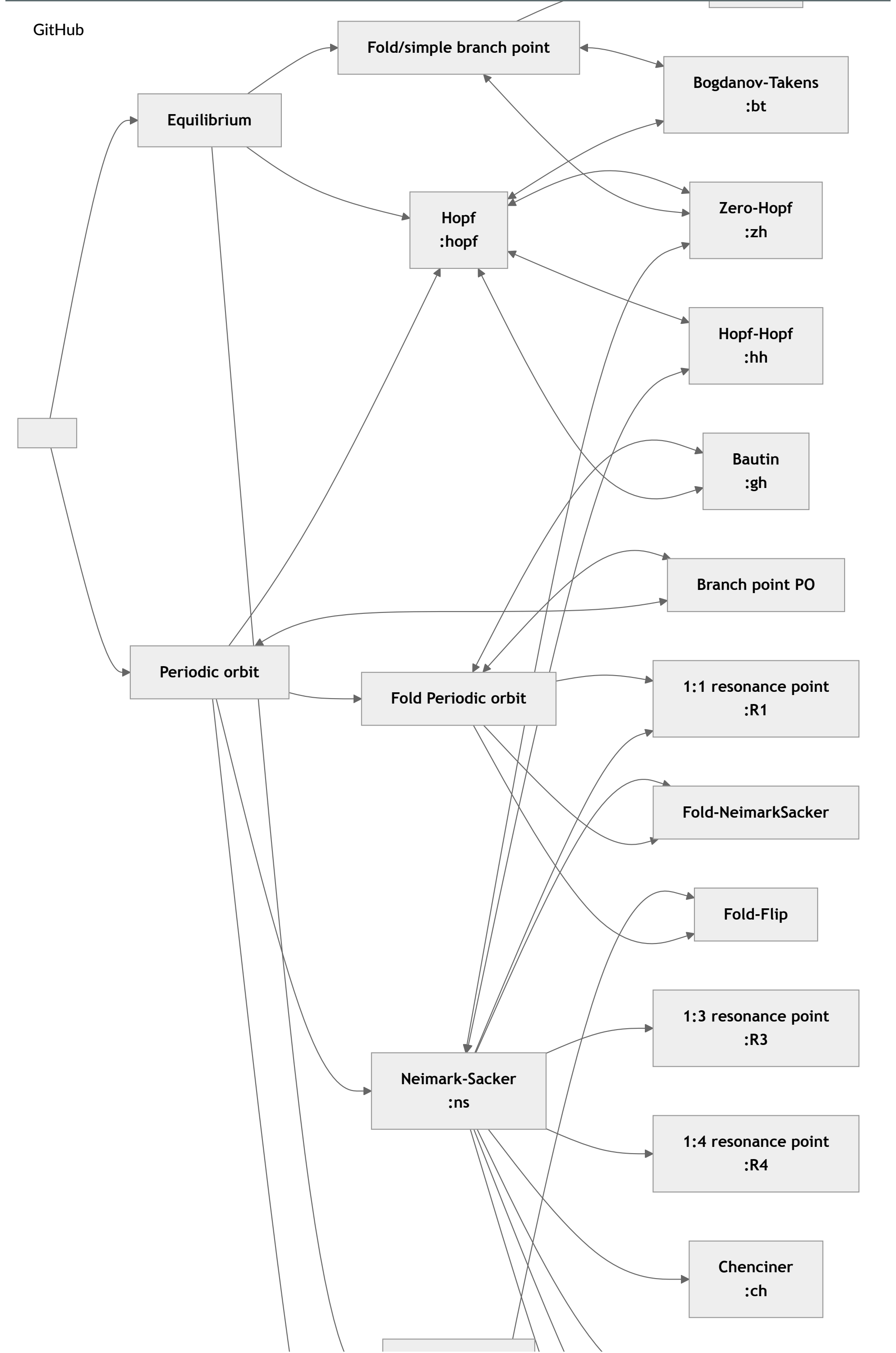
- compute Homoclinic to Hyperbolic Saddle Orbits (HomHS) using Orthogonal collocation or Standard shooting
- compute bifurcation of HomHS
- start HomHS from a direct simulation
- automatic branch switching to HomHS from Bogdanov-Takes bifurcation point

## List of detected bifurcations

A **left-to-right** arrow in the following graph from  $E_1$  to  $E_2$  means that  $E_2$  can be detected when continuing an object of type  $E_1$ .

A **right-to-left** arrow from  $E_2$  to  $E_1$  means that we can start the computation of object of type  $E_1$  from  $E_2$ .

Each object of codim 0 (resp. 1) can be continued with 1 (resp. 2) parameters.





GitHub

Period Doubling  
:pd

1:2 resonance point  
:R2

PDNS

Generalized period  
doubling  
:gpd

Codim0

Codim1

Codim2

« 🏠 Home

🚀 Getting Started with BifurcationKit »

Powered by [Documenter.jl](#) and the [Julia Programming Language](#).

🚀

# Get Started with BifurcationKit

- 🚀 Get Started with BifurcationKit
  - Example 1: solving the perturbed pitchfork equation
    - Step 1: Defining a problem
    - Step 2: Solving a problem
    - Choosing a continuation algorithm
    - Step 3: Analyzing the solution
    - Plotting branches
  - Example 2: simple branching
  - Example 3: continuing periodic orbits
    - Analyzing the solution
    - Plotting the periodic orbit during continuation

This tutorial will introduce you to the functionalities for computing bifurcation diagrams and follow branches of solutions.

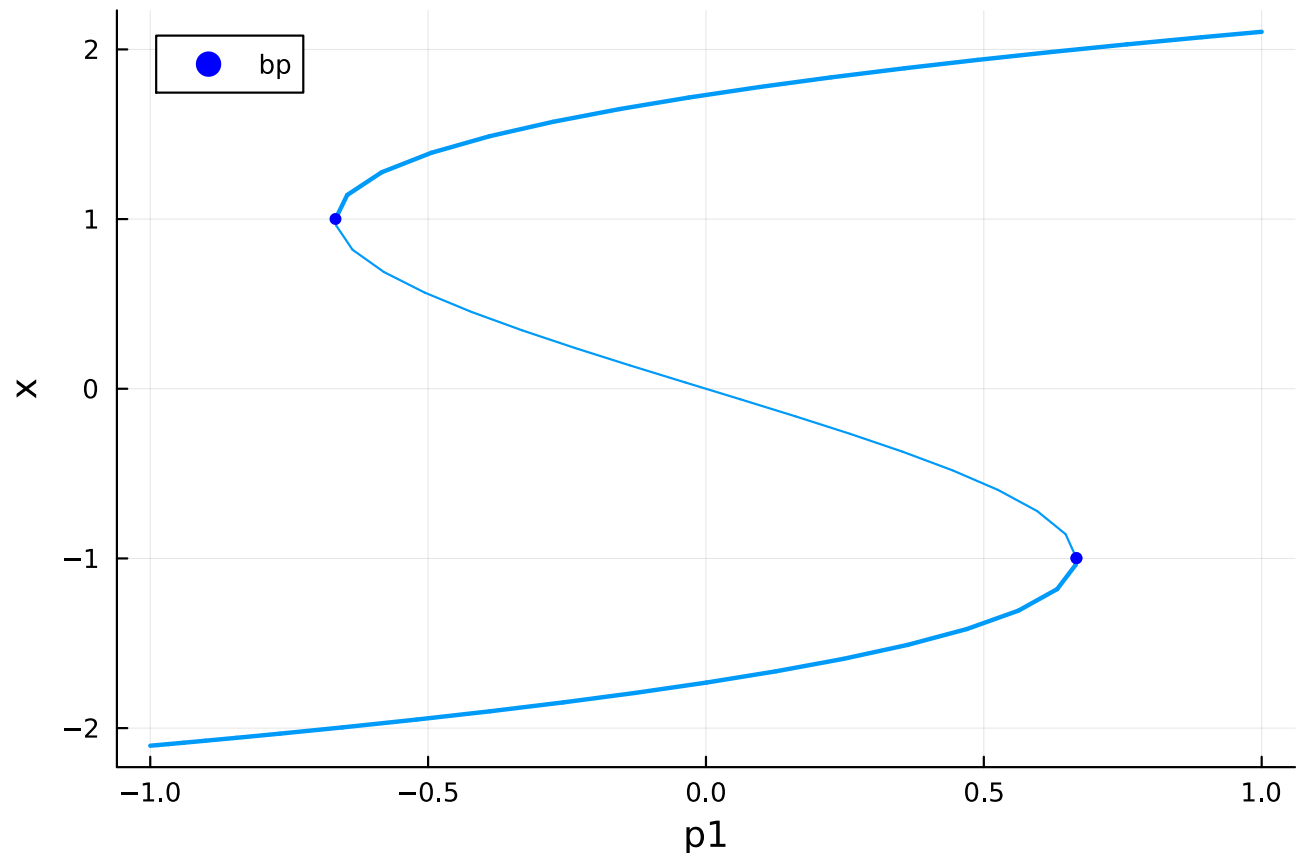
## Example 1: solving the perturbed pitchfork equation

In this example, we will solve the equation

$$\mu + x - \frac{x^3}{3} = 0$$

as function of  $\mu$  by looking at the solutions in the connected component of  $(x_0, \mu_0) \approx (-2, -1)$ . Here  $x \in \mathbb{R}$  is the state variable and  $\mu$  is our parameter. The general workflow is to define a problem, solve the problem, and then analyze the solution. The full code for solving this problem is:

```
using BifurcationKit, Plots
F(x, p) = @. p[1] + x - x^3/3
prob = ODEBifProblem(F, [-2.], [-1.], 1;
    record_from_solution = (x,p; k...) -> x[1])
br = continuation(prob, PALC(), ContinuationPar(p_min = -1., p_max = 1.))
plot(br)
```



## Step 1: Defining a problem

GitHub

To solve this numerically, we define a problem type by giving it the equation, the initial condition, the parameters and the parameter axis to solve over:

```
prob = ODEBifProblem(F,
    [-2.], # initial condition x0
    [-1.], # set of parameters
    1;     # parameter index for continuation
    record_from_solution = (x,p; k...) -> x[1])
```

```
└─ Bifurcation problem with uType Vector{Float64}
└─ Inplace: false
└─ Dimension: 1
└─ Symmetric: false
└─ Parameter: p1
```

Note that BifurcationKit.jl will choose the types for the problem based on the types used to define the problem type. For our example, notice that `u0` is a `Vector{Float64}`, and therefore this will solve with the dependent variables being `Vector{Float64}`. You can use this to choose to solve with `Float32` for example to run this on the GPU (see [example](#)).

You can customize a few scalar indicators for each step (for example if you don't want to save all solutions) by providing a function `record_from_solution`. You can also control how the solution is plotted during a continuation run by providing a function `plot_solution`. This is especially useful when studying PDE for example.

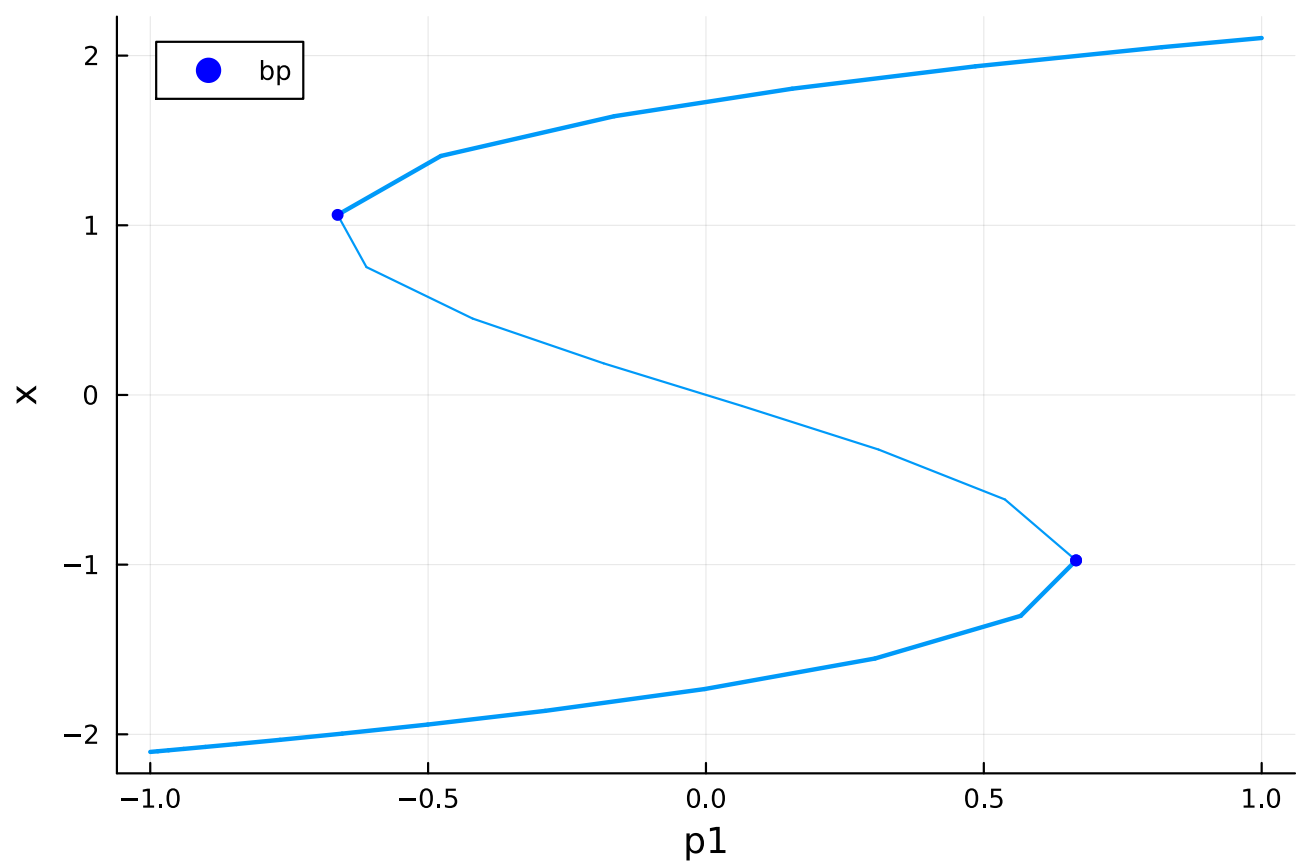
## Step 2: Solving a problem

After defining a problem, you "solve" it using continuation.

```
br = continuation(prob, PALC(), ContinuationPar(p_min = -1., p_max = 1.))
```

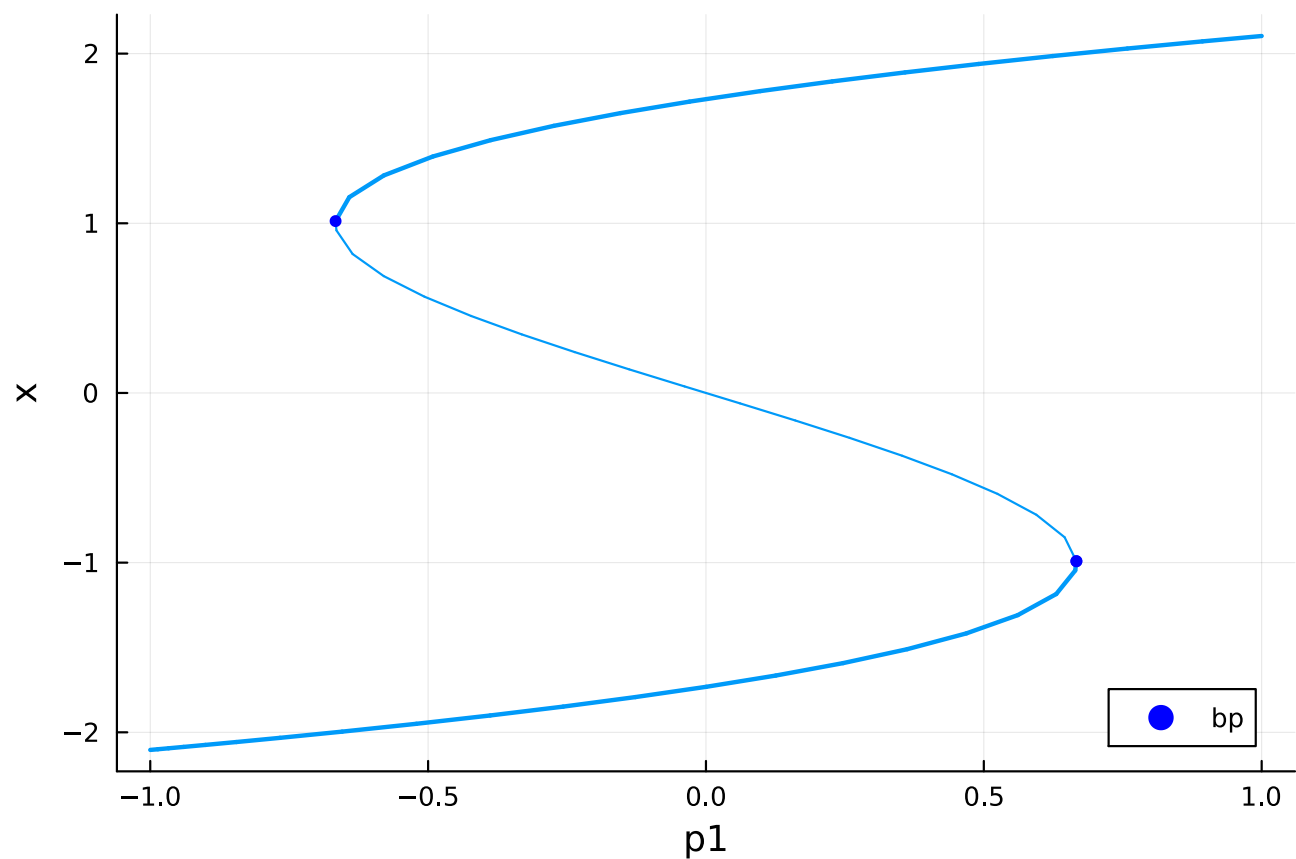
The solvers can be controlled using the available options `ContinuationPar`. For example, we can increase the maximum continuation step (in order to get a less points) by using the command `dsmax = 0.25`

```
using Plots
opts = ContinuationPar(p_min = -1., p_max = 1., dsmax = 0.25, max_steps = 1000)
br = continuation(prob, PALC(), opts)
scene = plot(br)
```



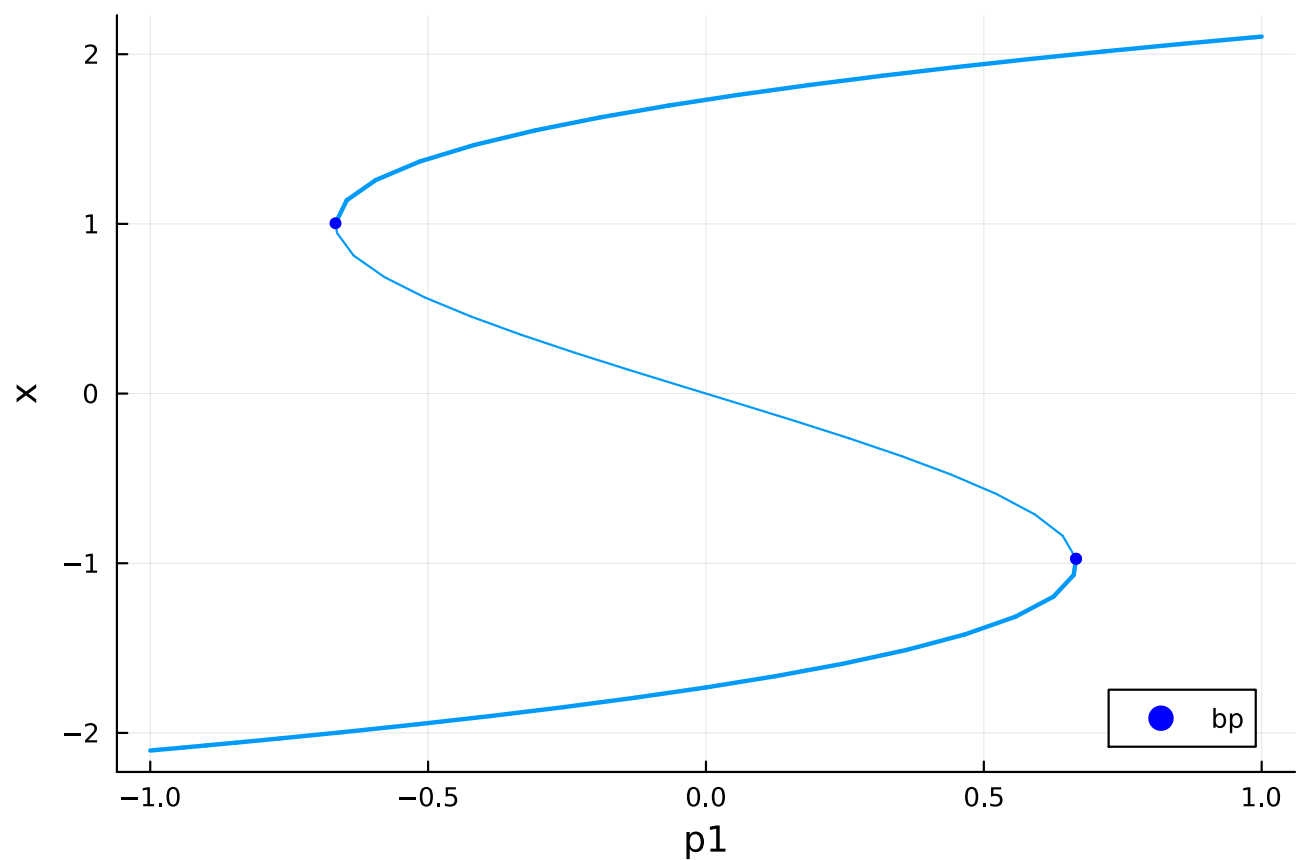
BifurcationKit.jl offers a much wider variety of continuation algorithms than traditional continuation softwares. Many of these algorithms are from recent research and have their own strengths and weaknesses. Each algorithm comes with a doc string, for example [PALC](#). You can also chose a different tangent predictor in PALC

```
opts = ContinuationPar(p_min = -1., p_max = 1.)
br = continuation(prob, PALC(tangent = Bordered()), opts)
scene = plot(br)
```



or you can use the Moore-Penrose continuation algorithm

```
opts = ContinuationPar(p_min = -1., p_max = 1.)
br = continuation(prob, MoorePenrose(), opts)
scene = plot(br)
```



### Step 3: Analyzing the solution

The result of continuation is a solution object. A summary of the result is provided by the `show` method:

```
show(br) # this is equivalent to the REPL julia> br
```

```
└─ Curve type: EquilibriumCont
└─ Number of points: 55
```

```
neg. real part (the [0,0] entry)
Special points:
GitHub
```

- # 1, bp at  $p_1 \approx +0.66597167 \in (+0.66597167, +0.66658738)$ ,  $|\delta p|=6e-04$ , [converged],  $\delta = (1, 0)$
- # 2, bp at  $p_1 \approx -0.66665277 \in (-0.66666619, -0.66665277)$ ,  $|\delta p|=1e-05$ , [converged],  $\delta = (-1, 0)$
- # 3, endpoint at  $p_1 \approx +1.00000000$ ,

From there, you can see that the branch has 55 points, the algorithm is also recalled because it can be modified internally. This summary shows that two bifurcation points where detected. At each such point, the couple  $\delta$  indicates how many real/complex eigenvalues crossed the imaginary axis. This is useful for debugging or when non generic bifurcations are encountered.

We can access the 5th value of the branch with:

```
br[5]
```

```
(x = -2.074261568428699, param = -0.9006174451280271, itnewton = 2, itlinear = 0, ds = 0.0410264312024309)
```

The solution contains many other fields:

```
propertynames(br)
```

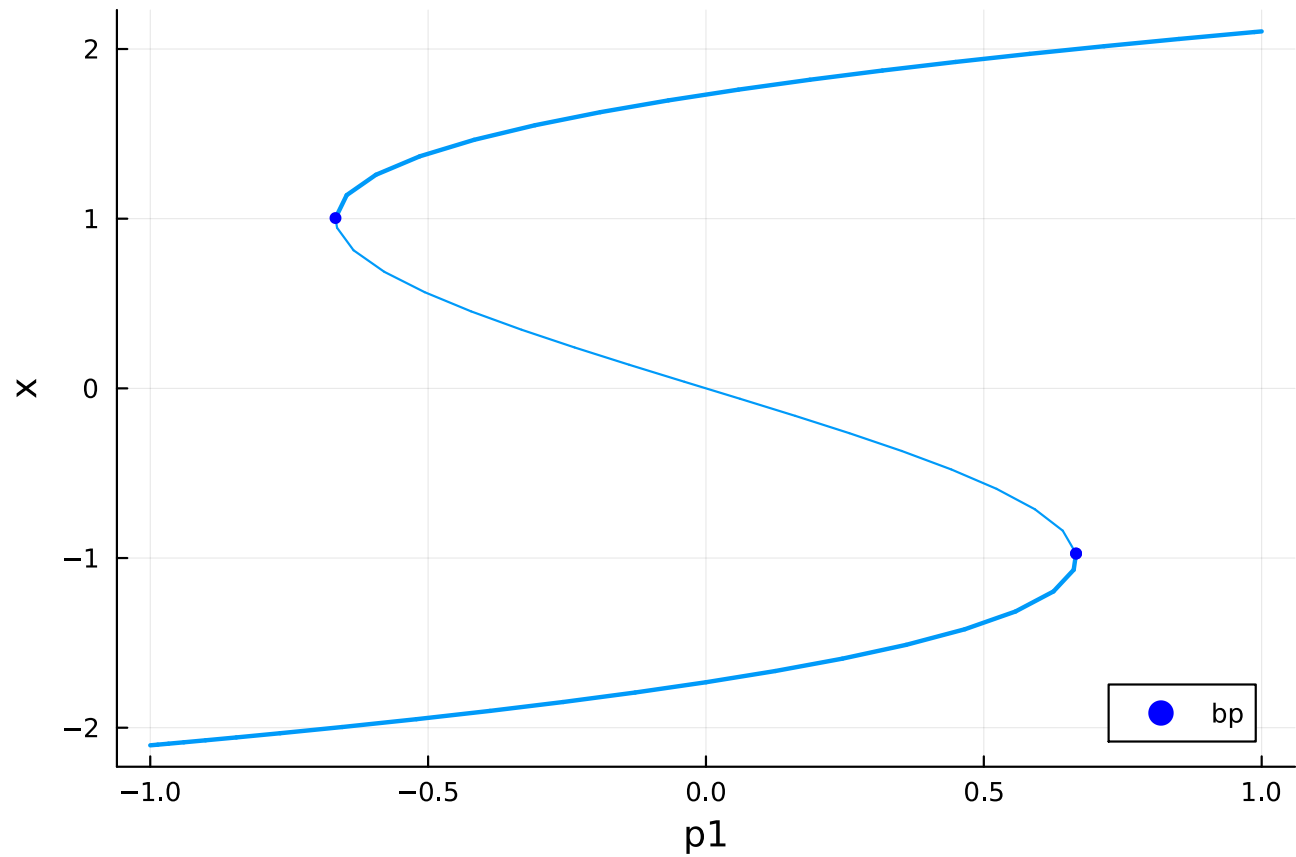
```
(:branch, :eig, :sol, :contparams, :kind, :prob, :specialpoint, :alg)
```

Hence, the eigenelements are saved in `br.eig`, the solutions are saved in `br.sol` and the bifurcation points in `br.specialpoint`.

## Plotting branches

While one can directly plot solution time points using the tools given above, convenience commands are defined by recipes for Plots.jl. To plot the solution object, simply call `plot`:

```
#]add Plots # You need to install Plots.jl before your first time using it!
using Plots
#plotly() # You can optionally choose a plotting backend
plot(br)
```



## Example 2: simple branching

$$0 = x \cdot (\mu - x)$$

### GitHub

as function of  $\mu$ . Here  $u \in \mathbb{R}$  is the state variable and  $\mu$  is our parameter.

In our example, we know by calculus that the solutions to this equation are  $u_0(\mu) = 0$  and  $u_1(\mu) = \mu$  but we will use BifurcationKit.jl to solve this problem numerically, which is essential for problems where a symbolic solution is not known.

In case we know there are many branches, the best is to use an automatic method to compute them all. We will focus on bifurcationdiagram which computes the connected component of the initial guess in the plane  $(x, \mu)$ . An alternative is to use [Deflated Continuation](#).

We define a problem type by giving it the equation, the initial condition, the parameters and the parameter axis to solve over:

```
using Plots
using BifurcationKit

Fbp(u, p) = @. u * (p[1] - u)

# bifurcation problem
prob = ODEBifProblem(Fbp, [0.0], [-0.2],
    # specify the continuation parameter or its index
    1,
    record_from_solution = (x, p; k...) -> x[1])
```

```
└─ Bifurcation problem with uType Vector{Float64}
└─ Inplace: false
└─ Dimension: 1
└─ Symmetric: false
└─ Parameter: p1
```

We then aim at calling bifurcationdiagram which will do the jobs of computing recursively the branches which are connected together. Compared to continuation, bifurcationdiagram requires the maximal level of recursion (in this case 2 because there are 2 branches) and a function providing the continuation parameters for each branch (which may differ from branch to branch if the user decides). This explains the following code:

```
# options for continuation
opts_br = ContinuationPar(
    # parameter interval
    p_max = 0.2, p_min = -0.2,
    # detect bifurcations with bisection method
    # we increase the precision of the bisection
    n_inversion = 4)

# automatic bifurcation diagram computation
diagram = bifurcationdiagram(prob, PALC(),
    # very important parameter. This specifies the maximum amount of recursion
    # when computing the bifurcation diagram. It means we allow computing branches of branches
    # at most in the present case.
    2,
    opts_br,
)
```

```
[Bifurcation diagram]
└─ From 0-th bifurcation point.
└─ Number of children: 2
└─ Root (recursion level 1)
    └─ Curve type: EquilibriumCont
    └─ Number of points: 9
    └─ Type of vectors: Vector{Float64}
    └─ Parameter p1 starts at -0.2, ends at 0.2
    └─ Algo: PALC [Secant]
```



## BifurcationKit

## MultiParamContinuation

DDEBifurcationKit

HclinicBifurcationKit

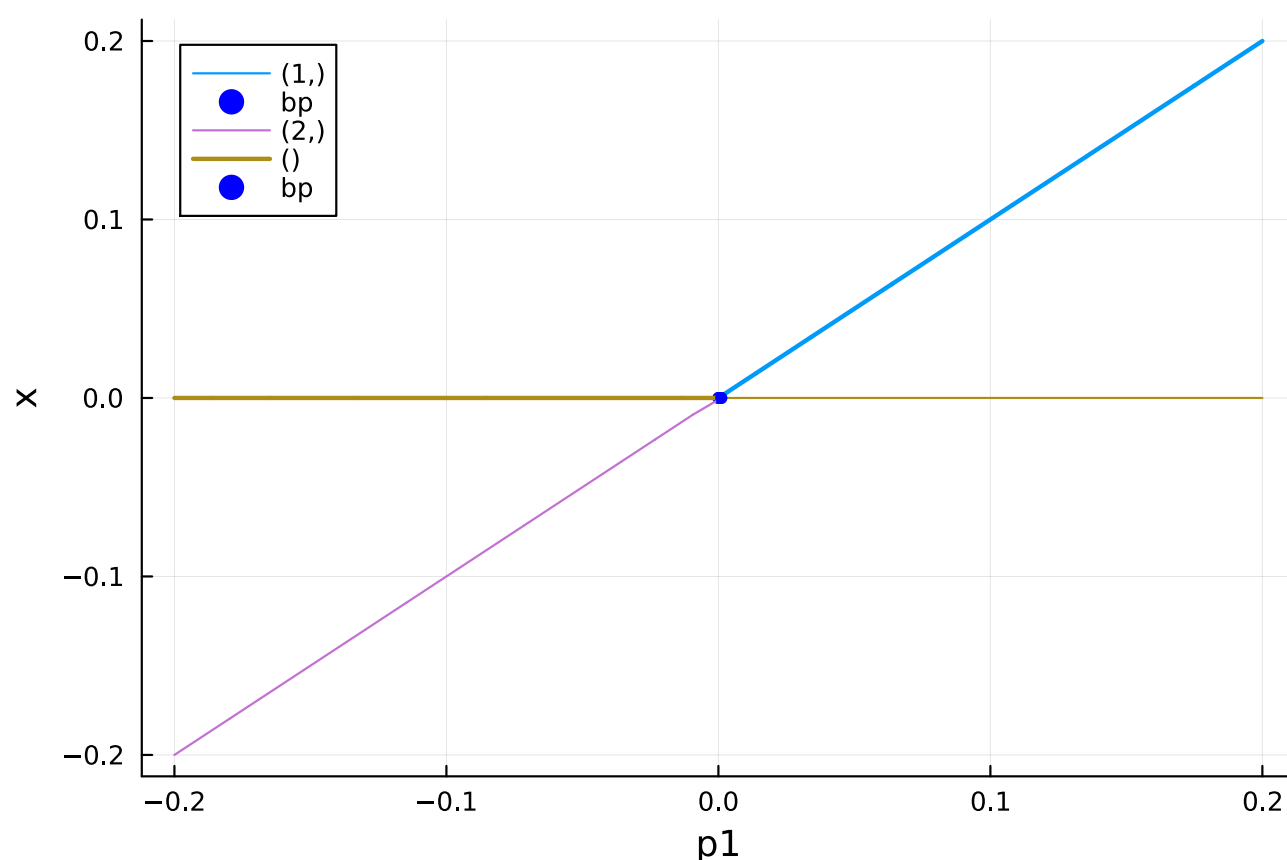
## Tutorials

## Citing

```
- #. 2, endpoint at p1 ≈ +0.20000000,
```

You can plot the diagram like

```
plot(diagram)
```



### Example 3: continuing periodic orbits

In this example, we will compute periodic orbits of the Stuart-Landau oscillator:

$$\begin{aligned}\frac{du}{dt} &= ru - \nu v - (u^2 + v^2)(c_3 u - \mu v) \\ \frac{dv}{dt} &= rv + \nu u - (u^2 + v^2)(c_3 v + \mu u).\end{aligned}$$

The ODE is easily written with a function:

```
using BifurcationKit, Plots

function Fsl(X, p)
    (;r, μ, ν, c3) = p
    u, v = X
    ua = u^2 + v^2
    [
        r * u - ν * v - ua * (c3 * u - μ * v)
        r * v + ν * u - ua * (c3 * v + μ * u)
    ]
end
```

Fsl (generic function with 1 method)

and then we can use this to define a bifurcation problem:

```
par_sl = (r = 0.1, μ = 0., ν = 1.0, c3 = 1.0)
u0 = zeros(2)
prob = ODEBifProblem(Fsl, u0, par_sl, (@optic _ .r))
```



```
└─ Dimension: 2
└─ Metric: false
└─ Parameter: r
```

For this simple problem, we detect the existence of periodic orbits by locating a Hopf bifurcation. This is done as in the previous example by continuing the zero solution:

```
br = continuation(prob, PALC(), ContinuationPar(), bothside = true)
```

```
└─ Curve type: EquilibriumCont
└─ Number of points: 25
└─ Type of vectors: Vector{Float64}
└─ Parameter r starts at -1.0, ends at 1.0
└─ Algo: PALC [Secant]
└─ Special points:

- # 1, endpoint at r ≈ -1.00000000,
- # 2, hopf at r ≈ -0.00595553 ∈ (-0.00595553, +0.00299379), |δp|=9e-03, [converged], δ = ( 2, 2),
- # 3, endpoint at r ≈ +1.00000000,
```

In the result above, we see that a Hopf bifurcation has been detected:

```
- # 2, hopf at r ≈ -0.00595553 ∈ (-0.00595553,...
```

We compute the branch of periodic orbits which is nearby. We thus provide the branch `br`, the index of the special point we want to branch from: 2 in this case and a method `Collocation(20, 5)` to compute periodic orbits. You can look at [Periodic orbits computation](#) for a list of all methods. Suffice it to say that `Collocation` is the default method in the case of ODEs.

```
br_po = continuation(br, 2, ContinuationPar(),
    Collocation(20, 5)
)
```

```
└─ Curve type: PeriodicOrbitCont from Hopf bifurcation point.
└─ Number of points: 15
└─ Type of vectors: Vector{Float64}
└─ Parameter r starts at 0.004044446825657828, ends at 1.0
└─ Algo: PALC [Secant]
└─ Special points:

- # 1, endpoint at r ≈ +1.00000000,
```

## Analyzing the solution

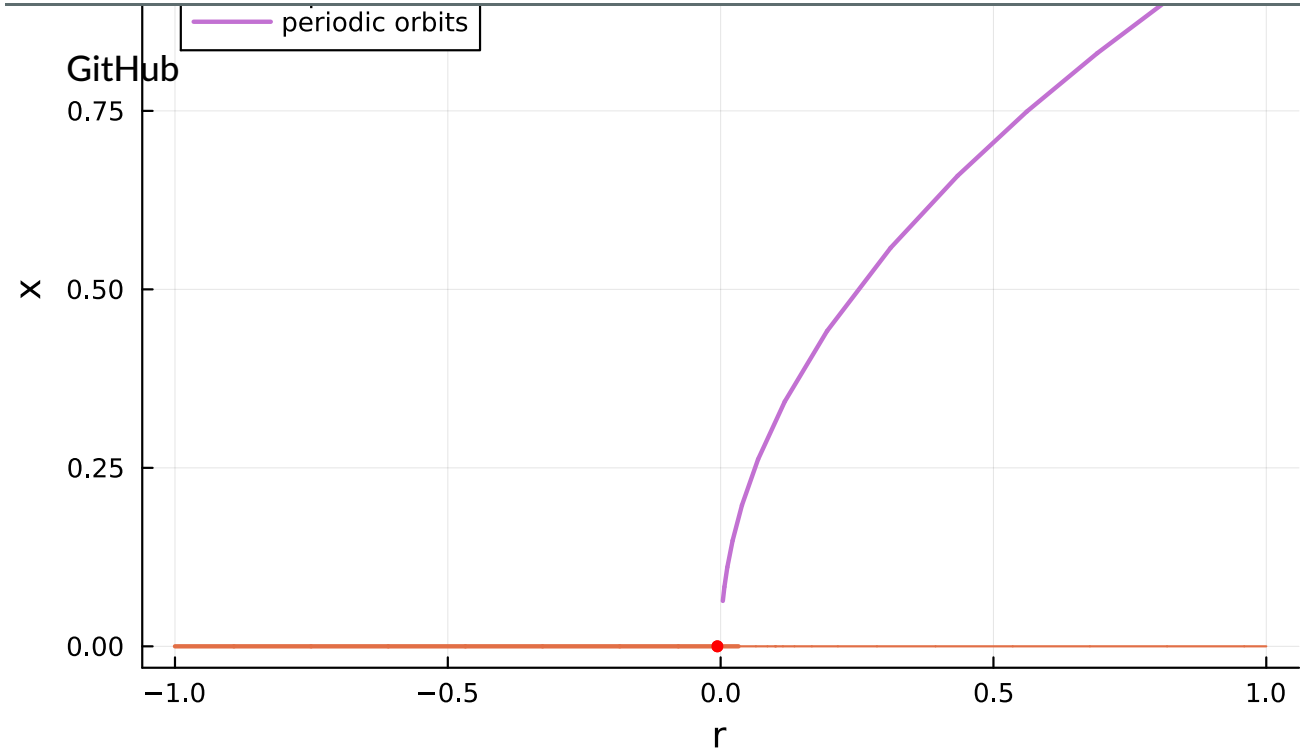
The branch of periodic orbits has been computed. You can look at what is recorded in the first point on the branch:

```
br_po[1]
```

```
(max = 0.06359613398726537, min = -0.06359613398726538, amplitude = 0.12719226797453076, period = 6.283185307179586)
```

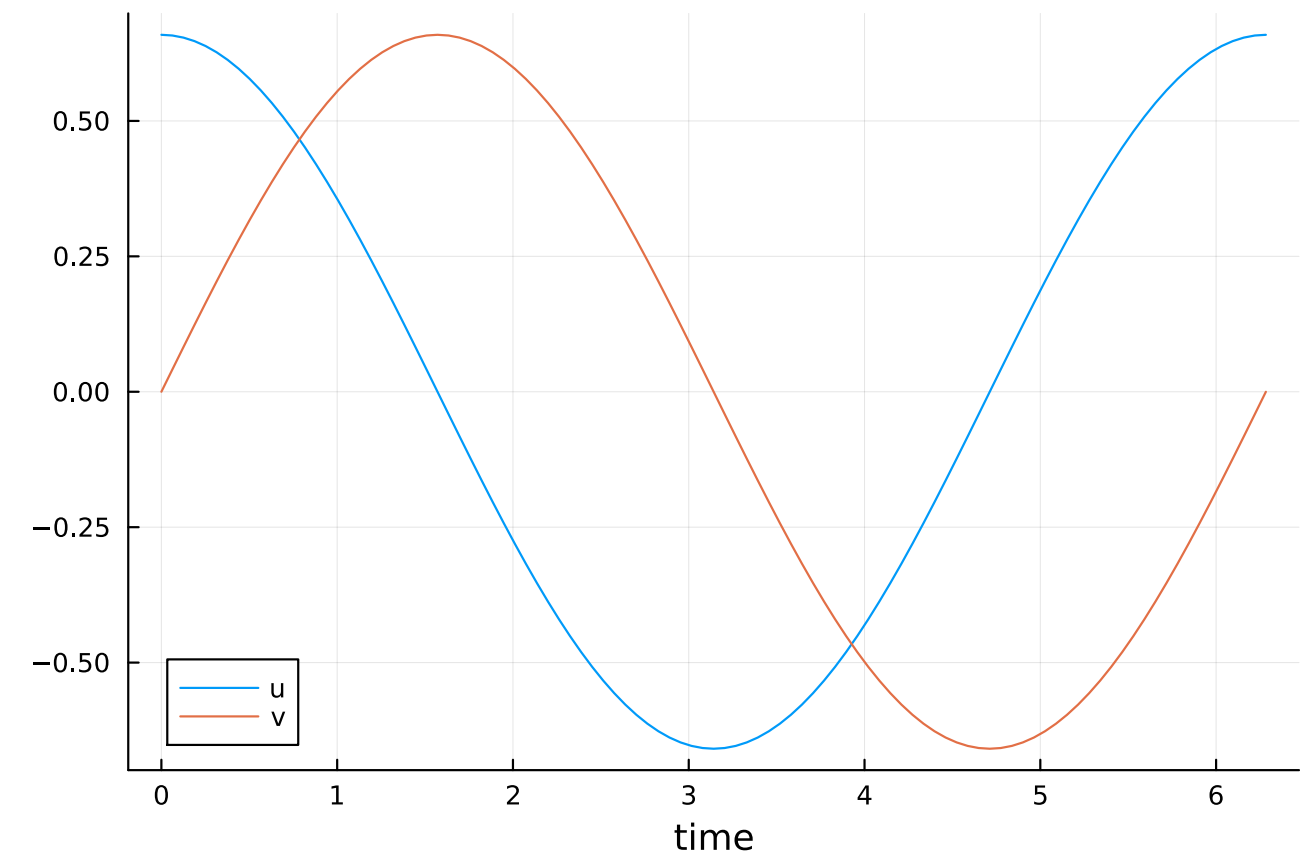
It shows that the maximum/minimum/amplitude/period of the periodic orbit are recorded by default. You can also plot all the branches as follows

```
plot(br, br_po, branchlabel = ["equilibria", "periodic orbits"])
```



Finally, if you are interested in the periodic orbits saved in `br_po`, for example to plot it, the method `get_periodic_orbit` is what you are looking for:

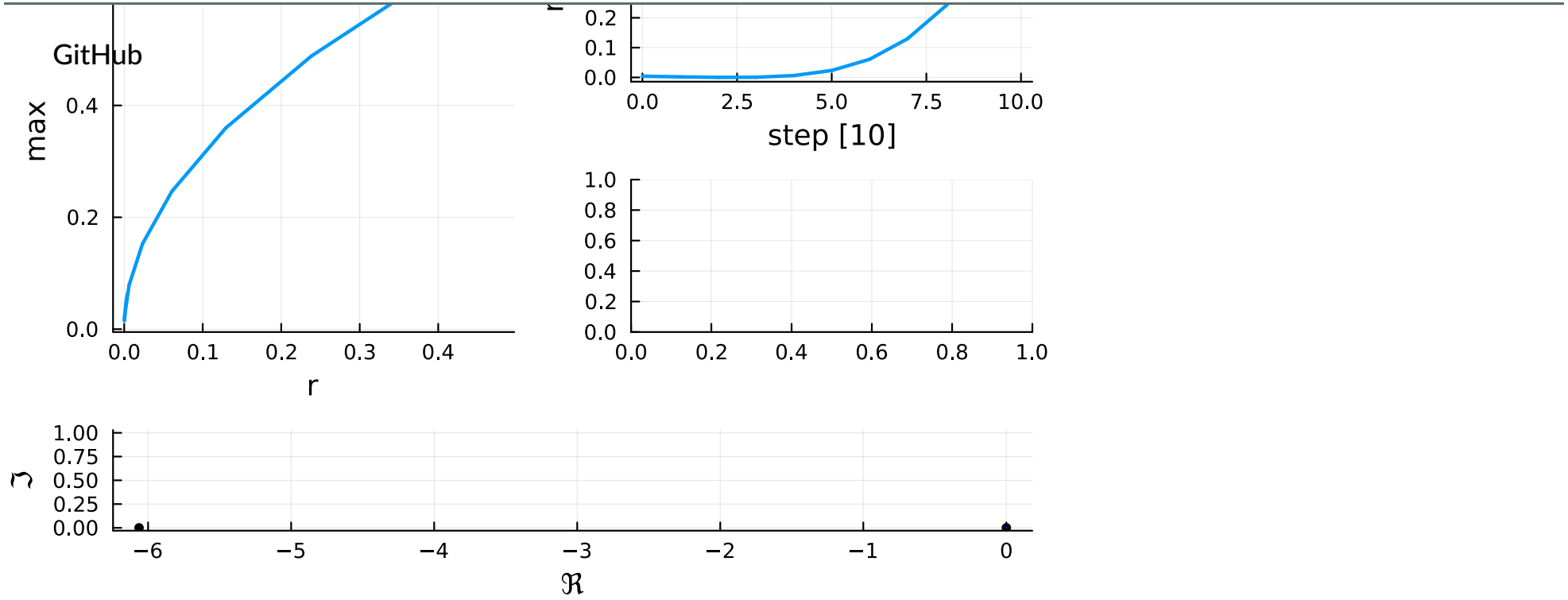
```
sol = get_periodic_orbit(br_po, 10)
plot( sol.t, sol[1,:], label = "u", xlabel = "time")
plot!(sol.t, sol[2,:], label = "v", xlabel = "time")
```



## Plotting the periodic orbit during continuation

If you plot the solution during continuation, you see that the right bottom panel is empty ; this panel is used to plot the solution at the current continuation step:

```
br_po = continuation(br, 2, br.contparams,
    Collocation(20, 5);
    plot = true,
)
```



1.00

0.75

0.50

0.25

0.00

Im

0.00

0.25

0.50

0.75

1.00

-6

-5

-4

-3

-2

-1

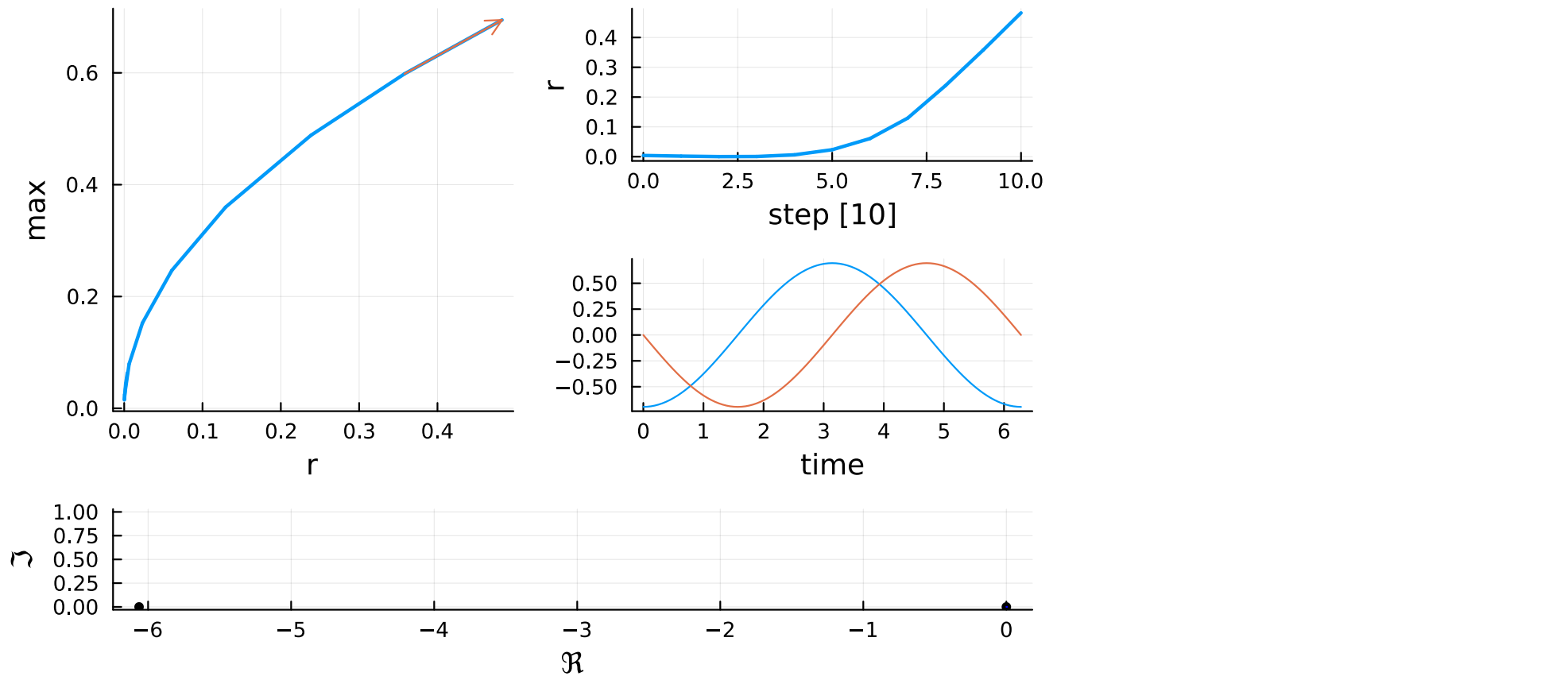
0

 $\Re$ 

(Note that the bottom panel is a plot of the eigenvalues of the jacobian in the complex plane at the current continuation step. )

In order to plot the periodic solution during continuation, you need to supply a `periodic_solution` to `continuation`. This is not done by default because in some cases, obtaining the solution is costly (e.g. for Shooting methods). Based on the previous paragraph, it is straightforward to implement this plotting function:

```
br_po = continuation(br, 2, br.contparams,
    Collocation(20, 5);
    plot = true,
    plot_solution = (x, par; k...) -> begin
        # par is a Named tuple which contains
        # the problem for computing periodic orbits
        # and the value of the parameter at the current step
        sol = get_periodic_orbit(par.prob, x, par.p)
        plot!(sol.t, sol.u'; xlabel = "time", label="", k...)
    end
)
```



0.50

0.25

0.00

-0.25

-0.50

0

1

2

3

4

5

6

time

1.00

0.75

0.50

0.25

0.00

Im

0.00

0.25

0.50

0.75

1.00

-6

-5

-4

-3

-2

-1

0

 $\Re$

# Periodic predator-prey model

- Periodic predator-prey model
  - Simulation of the model
  - Utility functions
  - Periodic orbits with trapezoid method
  - Periodic orbits with parallel standard shooting
  - Periodic orbits with orthogonal collocation
  - Continuation of Fold / PD of periodic orbits with Collocation
  - References

This example is found in the MatCont ecosystem.

The following is a periodically forced predator-prey model studied in [\[Kuznetsov\]](#) using shooting techniques.

$$\begin{cases} \dot{x} = r \left( 1 - \frac{x}{K} \right) x - \frac{axy}{b_0(1 + \varepsilon u) + x} \\ \dot{y} = e \frac{axy}{b_0(1 + \varepsilon u) + x} - dy \\ \dot{u} = u - 2\pi v - (u^2 + v^2) u \\ \dot{v} = 2\pi u + v - (u^2 + v^2) v \end{cases} \tag{E}$$

This tutorial is useful in that we show how to start periodic orbits continuation from solutions obtained from solving ODEs. We are interested in cases where  $(u, v) \neq (0, 0)$  for which we have only periodic solutions. Thus, we cannot rely on branching from equilibria to find these periodic orbits.

```
using Revise, Plots
import BifurcationKit as BK
import BifurcationKit: @optic, @set, @reset

function Pop!(du, X, p, t = 0)
    (;r,K,a,ε,b0,e,d) = p
    x, y, u, v = X
    p = a * x / (b0 * (1 + ε * u) + x)
    du[1] = r * (1 - x/K) * x - p * y
    du[2] = e * p * y - d * y
    s = u^2 + v^2
    du[3] = u-2pi * v - s * u
    du[4] = 2pi * u + v - s * v
    du
end

par_pop = ( K = 1., r = 6.28, a = 12.56, b0 = 0.25, e = 1., d = 6.28, ε = 0.2, )

z0 = [0.1,0.1,1,0]

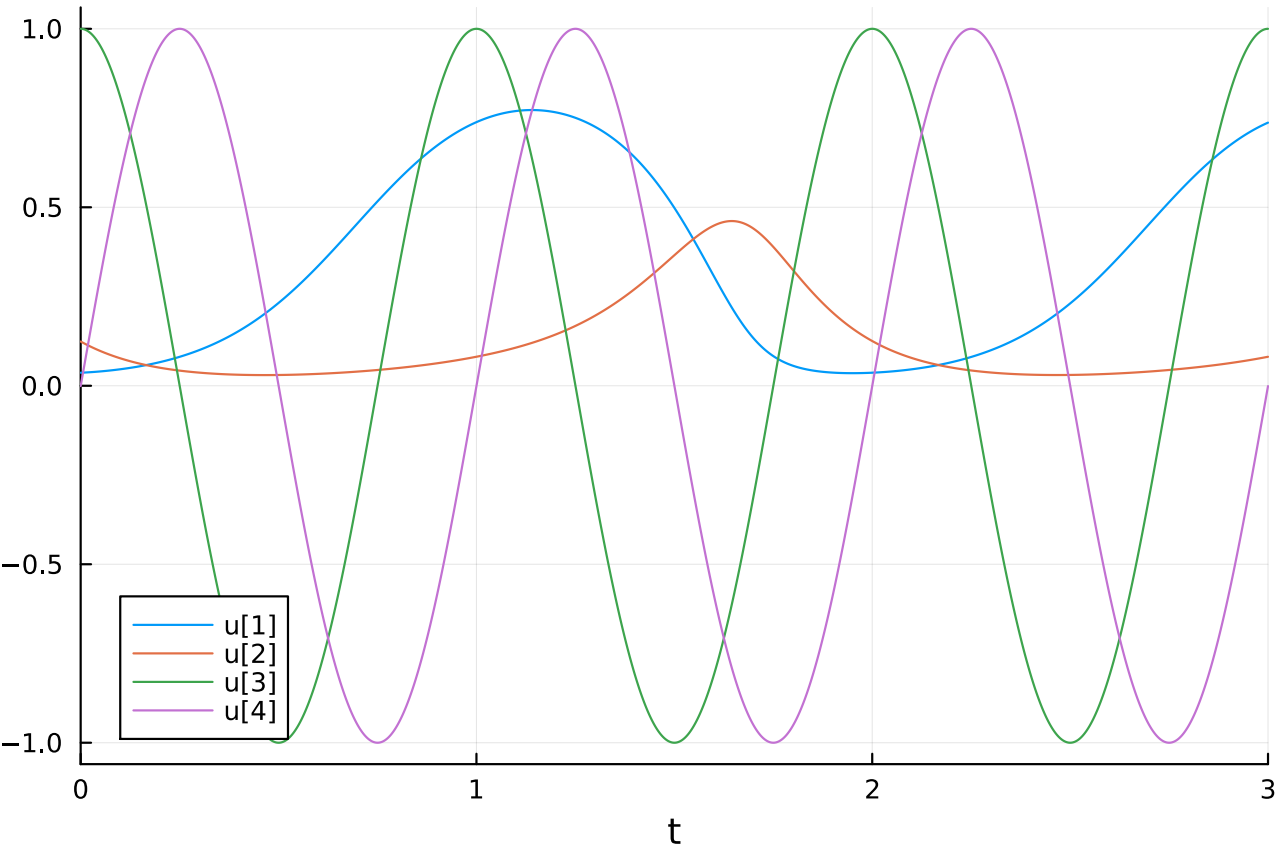
prob_bif = BK.ODEBifProblem(Pop!, z0, par_pop, (@optic _ .b0);
    record_from_solution = (x, p; k...) -> (x = x[1], y = x[2], u = x[3]))

opts_br = BK.ContinuationPar(p_min = 0., p_max = 20.0, ds = 0.002, dsmax = 0.01, n_inversion = 6, nev = 4)
```

This is very straightforward thanks to SciML.

GitHub

```
import OrdinaryDiffEq as DE
prob_de = DE.ODEProblem(Pop!, z0, (0,200.), par_pop)
sol = DE.solve(prob_de, DE.Vern9())
prob_de = DE.ODEProblem(Pop!, sol.u[end], (0,3.), par_pop, reltol = 1e-8, abstol = 1e-10)
sol = DE.solve(prob_de, DE.Vern9())
plot(sol)
```



## Utility functions

We start with two helper functions that record and plot the periodic orbits. The following works for shooting, collocation and trapezoid methods for computing periodic orbits.

```
function record_from_solution(x, p; k...)
    xtt = BK.get_periodic_orbit(p.prob, x, p.p)
    return (max = maximum(xtt[1,:]),
            min = minimum(xtt[1,:]),
            period = BK.getperiod(p.prob, x, p.p))
end

function plot_solution(x, p; k...)
    xtt = BK.get_periodic_orbit(p.prob, x, p.p)
    plot!(xtt.t, xtt[1,:]; label = "x", k...)
    plot!(xtt.t, xtt[2,:]; label = "y", k...)
    # plot!(br; subplot = 1, putspecialptlegend = false)
end

argspo = (;record_from_solution,
           plot_solution)
```

```
(record_from_solution = Main.record_from_solution, plot_solution = Main.plot_solution)
```

## Periodic orbits with trapezoid method

We compute periodic orbits of (E) using the Trapezoid method. We are now equipped to build a periodic orbit problem from a solution `sol::ODEProblem`.

```
# function to build probtrap from sol
probtrap, ci = BK.generate_ci_problem(BK.Trapeze(M = 150),
```

optspocont = BK.ContinuationPar(optsbr, maxsteps = 50, tolstability = 1e-3)

brpo\_fold = BK.continuation(probtrap, ci, BK.PALC(), opts\_po\_cont;  
verbosity = 3, plot = true,  
argspo...  
)

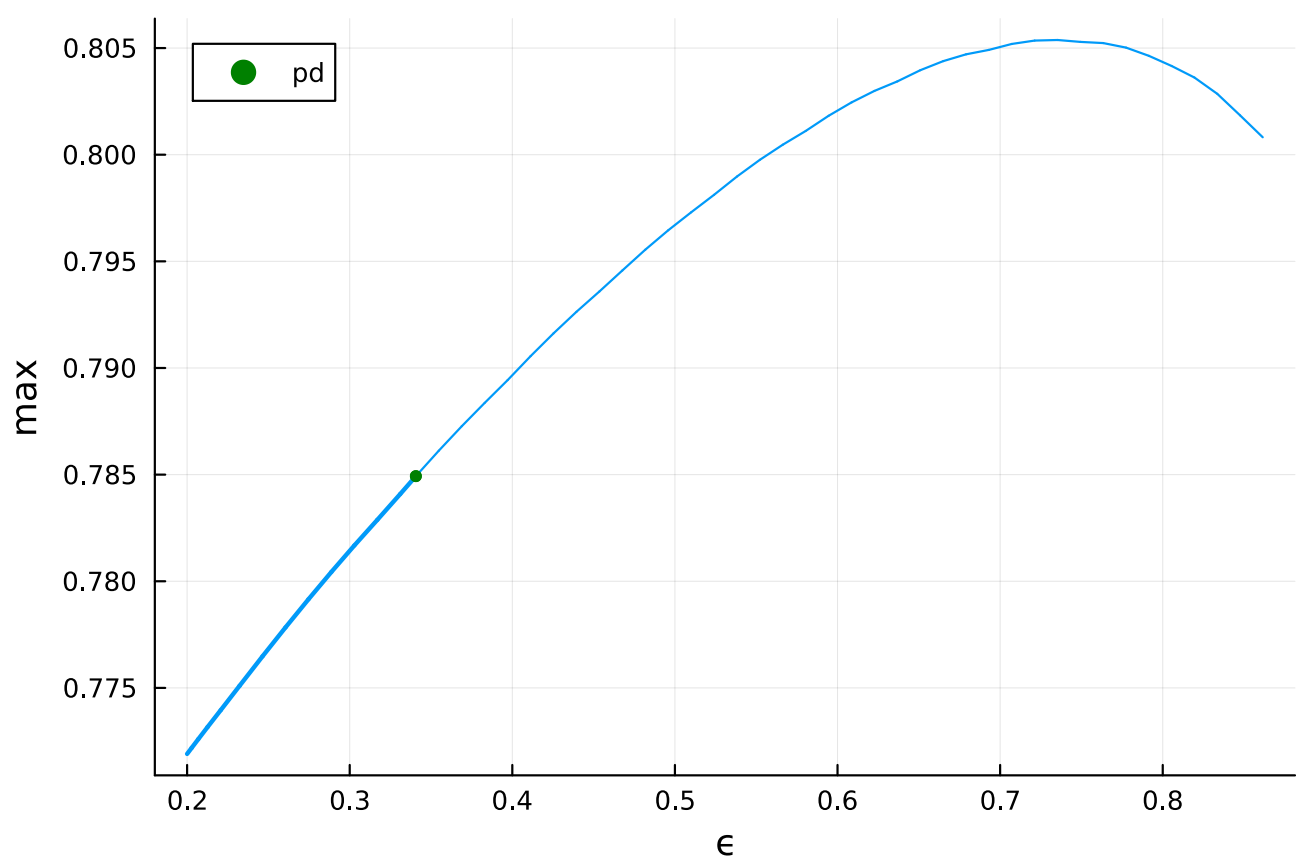
scene = plot(brpo\_fold)

We continue w.r.t. to  $\epsilon$  and find a period-doubling bifurcation.

prob2 = (@set probtrap.prob\_vf.lens = (@optic \_. $\epsilon$ ))

brpo\_pd = BK.continuation(prob2, ci, BK.PALC(), opts\_po\_cont;  
argspo...  
)

scene = plot(brpo\_pd)



## Periodic orbits with parallel standard shooting

We are now ready to build a periodic orbit problem from a solution `sol::ODEProblem`.

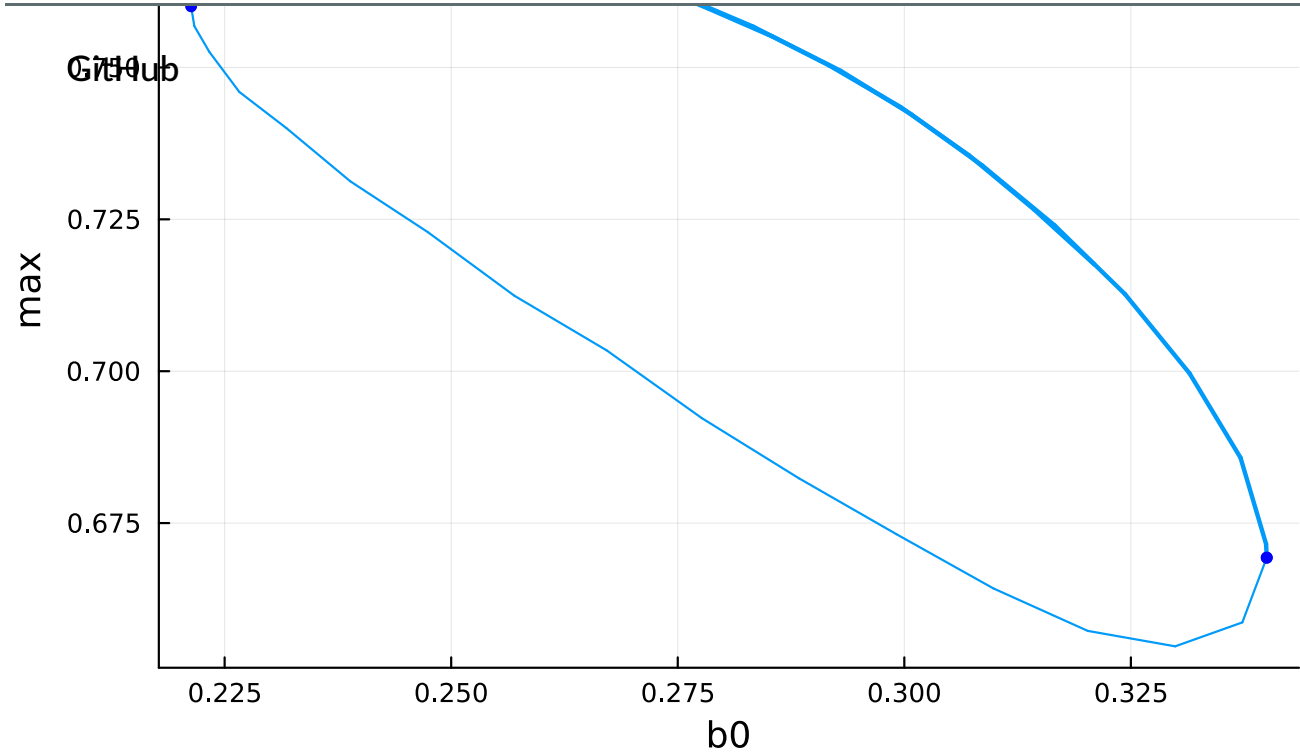
probsh, cish = BK.generate\_ci\_problem( BK.Shooting(M = 3),  
prob\_bif, prob\_de, sol, 2.; alg = DE.Vern9(), abstol = 1e-12, reltol = 1e-10)

opts\_po\_cont = BK.ContinuationPar(opts\_br, max\_steps = 50, tol\_stability = 1e-3)

br\_fold\_sh = BK.continuation(probsh, cish, BK.PALC(tangent = BK.Bordered()), opts\_po\_cont;  
argspo...  
)

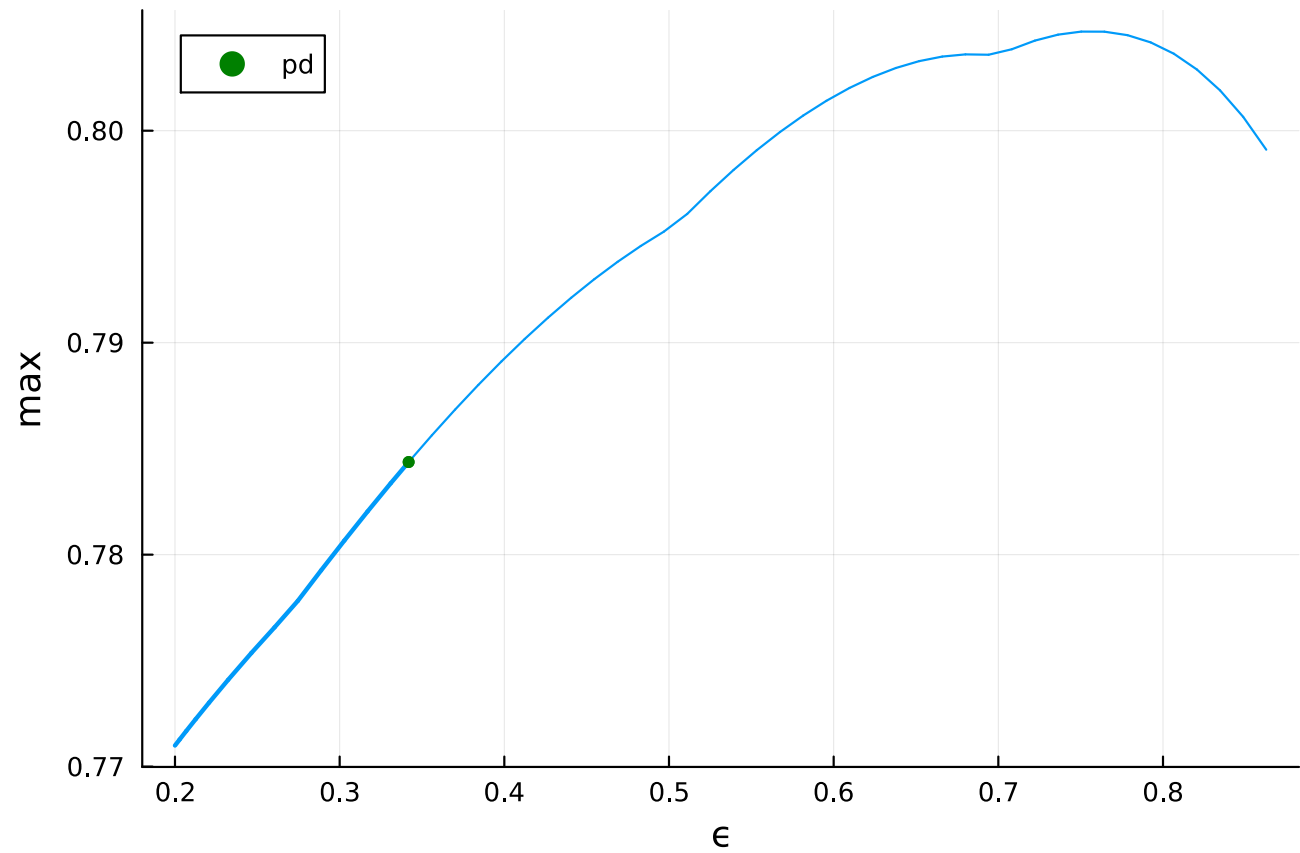
scene = plot(br\_fold\_sh)





We continue w.r.t. to  $\epsilon$  and find a period-doubling bifurcation.

```
probsh2 = (@set probsh.lens = (@optic _.\epsilon))
brpo_pd_sh = BK.continuation(probsh2, cish, BK.PALC(), opts_po_cont;
    argspo...
)
scene = plot(brpo_pd_sh)
```

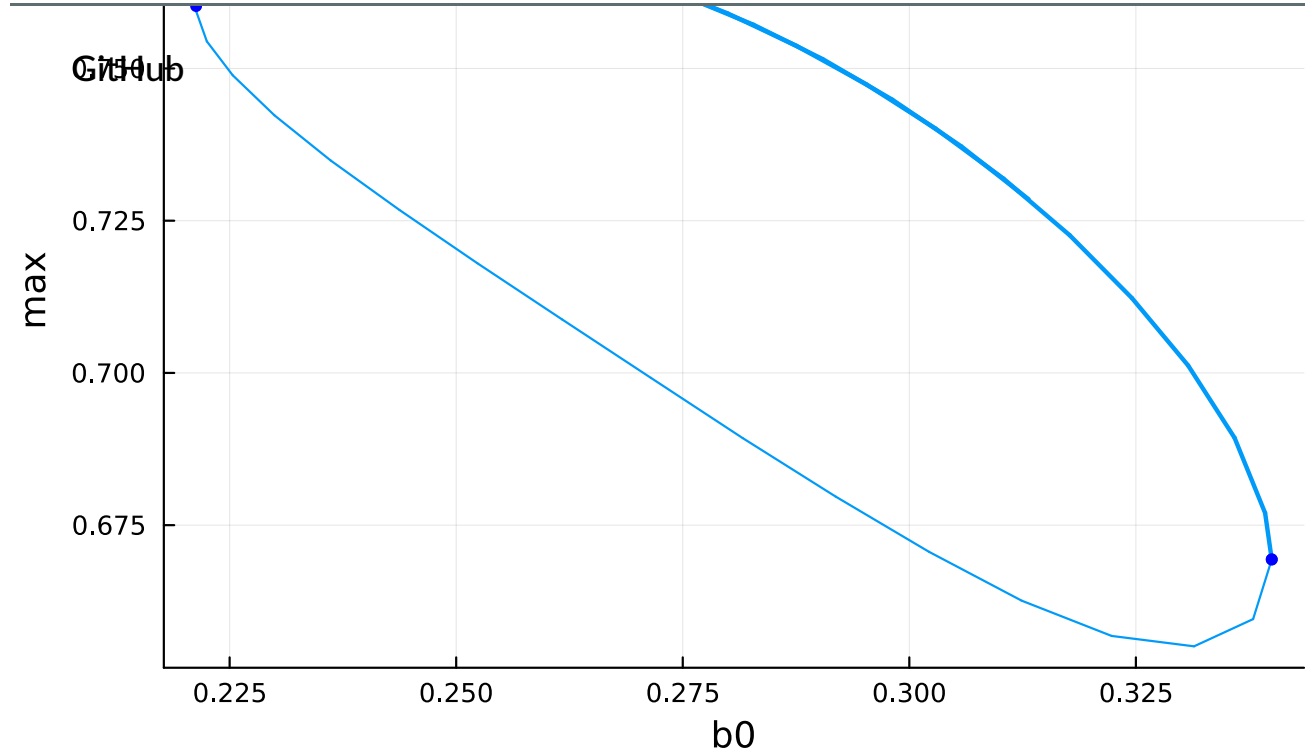


## Periodic orbits with orthogonal collocation

We do the same as in the previous section but using orthogonal collocation. This is the most reliable and precise method for ODE. When the dimension of the ODE is large, it becomes prohibitive.

```
# this is the function which builds probcoll from sol
probcoll, ci = BK.generate_ci_problem(BK.Collocation(30, 4),
    prob_bif, sol, 2)

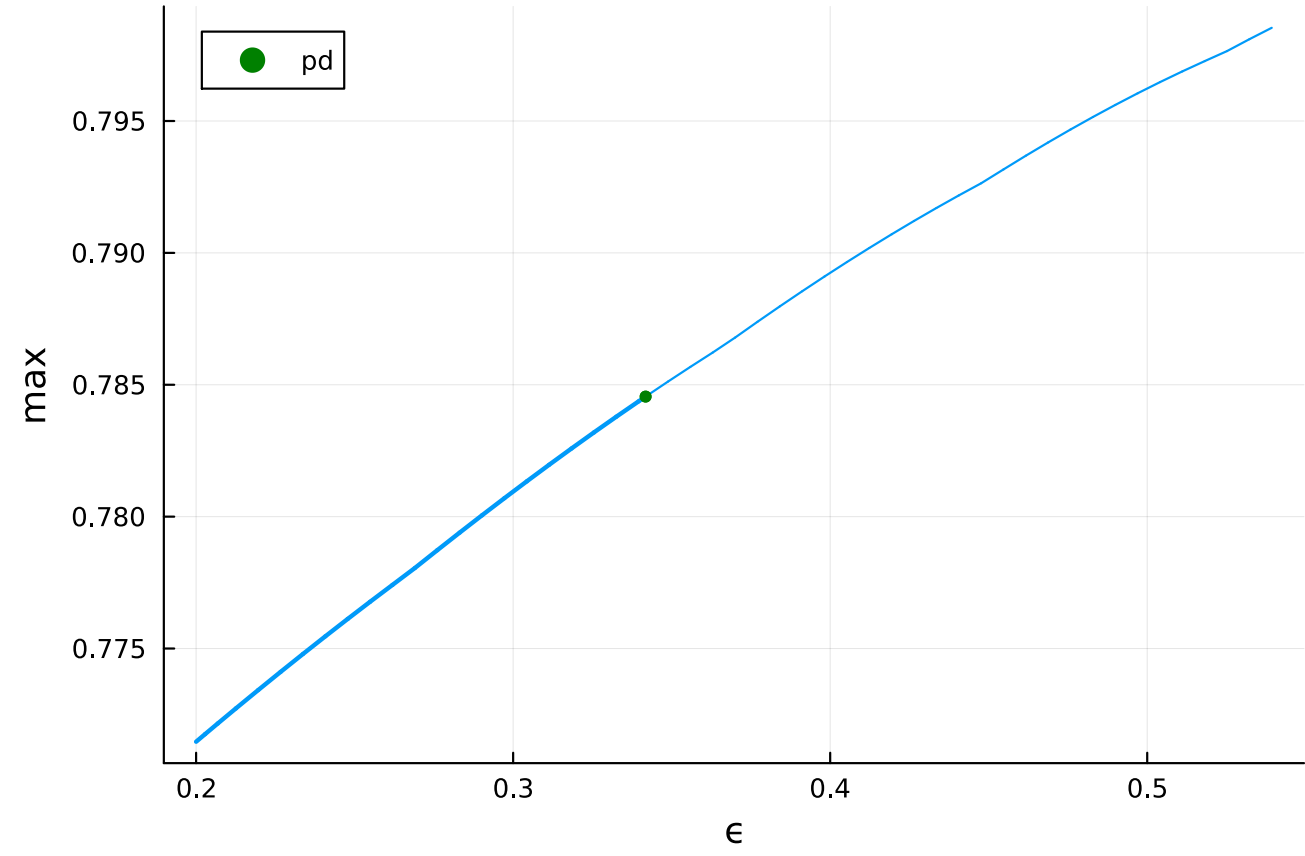
opts_po_cont = BK.ContinuationPar(opts_br, max_steps = 50, tol_stability = 1e-8)
brpo_fold = BK.continuation(probcoll, ci, BK.PALC(), opts_po_cont;
    argspo...
)
scene = plot(brpo_fold)
```



We continue w.r.t. to  $\epsilon$  and find a period-doubling bifurcation.

```
prob2 = (@set probcoll.prob_vf.lens = (@optic _.\epsilon))
brpo_pd = BK.continuation(prob2, ci, BK.PALC(), BK.ContinuationPar(opts_po_cont, dsmax = 5e-3);
    argspo...
)

scene = plot(brpo_pd)
```



## Continuation of Fold / PD of periodic orbits with Collocation

We continue the previously detected fold / period-doubling bifurcations as function of two parameters and detect codim 2 bifurcations. We first start with the computation of the curve of Folds.

```
opts_pocoll_fold = BK.ContinuationPar(brpo_fold.contparams, max_steps = 100, p_min = 0.01, p_max = 1.2)
# we can increase the precision for collocation
@reset opts_pocoll_fold.newton_options.tol = 1e-12

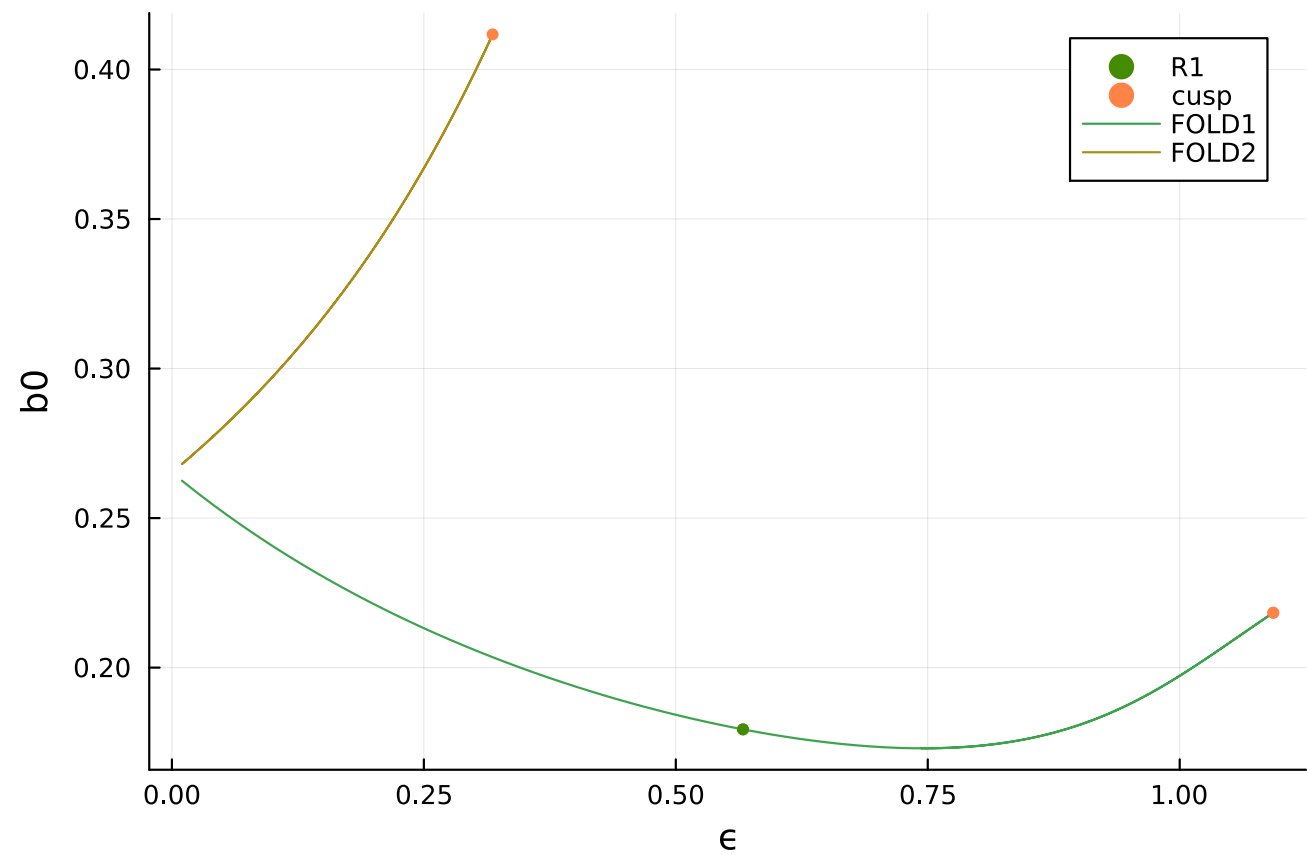
fold_po_coll2 = BK.continuation(brpo_fold, 1, (@optic _.\epsilon), opts_pocoll_fold;
    # verbosity = 2, plot = true,
    detect_codim2_bifurcation = 2,
    jacobian_ma = BK.MinAug(),
    start_with_eigen = false,
    bothside = true,
)
```



GitHub

```
# verbosity = 2, plot = true,
detect_codim2_bifurcation = 2,
jacobian_ma = BK.MinAug(),
start_with_eigen = false,
bothside = true,
usehessian = true,
)

plot(fold_po_coll1, fold_po_coll2, branchlabel = ["FOLD1", "FOLD2"])
```



We turn to the computation of the curve of PD points.

```
par_pop2 = @set par_pop.b0 = 0.45
sol2 = DE.solve(DE remake(prob_de, p = par_pop2, u0 = [0.1, 0.1, 1, 0], tspan=(0, 1000)), DE.Vern9())
sol2 = DE.solve(DE remake(sol2.prob, tspan = (0, 10), u0 = sol2.u[end]), DE.Vern9())
plot(sol2, xlims = (8, 10))

probcoll, ci = BK.generate_ci_problem(BK.Collocation(30, 3), BK.re_make(prob_bif, params = sol2.prob.p),

prob2 = @set probcoll.prob_vf.lens = @optic _.ε
brpo_pd = BK.continuation(prob2, ci, BK.PALC(), BK.ContinuationPar(opts_po_cont, dsmax = 5e-3);
    argspo...,
    bothside = true,
)

opts_pocoll_pd = BK.ContinuationPar(brpo_pd.contparams, max_steps = 40, p_min = 1.e-2, dsmax = 1e-2, ds =
@reset opts_pocoll_pd.newton_options.tol = 1e-12
pd_po_coll2 = BK.continuation(brpo_pd, 2, (@optic _.b0), opts_pocoll_pd;
    # verbosity = 2, plot = true,
    detect_codim2_bifurcation = 1,
    start_with_eigen = false,
    jacobian_ma = BK.MinAug(),
    normC = BK.norminf,
    bothside = true,
)

plot(pd_po_coll2, vars = (:ε, :b0), branchlabel = "PD")
plot!(fold_po_coll1, fold_po_coll2, branchlabel = ["FOLD1", "FOLD2"])
```



- Kuznetsov

Yu.A. Kuznetsov, S. Muratori, and S. Rinaldi. Bifurcations and chaos in a periodic predator-prey model, *Internat. J. Bifur. Chaos Appl. Sci. Engr.*, 2 (1992), 117-128.

# ● Lorenz-84 model, take 2

- Lorenz-84 model, take 2
  - Two parameters curves of Fold / Hopf bifurcation
  - Fold bifurcations of periodic orbits from Bautin bifurcation
  - NS bifurcations of periodic orbits from Hopf-Hopf bifurcation
  - References

In this tutorial, we study the extended Lorenz-84 model which is also treated in MatCont [\[Kuznetsov\]](#). We use this model to showcase the automatic branch switching procedure to

- Fold of periodic orbits from Bautin bifurcation point
- NS of periodic orbits from ZH bifurcation point
- NS of periodic orbits from HH bifurcation point.

As this model has been studied in this [tutorial](#), we do not give much details and refer to the corresponding tutorial to get the 2 parameters curves of Hopf / Fold bifurcations.

The model is as follows

$$\begin{cases} \dot{X} = -Y^2 - Z^2 - \alpha X + \alpha F - \gamma U^2 \\ \dot{Y} = XY - \beta XZ - Y + G \\ \dot{Z} = \beta XY + XZ - Z \\ \dot{U} = -\delta U + \gamma UX + T \end{cases} \tag{E}$$

We recall the problem setting:

```
using Revise, Plots, LinearAlgebra
import BifurcationKit as BK
import BifurcationKit: @optic, @reset

# vector field
function Lor!(out, u, p, t = 0)
    (;α,β,γ,δ,G,F,T) = p
    X,Y,Z,U = u
    out[1] = -Y^2 - Z^2 - α*X + α*F - γ*U^2
    out[2] = X*Y - β*X*Z - Y + G
    out[3] = β*X*Y + X*Z - Z
    out[4] = -δ*U + γ*U*X + T
    out
end

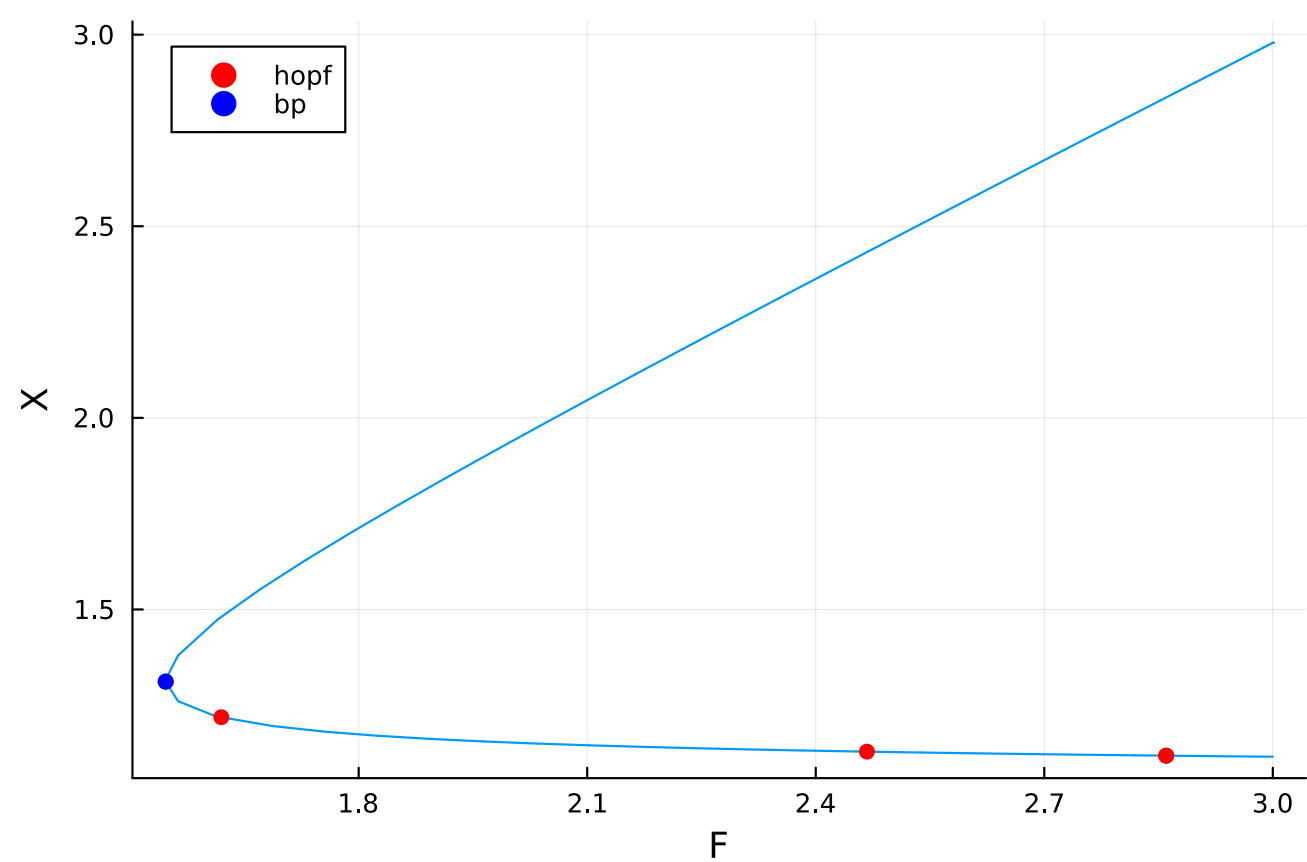
parlor = (α = 1//4, β = 1., G = .25, δ = 1.04, γ = 0.987, F = 1.7620532879639, T = .0001265)

z0 = [2.9787004394953343, -0.03868302503393752, 0.058232737694740085, -0.02105288273117459]

recordFromSolutionLor(x, p; k...) = (u = BK.getvec(x); (X = u[1], Y = u[2], Z = u[3], U = u[4]))
prob = BK.ODEBifProblem(Lor!, z0, parlor, (@optic _ .F);
    record_from_solution = (x, p; k...) -> (X = x[1], Y = x[2], Z = x[3], U = x[4]),)

opts_br = BK.ContinuationPar(p_min = -1.5, p_max = 3.0, ds = 0.002, dsmax = 0.05, n_inversion = 6, max_steps = 1000000, @reset opts_br.newton_options.tol = 1e-12)
br = BK.continuation(BK.re_make(prob, params = (parlor..., T=0.04, F=3.)),
    BK.PALC(), opts_br;
```

GitHub



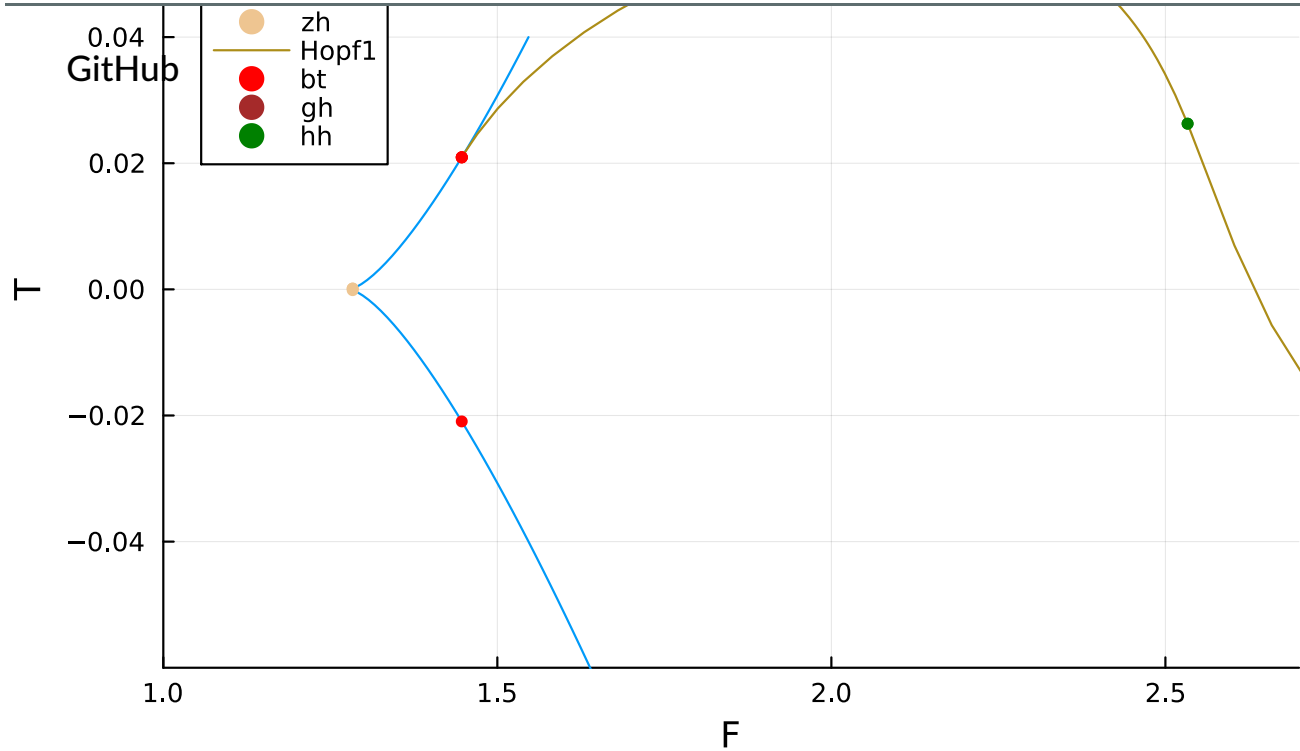
## Two parameters curves of Fold / Hopf bifurcation

We follow the Fold points in the parameter plane  $(T, F)$ . We tell the solver to consider `br.specialpoint[5]` and continue it.

```
sn_codim2 = BK.continuation(br, 5, (@optic _..T), BK.ContinuationPar(opts_br, p_max = 3.2, p_min = -0.1, c
    # plot = true,
    normC = BK.norminf,
    detect_codim2_bifurcation = 2,
    bothside = false,
)

hp_codim2_1 = BK.continuation(br, 3, (@optic _..T), BK.ContinuationPar(opts_br, ds = -0.001, dsmax = 0.02,
    normC = BK.norminf,
    detect_codim2_bifurcation = 2,
    bothside = true,
)

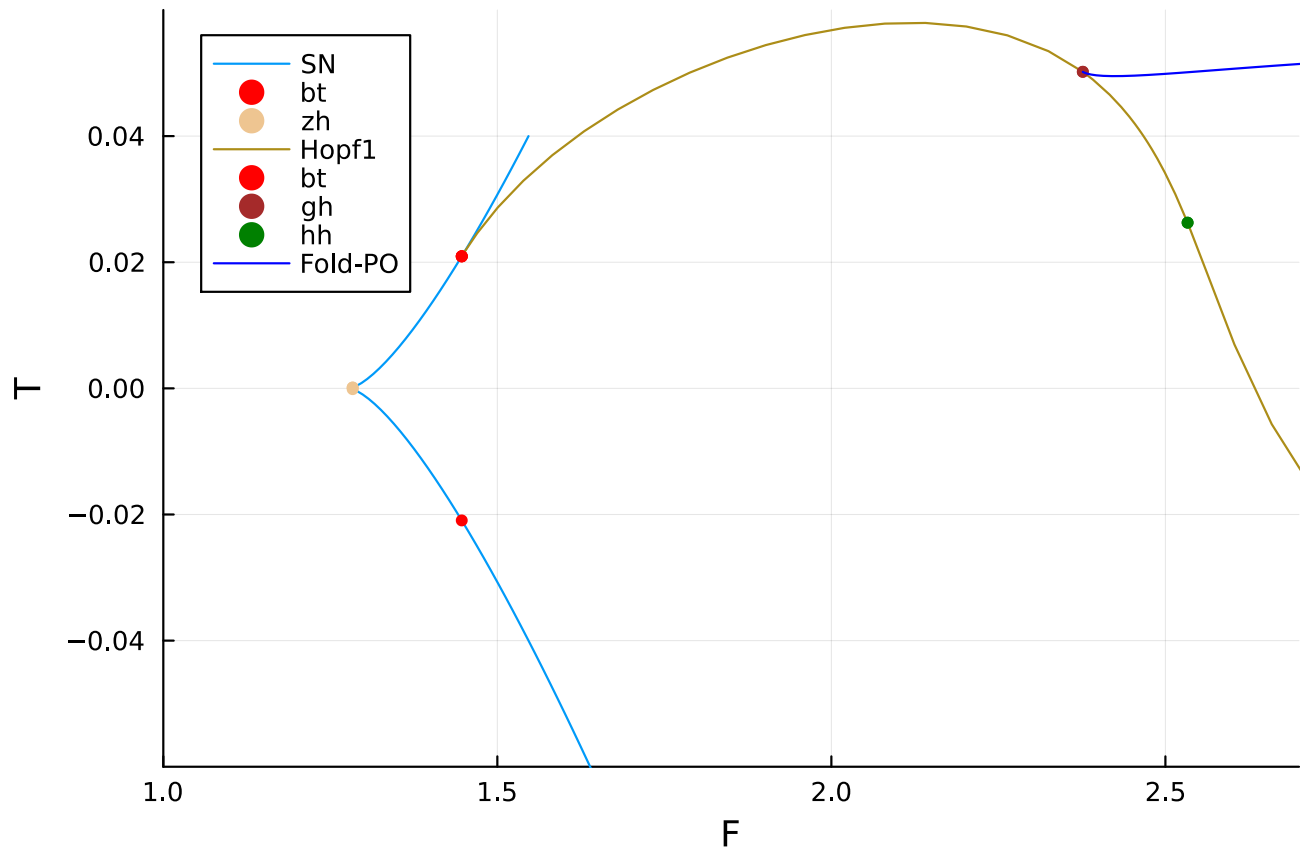
plot(sn_codim2, vars=(:F, :T), branchlabel = "SN")
plot!(hp_codim2_1, vars=(:F, :T), branchlabel = "Hopf1", xlims = (1,2.7), ylims = (-0.06,0.06))
```



## Fold bifurcations of periodic orbits from Bautin bifurcation

We compute the branch of Fold of periodic orbits from the Bautin bifurcation (labelled :gh) in the previous figure. In this tutorial, we focus on orthogonal collocation but standard shooting would do too.

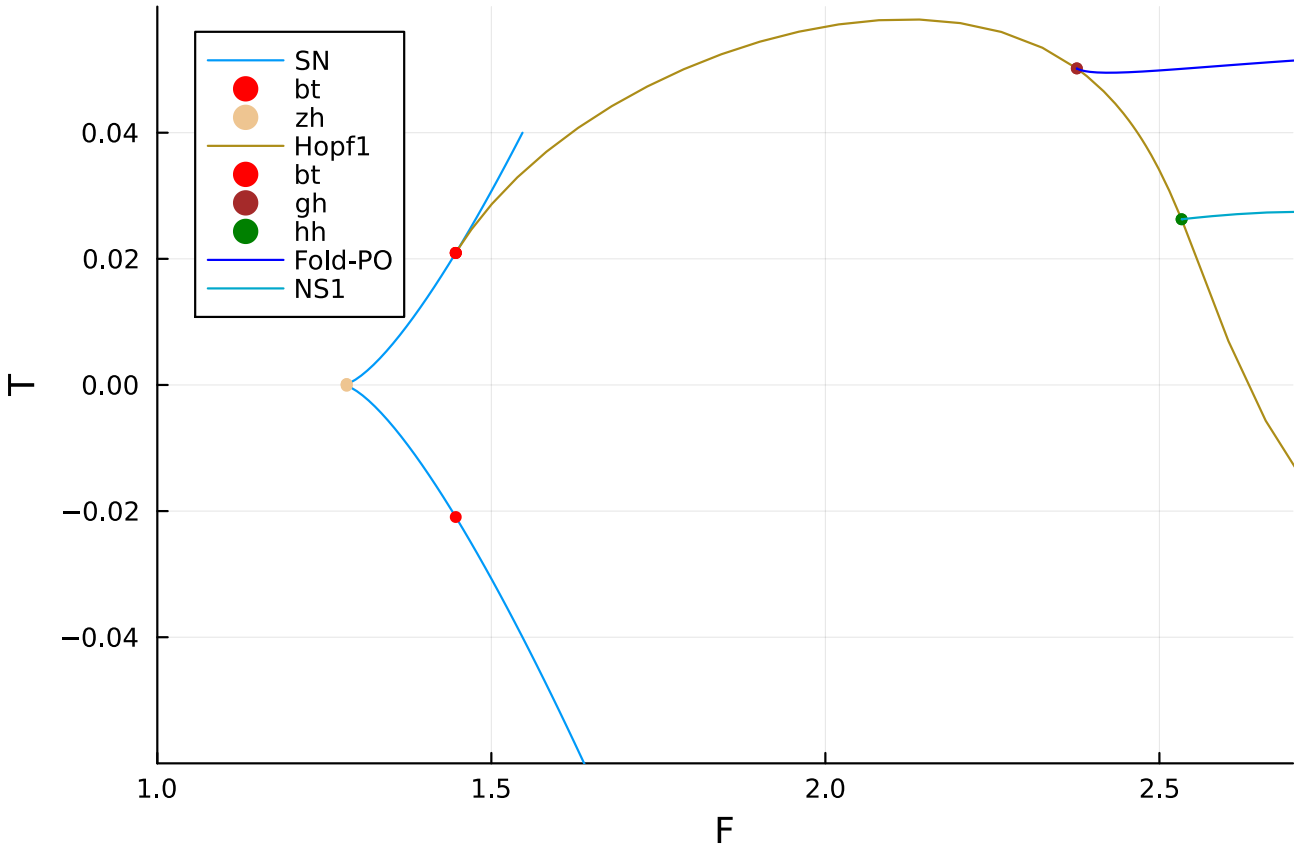
```
opts_fold_po = BK.ContinuationPar(hp_codim2_1.contparams, dsmax = 0.01, detect_bifurcation = 0, max_steps = 10000)
# @reset opts_fold_po.newton_options.verbose = false
@reset opts_fold_po.newton_options.tol = 1e-8
ind_gh = findfirst(x->x.type == :gh, hp_codim2_1.specialpoint)
fold_po = BK.continuation(hp_codim2_1, ind_gh, opts_fold_po,
    BK.Collocation(20, 3, meshadapt = true);
    normC = BK.norminf,
    δp = 0.02,
    jacobian_ma = BK.MinAug(),
    verbosity = 0, plot = false,
)
plot!(fold_po, vars=(:F, :T), branchlabel = "Fold-PO", color=:blue)
```



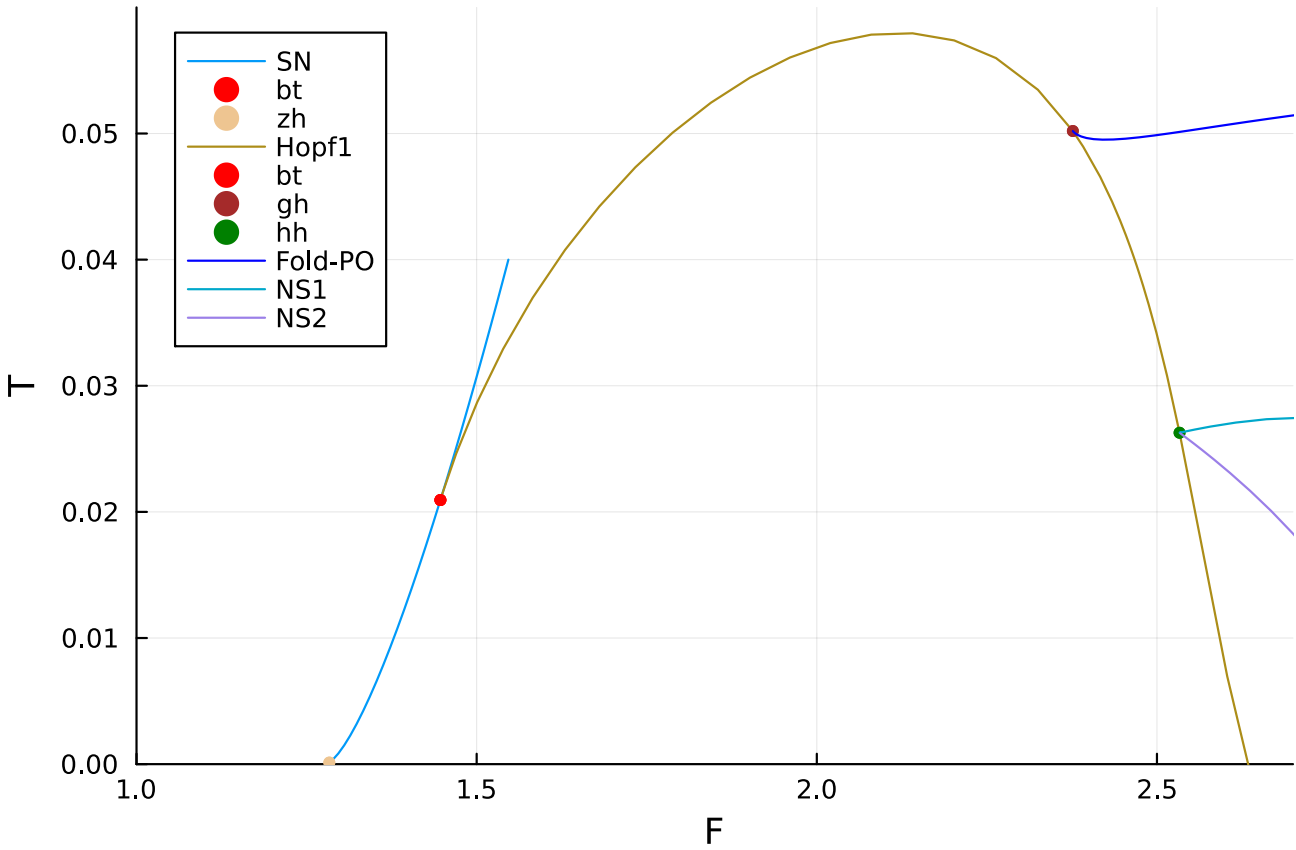
## NS bifurcations of periodic orbits from Hopf-Hopf bifurcation

When we computed the curve of Hopf points, we detected a Hopf-Hopf bifurcation. We can branch from it to get the curve of NS points. This is done as follows:

```
# @reset opts_ns_po.newton_options.tol = 1e-9
@repl{
  opts_ns_po.newton_options.max_iterations = 10
  ind_hh = findfirst(x->x.type == :hh, hp_codim2_1.specialpoint)
  ns_po1 = BK.continuation(hp_codim2_1, ind_hh, opts_ns_po,
    BK.Collocation(20, 3);
    normC = BK.norminf,
    δp = 0.02,
    # which of the 2 NS curves should we compute?
    whichns = 1,
    jacobian_ma = BK.MinAug(),
  )
  plot!(ns_po1, vars=(:F, :T), branchlabel = "NS1")
}
```



```
ns_po2 = BK.continuation(hp_codim2_1, ind_hh, opts_ns_po,
  BK.Collocation(30, 3);
  normC = BK.norminf,
  δp = 0.02,
  # which of the 2 NS curves should we compute?
  whichns = 2,
  jacobian_ma = BK.MinAug(),
)
plot!(ns_po2, vars=(:F, :T), branchlabel = "NS2", ylims = (0, 0.06))
```





- [Kuznetsov](#)  
**GitHub**

Kuznetsov, Yu A., H. G. E. Meijer, W. Govaerts, and B. Sautois. “Switching to Nonhyperbolic Cycles from Codim 2 Bifurcations of Equilibria in ODEs.” *Physica D: Nonlinear Phenomena* 237, no. 23 (December 2008): 3061–68.

Powered by [Documenter.jl](#) and the [Julia Programming Language](#).

# ● Period doubling in Lur'e problem (PD aBS)

- Period doubling in Lur'e problem (PD aBS)
  - Periodic orbits with orthogonal collocation
  - Periodic orbits with Parallel Standard Shooting
  - Branch of periodic orbits with finite differences

The following model is an adaptive control system of Lur'e type. It is an example from the MatCont library.

$$\begin{cases} \dot{x} = y \\ \dot{y} = z \\ \dot{z} = -\alpha z - \beta y - x + x^2 \end{cases}$$

The model is interesting because there is a period doubling bifurcation and we want to show the branch switching capabilities of `BifurcationKit.jl` in this case. We provide 3 different ways to compute the periodic orbits and highlight their pro / cons.

It is easy to encode the ODE as follows

```
using Revise, Plots
import BifurcationKit as BK
import BifurcationKit: @optic

recordFromSolution(x, p; k...) = (u1 = x[1], u2 = x[2])

function lur!(dz, u, p, t = 0)
    (;α, β) = p
    x, y, z = u
    dz[1] = y
    dz[2] = z
    dz[3] = -α * z - β * y - x + x^2
    dz
end

# bifurcation problem
prob_bif = BK.ODEBifProblem(lur!, zeros(3), (α = -1.0, β = 1.0), (@optic _ .α);
    record_from_solution = recordFromSolution)
```

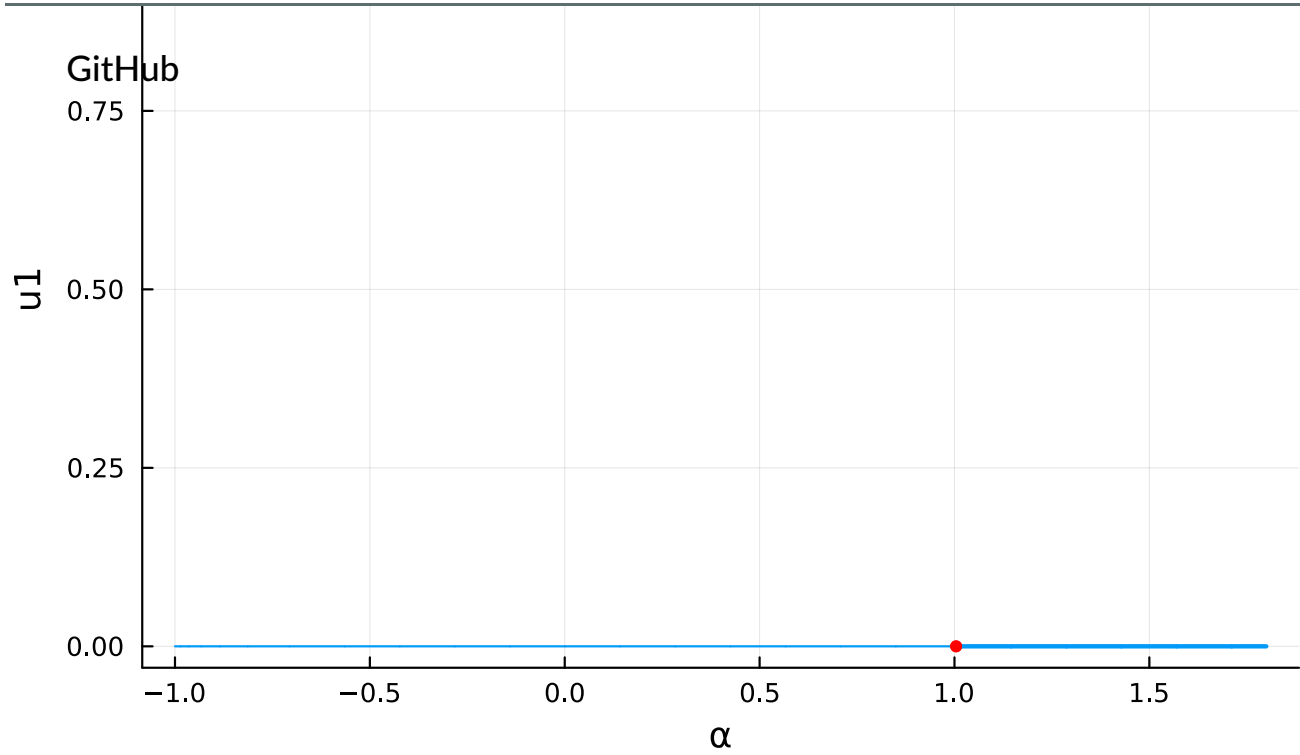
We first compute the branch of equilibria

```
# continuation options
opts_br = BK.ContinuationPar(p_min = -1.4, p_max = 1.8, nev = 3)

# computation of the branch
br = BK.continuation(prob_bif, BK.PALC(), opts_br)

scene = plot(br)
```





With detailed information:

br

- Curve type: **EquilibriumCont**
- Number of points: 26
- Type of vectors: **Vector{Float64}**
- Parameter  $\alpha$  starts at -1.0, ends at 1.8
- Algo: **PALC [Secant]**
- Special points:

- # 1, **hopf** at  $\alpha \approx +1.00420578 \in (+0.99978637, +1.00420578)$ ,  $|\delta p|=4e-03$ , [**converged**],  $\delta = (-2, -2)$ ,

- # 2, **endpoint** at  $\alpha \approx +1.80000000$ ,

We note the Hopf bifurcation point which we shall investigate now.

## Periodic orbits with orthogonal collocation

We compute the branch of periodic orbits from the Hopf bifurcation point. We rely on a the state of the art method for computing periodic orbits of ODE: orthogonal collocation.

We first define a plotting function and a record function which are used for all cases below:

```
# plotting function
function plotPO(x, p; k...)
    xtt = BK.get_periodic_orbit(p.prob, x, p.p)
    plot!(xtt.t, xtt[1,:]; markersize = 2, k...)
    plot!(xtt.t, xtt[2,:]; k...)
    plot!(xtt.t, xtt[3,:]; legend = false, k...)
end

# record function
function recordPO(x, p; k...)
    xtt = BK.get_periodic_orbit(p.prob, x, p.p)
    period = BK.getperiod(p.prob, x, p.p)
    mn, mx = extrema(xtt[1,:])
    return (;max = mx, min = mn, period)
end
```

recordPO (generic function with 1 method)

Continuation of periodic orbits from the Hopf point:

GitHub

```
br = BK.continuation(
    br, 1, opts_po_cont,
    BK.Collocation(40, 4);
    plot = true,
    # linear_algo = BK.COPBLS(),
    plot_solution = plotPO,
    normC = BK.norminf)

scene = plot(br, br_po)
```

Note that you can compute the PD normal form

```
BK.get_normal_form(br_po, 1)
```

We provide Automatic Branch Switching from the PD point and computing the bifurcated branch is as simple as:

```
# aBS from PD
br_po_pd = BK.continuation(deepcopy(br_po), 1, BK.ContinuationPar(br_po.contparams, max_steps = 100, dsma
    plot_solution = plotPO,
    normC = BK.norminf,
    )

scene = plot(br_po, br_po_pd, title = "Collocation based")
```

## Periodic orbits with Parallel Standard Shooting

We use a different method to compute periodic orbits: we rely on a fixed point of the flow. To compute the flow, we use OrdinaryDiffEq.jl. This way of computing periodic orbits should be less precise than the previous one. We rely on parallel multiple shooting. Finally, please note the close similarity to the code of the previous part. As before, we first rely on Hopf aBS.

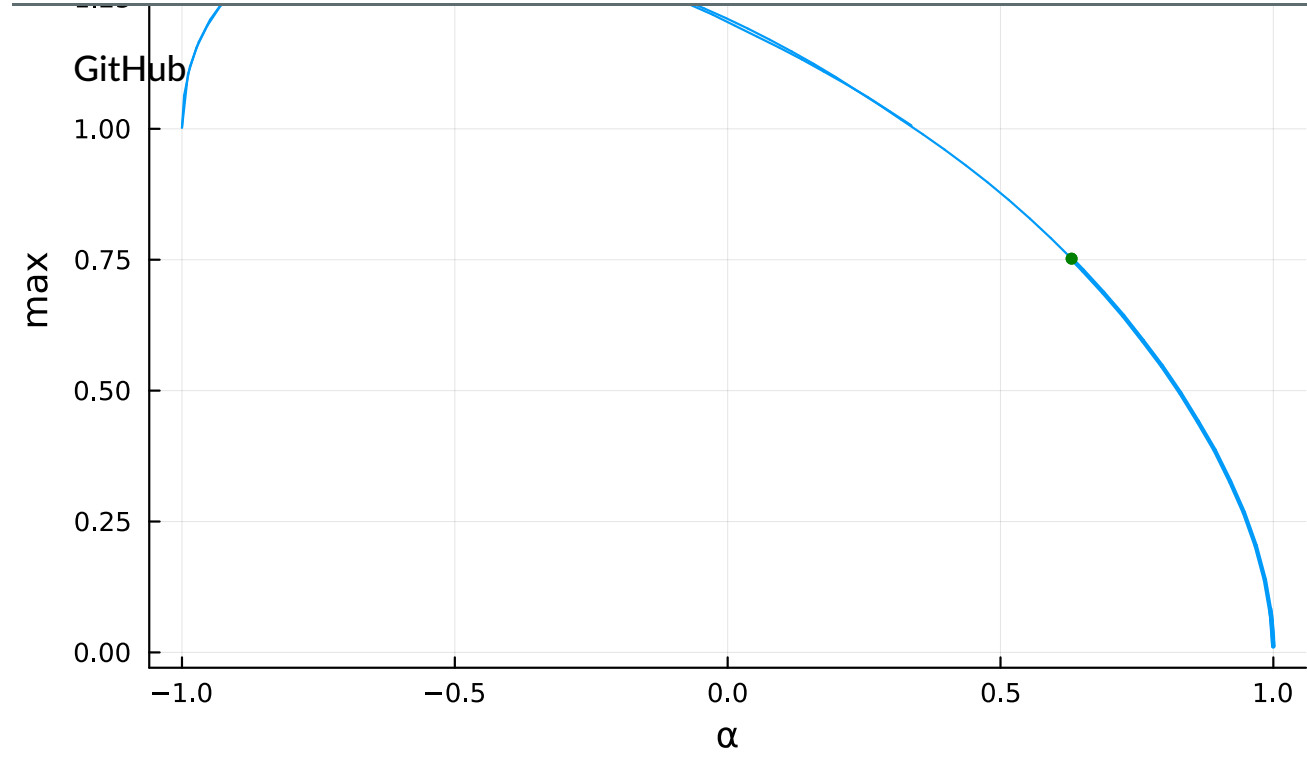
```
import OrdinaryDiffEq as ODE

# ODE problem for using DifferentialEquations
prob_ode = ODE.ODEProblem(lur!, prob_bif.u0, (0, 1), prob_bif.params; abstol = 1e-12, reltol = 1e-10)

# continuation parameters
opts_po_cont = BK.ContinuationPar(opts_br, dsmax = 0.03, newton_options = BK.NewtonPar(tol = 1e-10), tol

br_po = BK.continuation(
    br, 1, opts_po_cont,
    # parallel shooting functional with 10 sections
    BK.Shooting(5, prob_ode, ODE.Vern9(); parallel = true, abstol = 1e-10, reltol = 1e-8);
    # plot = true,
    record_from_solution = recordPO,
    plot_solution = plotPO,
    # limit the residual, useful to help DifferentialEquations
    callback_newton = BK.cbMaxNorm(10),
    normC = BK.norminf)

plot(br_po)
```



Note that you can compute the PD normal form

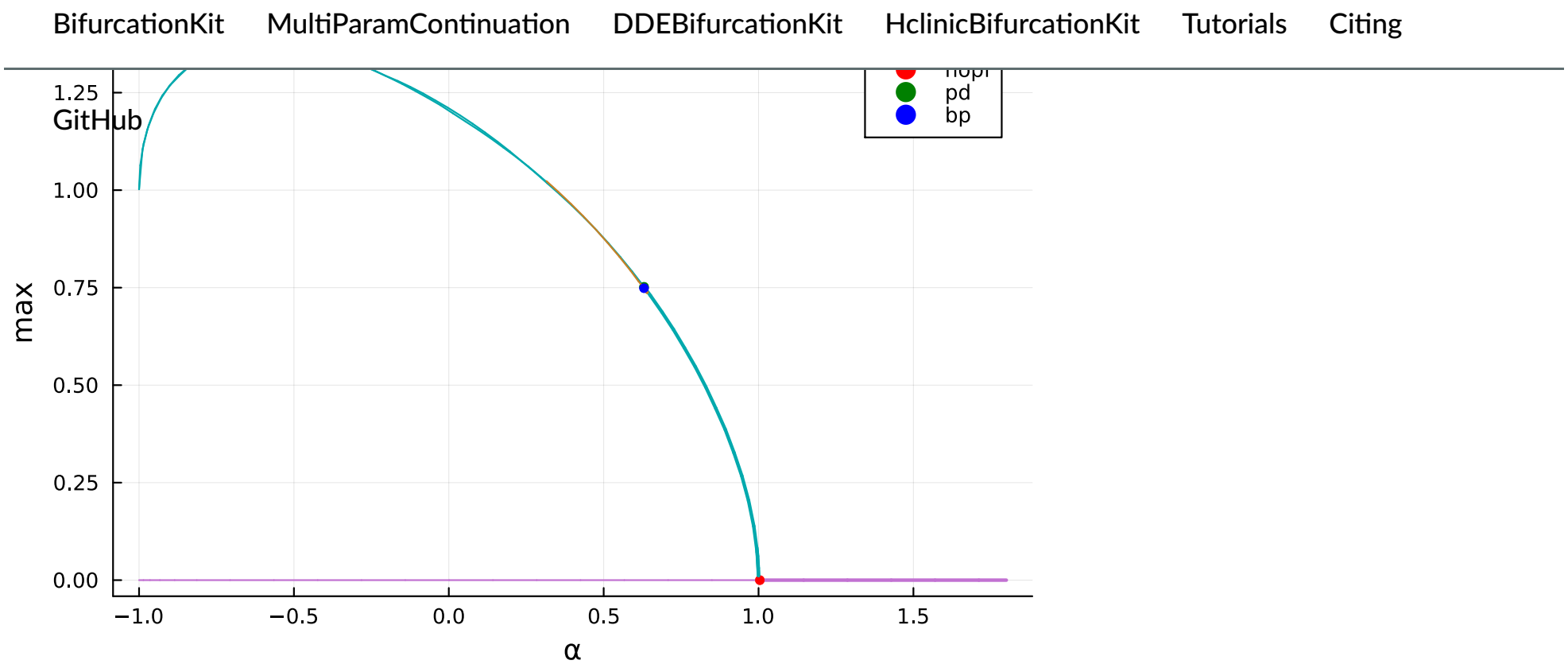
```
BK.get_normal_form(br_po, 1; autodiff = true)
```

```
Period-Doubling bifurcation point of periodic orbit
├─ Period = 6.36407292358091 -> 12.72814584716182
├─ Problem: Shooting
SuperCritical - Period-Doubling └─ Normal form:
├─  $x \rightarrow x \cdot (a \cdot \delta\alpha - 1 + c \cdot x^2)$ 
├─ a = -65.09708602337587
└─ c = 73.88795071377197
```

We provide Automatic Branch Switching from the PD point and computing the bifurcated branch is as simple as:

```
# aBS from PD
br_po_pd = BK.continuation(deepcopy(br_po), 1,
    BK.ContinuationPar(br_po.contparams, max_steps = 10, ds = -0.01);
    plot_solution = (x, p; k...) -> begin
        plotPO(x, p; k...)
        # add previous branch
        plot!(br_po; subplot = 1)
    end,
    record_from_solution = recordPO,
    # autodiff_nf = true,
    normC = BK.norminf,
    callback_newton = BK.cbMaxNorm(10),
)

scene = plot(br, br_po, br_po_pd, title = "Shooting based")
```



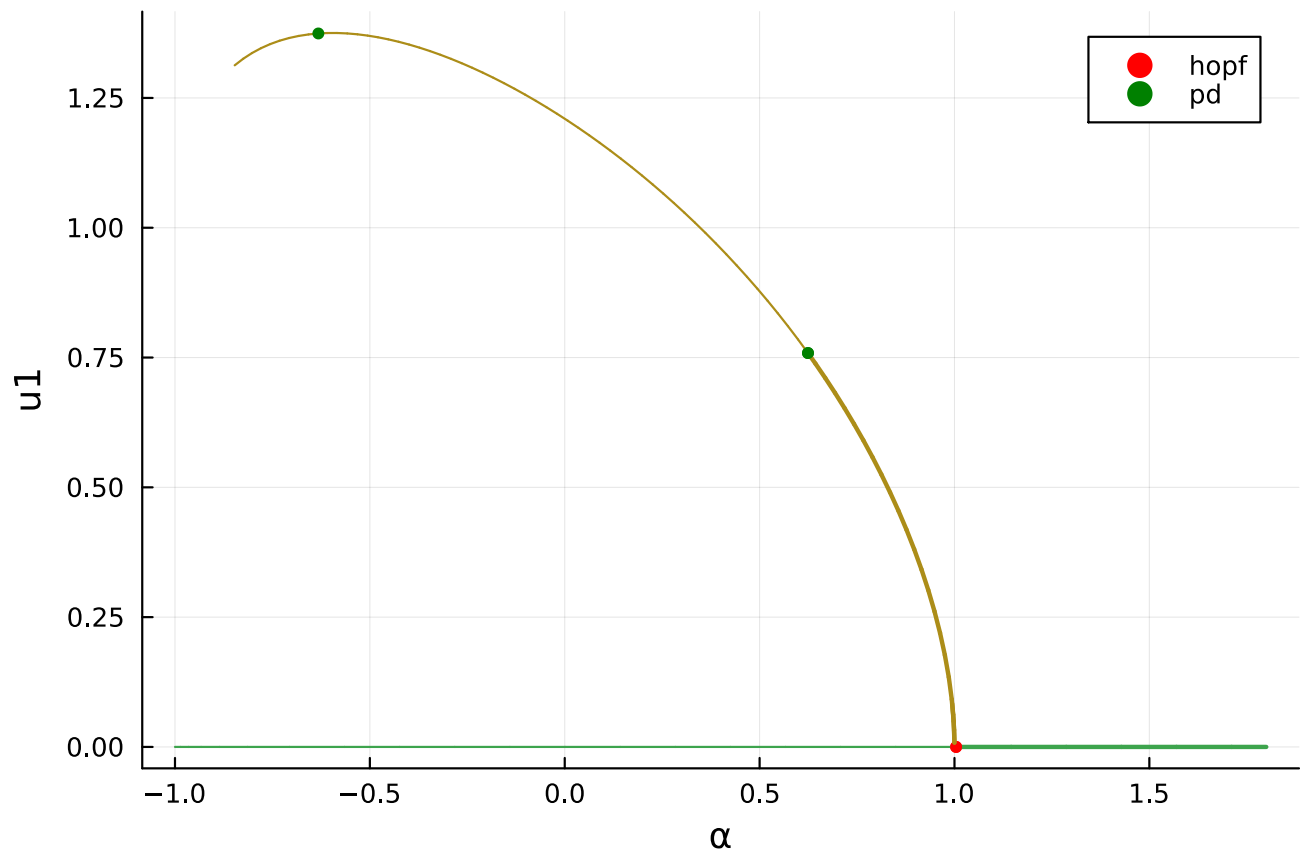
## Branch of periodic orbits with finite differences

We use finite differences to discretize the problem for finding periodic orbits. We appeal to automatic branch switching from the Hopf point as follows

```
# continuation parameters
opts_po_cont = BK.ContinuationPar(dsmax = 0.02, dsmin = 1e-4, p_max = 1.1, max_steps = 80, tol_stability

br_po = BK.continuation(
    br, 1, opts_po_cont,
    BK.Trapeze(M = 120);
    record_from_solution = recordPO,
    plot_solution = (x, p; k...) -> begin
        plotPO(x, p; k...)
        ## plot previous branch
        plot!(br, subplot=1, putbifptlegend = false)
    end,
    normC = BK.norminf)

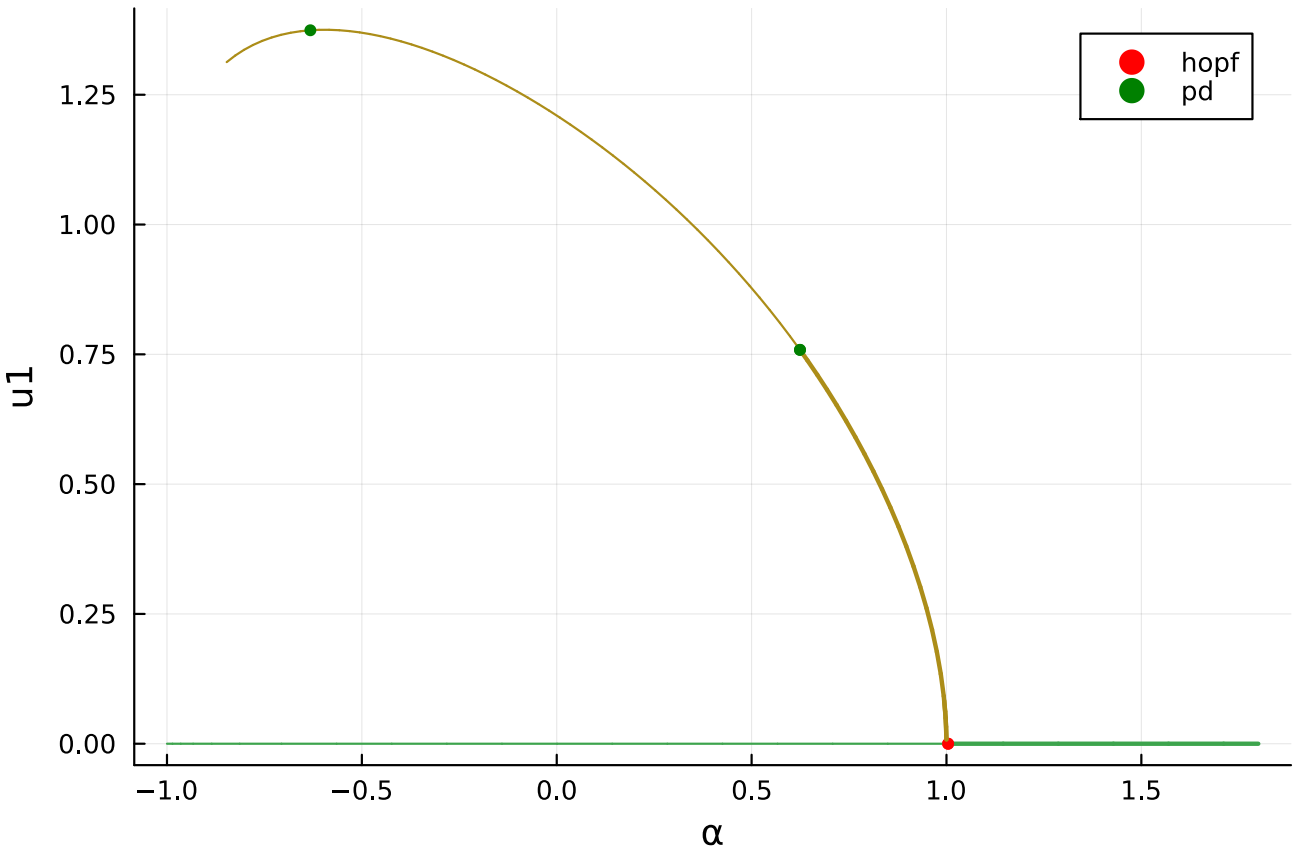
scene = plot(br, br_po)
```



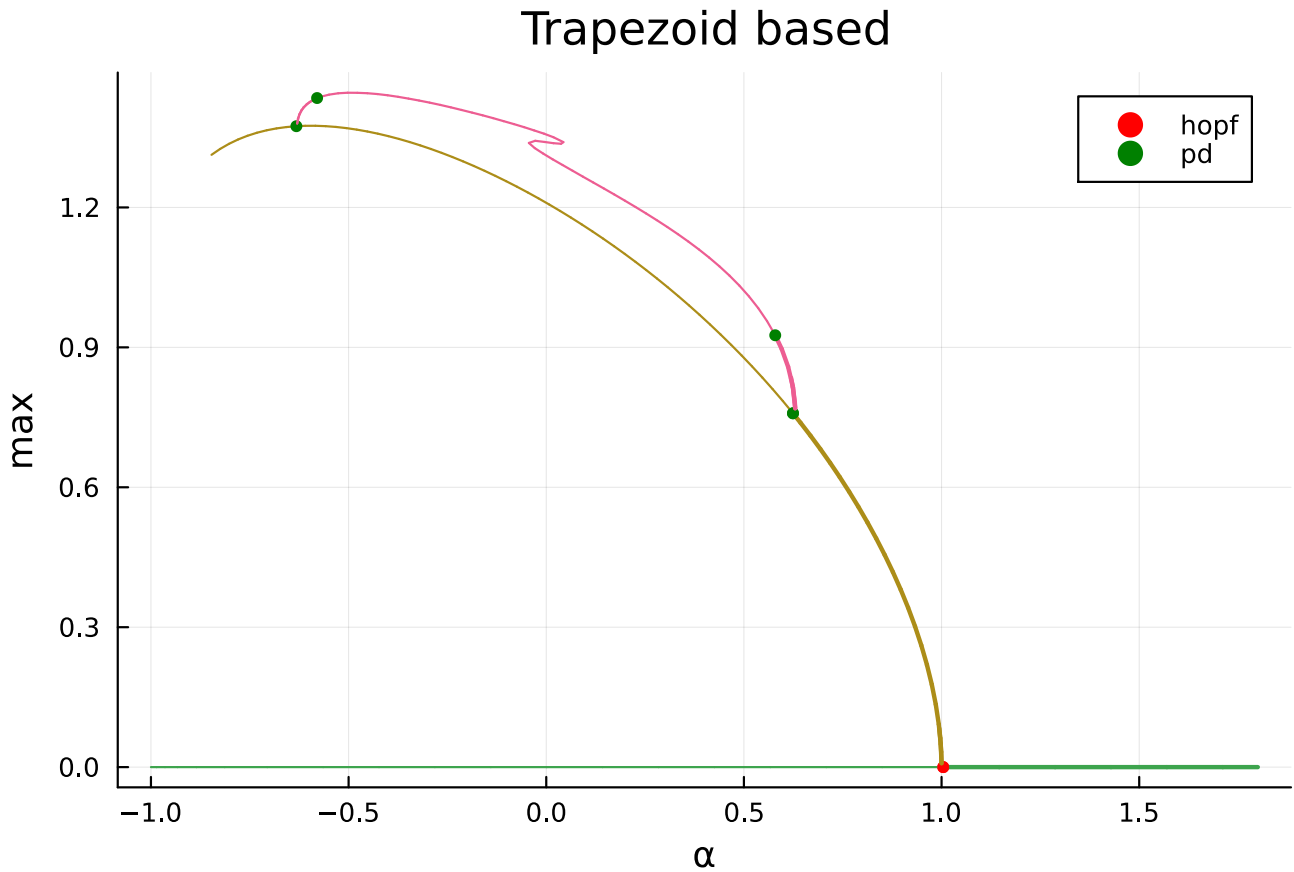
Two period doubling bifurcations were detected. We shall now compute the branch of periodic orbits from these PD points. We do not provide Automatic Branch Switching for Trapezoid method as we do not have yet the PD normal form computed for Trapeze. Hence, it takes some trial and error to find the ampfactor of the PD branch.

GitHub

```
# aBS from PD
br_po_pd = BK.continuation(deepcopy(br_po), 1, BK.ContinuationPar(br_po.contparams, max_steps = 70);
    # plot = true,
    ampfactor = .2, δp = -0.005,
    plot_solution = (x, p; k...) -> begin
        plotP0(x, p; k...)
        # add previous branch
        plot!(br_po; legend=false, subplot=1)
    end,
    record_from_solution = recordP0,
    normC = BK.norminf
)
Scene = title!("")
```



```
plot(br, br_po, br_po_pd, title = "Trapezoid based")
```



# Neural mass equation (Hopf aBS)

- Neural mass equation (Hopf aBS)
  - Branch of periodic orbits with Orthogonal Collocation
  - Periodic orbits with Parallel Standard Shooting
  - Branch of periodic orbits with Trapezoid method
    - Plot of some of the periodic orbits as function of  $E_0$
  - References

The following model is taken from [\[Cortes\]](#):

$$\begin{cases} \tau \dot{E} = -E + g(JuxE + E_0) \\ \dot{x} = \tau_D^{-1}(1 - x) - uEx \\ \dot{u} = U_0E(1 - u) - \tau_F^{-1}(u - U_0) \end{cases}$$

with

$$g(y) := \alpha \log(1 + \exp(y/\alpha)).$$

We use this model as a mean to introduce the basics of `BifurcationKit.jl`, namely the continuation of equilibria and periodic orbits (with the different methods).

The model is interesting on its own because the branch of periodic solutions converges to an homoclinic orbit which can be challenging to compute. We provide three different ways to compute these periodic orbits and highlight their pro / cons.

It is easy to encode the ODE as follows

```
using Revise, Plots
import BifurcationKit as BK
import BifurcationKit: @optic, @reset

# vector field
function TMvf!(dz, z, p, t = 0)
    (;J, α, E0, τ, τD, τF, U0) = p
    E, x, u = z
    SS0 = J * u * x * E + E0
    SS1 = α * log(1 + exp(SS0 / α))
    dz[1] = (-E + SS1) / τ
    dz[2] = (1 - x) / τD - u * x * E
    dz[3] = (U0 - u) / τF + U0 * (1 - u) * E
    dz
end

# parameter values
par_tm = (α = 1.4, τ = 0.013, J = 3.07, E0 = -2.0, τD = 0.20, U0 = 0.3, τF = 1.5)

# initial condition
z0 = [0.238616, 0.982747, 0.367876]

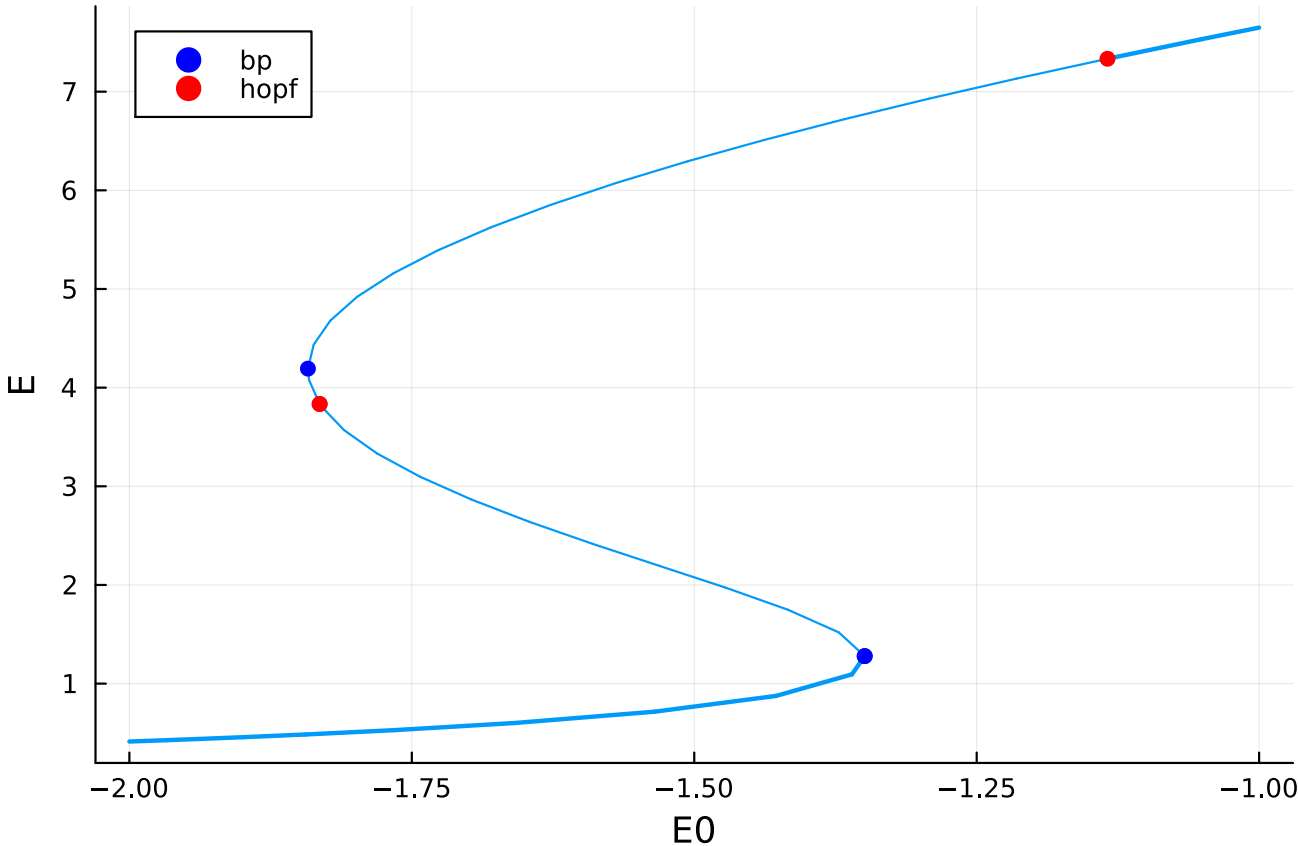
# Bifurcation Problem
prob = BK.ODEBifProblem(TMvf!, z0, par_tm, (@optic _ .E0);
    record_from_solution = (x, p; k...) -> (E = x[1], x = x[2], u = x[3]),)
```

We first compute the branch of equilibria



```
# Continuation of equilibria
br = BK.continuation(prob, BK.PALC(tangent = BK.Bordered()), opts_br; normC = BK.norminf)

scene = plot(br, plotfold=false, markersize=4, legend=:topleft)
```



With detailed information:

```
br
```

Curve type: **EquilibriumCont**

Number of points: 42

Type of vectors: **Vector{Float64}**

Parameter **E0** starts at -2.0, ends at -1.0

Algo: **PALC [Bordered]**

Special points:

- # 1, bp at E0 ≈ -1.34913909 ∈ (-1.34938031, -1.34913909), |δp|=2e-04, [converged], δ = ( 1, 0)

- # 2, hopf at E0 ≈ -1.83157834 ∈ (-1.83157834, -1.83112490), |δp|=5e-04, [converged], δ = ( 2, 2)

- # 3, bp at E0 ≈ -1.84196262 ∈ (-1.84196538, -1.84196262), |δp|=3e-06, [converged], δ = (-1, 0)

- # 4, hopf at E0 ≈ -1.13421119 ∈ (-1.13429114, -1.13421119), |δp|=8e-05, [converged], δ = (-2, -2)

- # 5, endpoint at E0 ≈ -1.00000000,

## Branch of periodic orbits with Orthogonal Collocation

We compute the branch of periodic orbits from the last Hopf bifurcation point (on the right). We use Orthogonal Collocation to discretize the problem of finding periodic orbits. This is vastly more precise than the following methods because we use mesh adaptation.

```
# arguments for periodic orbits
# one function to record information and one
# function for plotting
@views function record_from_solution(x, p; k...)
    xtt = BK.get_periodic_orbit(p.prob, x, p.p)
    return (max = maximum(xtt[1,:]),
            min = minimum(xtt[1,:]),
            period = BK.getperiod(p.prob, x, p.p))
end

@views function plot_solution(x, p; k...)
    xtt = BK.get_periodic_orbit(p.prob, x, p.p)
```

GitHub

```
plot!(xtt.t, xtt[3,:]; label = "u", arg..., k...)
plot!(br; subplot = 1, putspecialptlegend = false)

end

args_po = (      ;record_from_solution,
            plot_solution,
            # we use the supremum norm
            normC = BK.norminf)

# continuation parameters
opts_po_cont = BK.ContinuationPar(opts_br, ds= 0.001, dsmin = 1e-4, dsmax = 0.1,
                                   max_steps = 110,
                                   tol_stability = 1e-5)

br_pocoll = @time BK.continuation(
    # we want to branch form the 4th bif. point
    br, 4, opts_po_cont,
    # we want to use the Collocation method to locate PO, with polynomial degree 4
    BK.Collocation(50, 4; meshadapt = true);
    # regular continuation options
    plot = true,
    args_po...)

Scene = title!("")
```

Let us plot the periodic orbit close to the end of the branch

```
sol = BK.get_periodic_orbit(br_pocoll, 100)
plot(sol, title = "Periodic orbit", marker = :d, markersize = 1)
```

## Periodic orbits with Parallel Standard Shooting

We use a different method to compute periodic orbits: we rely on a fixed point of the flow. To compute the flow, we use `DifferentialEquations.jl`. This way of computing periodic orbits should be more precise than the Trapezoid method. We use a particular instance called multiple shooting which is computed in parallel. This is an additional advantage compared to the two other methods.

```
import OrdinaryDiffEq as ODE

probsh = ODE.ODEProblem(TMvf!, copy(z0), (0, 1), par_tm; abstol = 1e-12, reltol = 1e-10)

opts_po_cont = BK.ContinuationPar(opts_br, dsmax = 0.1, ds= -0.0001, dsmin = 1e-4, max_steps = 110, tol_s

br_posh = @time BK.continuation(
    br, 4, opts_po_cont,
    # this is where we tell that we want Standard Shooting
    # with 15 time sections
    BK.Shooting(15, probsh, ODE.Vern9(), parallel = true);
    # regular continuation parameters
    plot = true,
    args_po...,
    # we reject the step when the residual is high
    callback_newton = BK.cbMaxNorm(10)
)

Scene = title!("")
```

## Branch of periodic orbits with Trapezoid method



unbounded close to the homoclinic orbit.

GitHub

```
# continuation parameters
opts_po_cont = BK.ContinuationPar(opts_br, dsmax = 0.1, ds = 0.004, dsmin = 1e-4,
    max_steps = 80, tol_stability = 1e-7)

@reset opts_po_cont.newton_options.tol = 1e-7

Mt = 250 # number of time sections
br_potrap = BK.continuation(
    # we want to branch form the 4th bif. point
    br, 4, opts_po_cont,
    # we want to use the Trapeze method to locate P0
    BK.Trapeze(M = Mt);
    δp = 0.001,
    plot = true,
    args_po...,
)

scene = plot(br, br_potrap, markersize = 3)
plot!(scene, br_potrap.param, br_potrap.min, label = "")
```

We plot the maximum (resp. minimum) of the limit cycle. We can see that the min converges to the smallest equilibrium indicating a homoclinic orbit.

## Plot of some of the periodic orbits as function of $E_0$

We can plot some of the previously computed periodic orbits in the plane  $(E, x)$  as function of  $E_0$ :

```
plot()
# fetch the saved solutions
for i_sol in 1:2:40
    traj = BK.get_periodic_orbit(br_potrap, i_sol)
    plot!(traj[1,:), traj[2,:), xlabel = "E", ylabel = "x", label = "")
end
title!("")
```

## References

- Cortes

Cortes, Jesus M., Mathieu Desroches, Serafim Rodrigues, Romain Veltz, Miguel A. Muñoz, and Terrence J. Sejnowski. **Short-Term Synaptic Plasticity in the Deterministic Tsodyks–Markram Model Leads to Unpredictable Network Dynamics.**” Proceedings of the National Academy of Sciences 110, no. 41 (October 8, 2013): 16610–15. <https://doi.org/10.1073/pnas.1316071110>.

# Steinmetz-Larter model

- Steinmetz-Larter model
  - Computation with Shooting
    - Curve of Fold points of periodic orbits
    - Curve of NS points of periodic orbits
  - Computation with collocation
    - Curve of Fold points of periodic orbits
    - Curve of NS points of periodic orbits

This example is found in the MatCont ecosystem.

The Steinmetz-Larter model is studied in the MatCont ecosystem because it is a simple example where a Chenciner bifurcation occurs.

$$\begin{cases} \dot{A} = -k_1ABX - k_3ABY + k_7 - k_{-7}A, \\ \dot{B} = -k_1ABX - k_3ABY + k_8, \\ \dot{X} = k_1ABX - 2k_2X^2 + 2k_3ABY - k_4X + k_6, \\ \dot{Y} = -k_3ABY + 2k_2X^2 - k_5Y, \end{cases} \tag{E}$$

This tutorial is also useful in that we show how to start periodic orbits continuation from solutions obtained from solving ODEs. Being a relatively advanced tutorial, we do not give too many details.

We start by coding the bifurcation problem.

```
using Revise, Plots
import BifurcationKit as BK
import BifurcationKit: @optic, @reset, norminf

function SL!(du, u, p, t = 0)
    (;k1, k2, k3, k4, k5, k6, k7, k_7, k8) = p
    A,B,X,Y = u

    du[1] = -k1*A*B*X - k3*A*B*Y + k7 - k_7*A
    du[2] = -k1*A*B*X - k3*A*B*Y + k8
    du[3] = k1*A*B*X - 2*k2*X^2 + 2*k3*A*B*Y - k4*X+k6
    du[4] = -k3*A*B*Y + 2*k2*X^2 - k5*Y
    du
end

z0 = rand(4)
par_sl = (k1=0.1631021, k2=1250., k3=0.046875, k4=20., k5=1.104, k6=0.001, k_7=0.1175, k7=1.5, k8=0.75)
bifprob = BK.ODEBifProblem(SL!, z0, par_sl, (@optic _ .k8);)

# record variables for plotting
function recordFromSolution(x, p; k...)
    xtt = BK.get_periodic_orbit(p.prob, x, p.p)
    mi, ma = @views extrema(xtt[1, :])
    return (max = ma,
            min = mi,
            amplitude = ma - mi,
            period = BK.getperiod(p.prob, x, p.p))
end
```

```

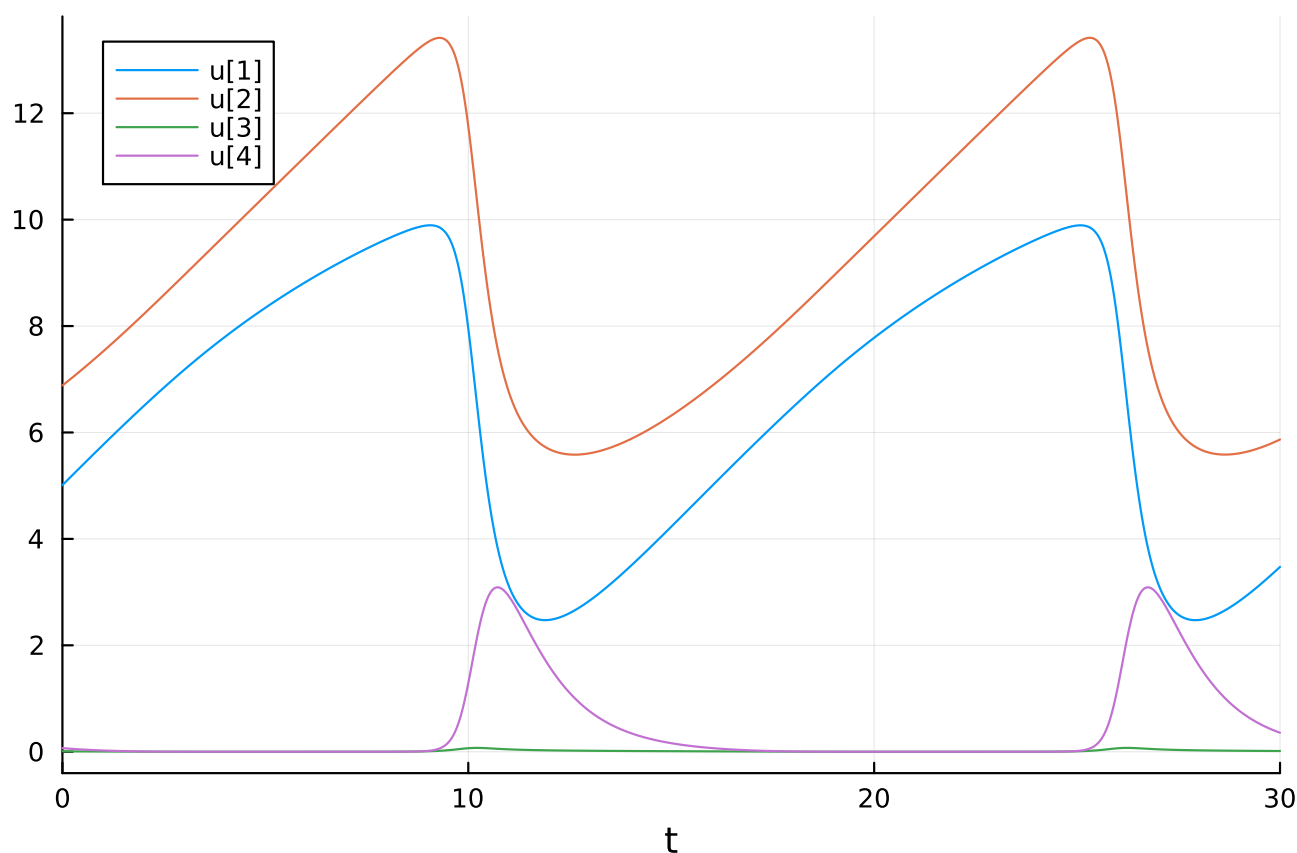
GitHub xtt = BK.get_periodic_orbit(p.prob, x, p.p)
      plot!(xtt.t, xtt.u[1:4,:]' ; label = "", k...)
end

```

```
# group parameters
argspo = (
    record_from_solution = recordFromSolution,
    plot_solution = plotSolution,
    normC = norminf)
```

We obtain some trajectories as seeds for computing periodic orbits.

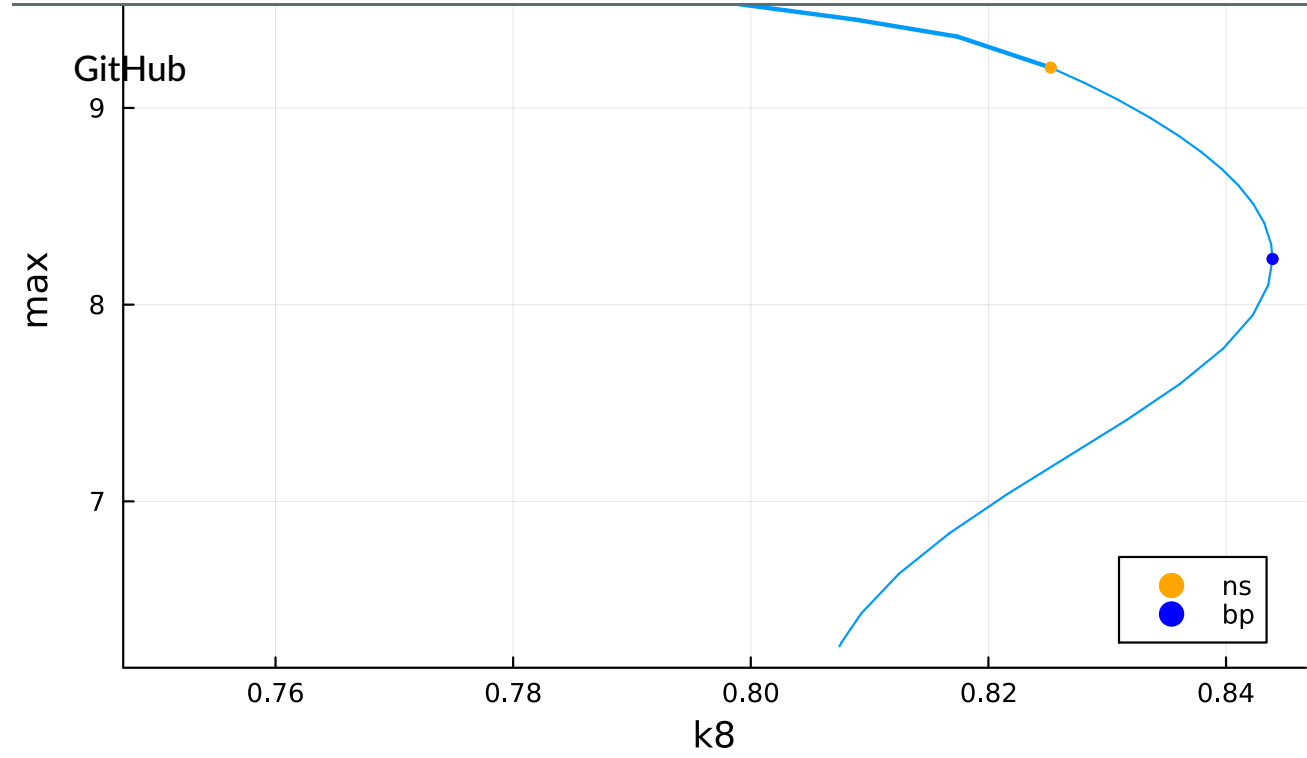
```
import OrdinaryDiffEq as ODE
alg_ode = ODE.Vern9()
prob_de = ODE.ODEProblem(SL!, z0, (0, 136.), par_sl)
sol_ode = ODE.solve(prob_de, alg_ode)
prob_de = ODE.ODEProblem(SL!, sol_ode.u[end], (0, 30.), sol_ode.prob.p, reltol = 1e-11, abstol = 1e-13)
sol_ode = ODE.solve(prob_de, alg_ode)
plot(sol_ode)
```



## Computation with Shooting

We generate a shooting problem from the computed trajectories and continue the periodic orbits as function of  $k_8$

```
probsh, cish = BK.generate_ci_problem( BK.Shooting(M = 5; jacobian = BK.AutoDiffDenseAnalytical() ), deep
opts_po_cont = BK.ContinuationPar(p_min = 0., p_max = 20.0, ds = 0.002, n_inversion = 6, nev = 4, max_ste
br_sh = BK.continuation(deepcopy(probsh), cish, BK.PALC(tangent = BK.Bordered()), opts_po_cont;
    # verbosity = 3, plot = true,
    callback_newton = BK.cbMaxNorm(10),
    argspo...
)
scene = plot(br_sh)
```



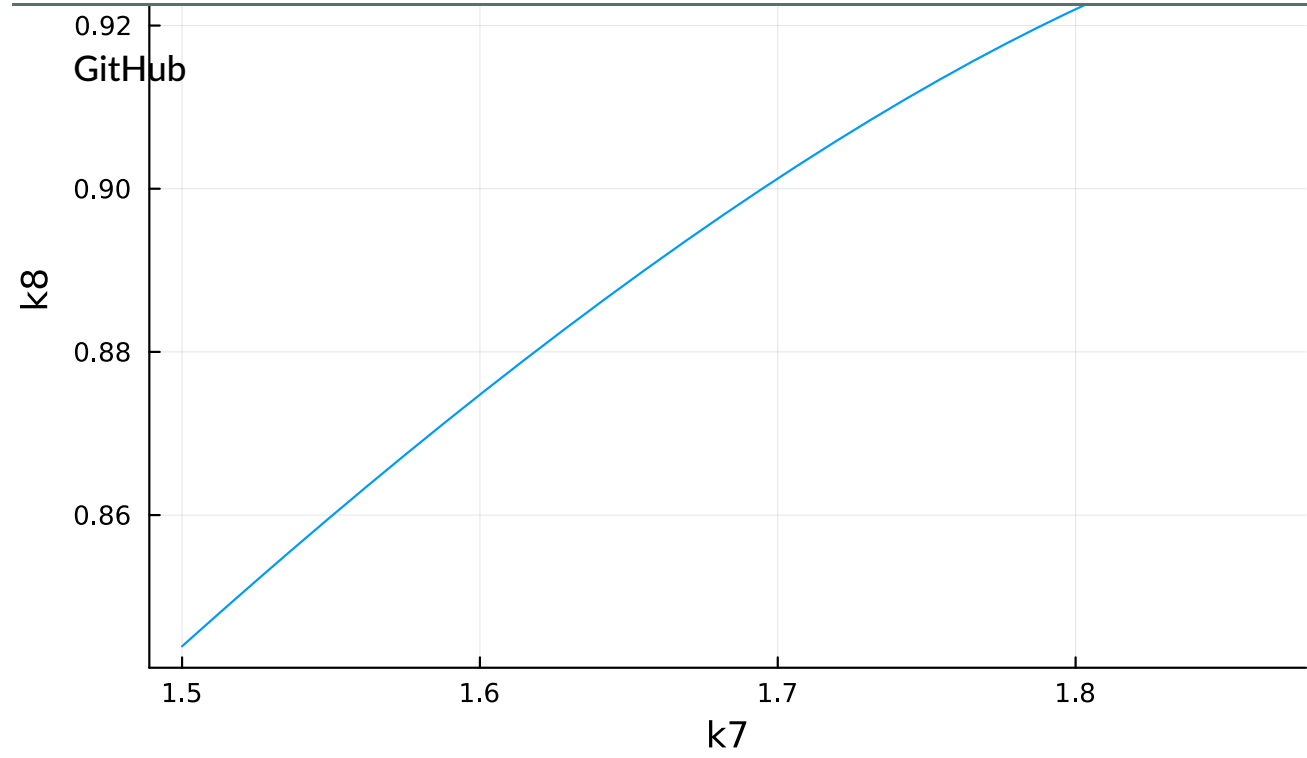
## Curve of Fold points of periodic orbits

```
opts_posh_fold = BK.ContinuationPar(br_sh.contparams, detect_bifurcation = 3, max_steps = 35, p_max = 1.9
@reset opts_posh_fold.newton_options.tol = 1e-12
fold_po_sh = BK.continuation(deepcopy(br_sh), 2, (@optic _ .k7), opts_posh_fold;
    # verbosity = 2, plot = true,
    alg = BK.PALC(),
    detect_codim2_bifurcation = 0,
    start_with_eigen = false,
    usehessian = true,
    jacobian_ma = BK.MinAug(),
    normC = norminf,
    callback_newton = BK.cbMaxNorm(1e1),
)
```

```
└─ Curve type: FoldPeriodicOrbitCont
└─ Number of points: 36
└─ Type of vectors: BorderedArray{Vector{Float64}, Float64}
└─ Parameters (:k8, :k7)
└─ Parameter k7 starts at 1.5, ends at 1.8663709776369324
└─ Algo: PALC [Secant]
└─ Special points:

- # 1,      R1 at k7 ≈ +1.85749194 ∈ (+1.85747448, +1.85749194), |δp|=2e-05, [converged], δ = (-1, 0)
- # 2, endpoint at k7 ≈ +1.87513072,
```

```
plot(fold_po_sh)
```



## Curve of NS points of periodic orbits

```
opts_posh_ns = BK.ContinuationPar(br_sh.contparams, detect_bifurcation = 0, max_steps = 35, p_max = 1.9,
@reset opts_posh_ns.newton_options.tol = 1e-11
ns_po_sh = BK.continuation(deepcopy(br_sh), 1, (@optic _ .k7), opts_posh_ns;
    # verbosity = 2,
    # plot = true,
    detect_codim2_bifurcation = 2,
    # update_minaug_every_step = 1,
    jacobian_ma = BK.MinAug(),
    normC = norminf,
    callback_newton = BK.cbMaxNorm(1e1),
    )
```

```
scene = plot(ns_po_sh, vars = (:k7, :k8), branchlabel = "NS")
plot!(scene, fold_po_sh, branchlabel = "Fold-PO")
scene
```

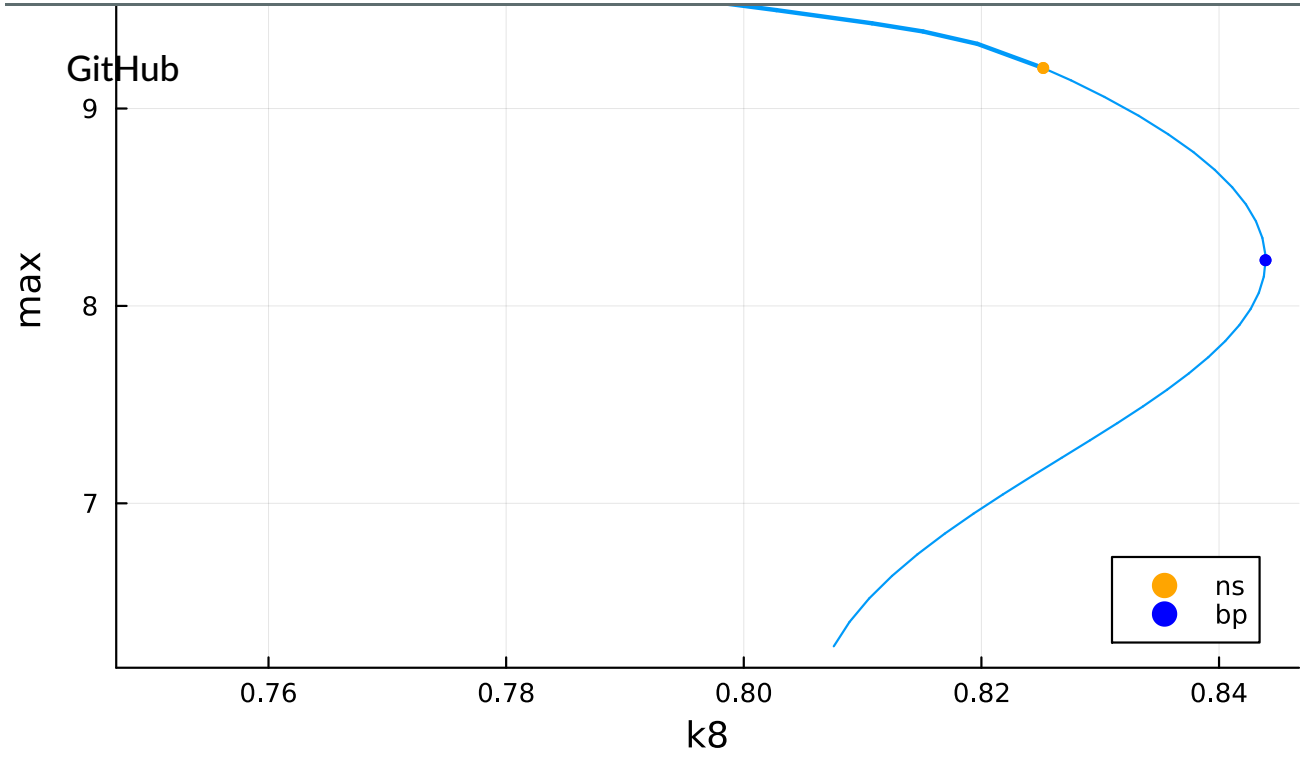
## Computation with collocation

```
probcoll, cicoll = BK.generate_ci_problem( BK.Collocation(50, 4),
    deepcopy(bifprob),
    sol_ode,
    16.)

opts_po_cont = BK.ContinuationPar(p_min = 0., p_max = 2.0,
    ds = 0.002, dsmax = 0.05,
    n_inversion = 6,
    nev = 4,
    max_steps = 50,
    detect_bifurcation = 3,
    tol_stability = 1e-5)

br_coll = @time BK.continuation(deepcopy(probcoll), copy(cicoll), BK.PALC(), opts_po_cont;
    argspo...,
    callback_newton = BK.cbMaxNorm(1e1),
    )

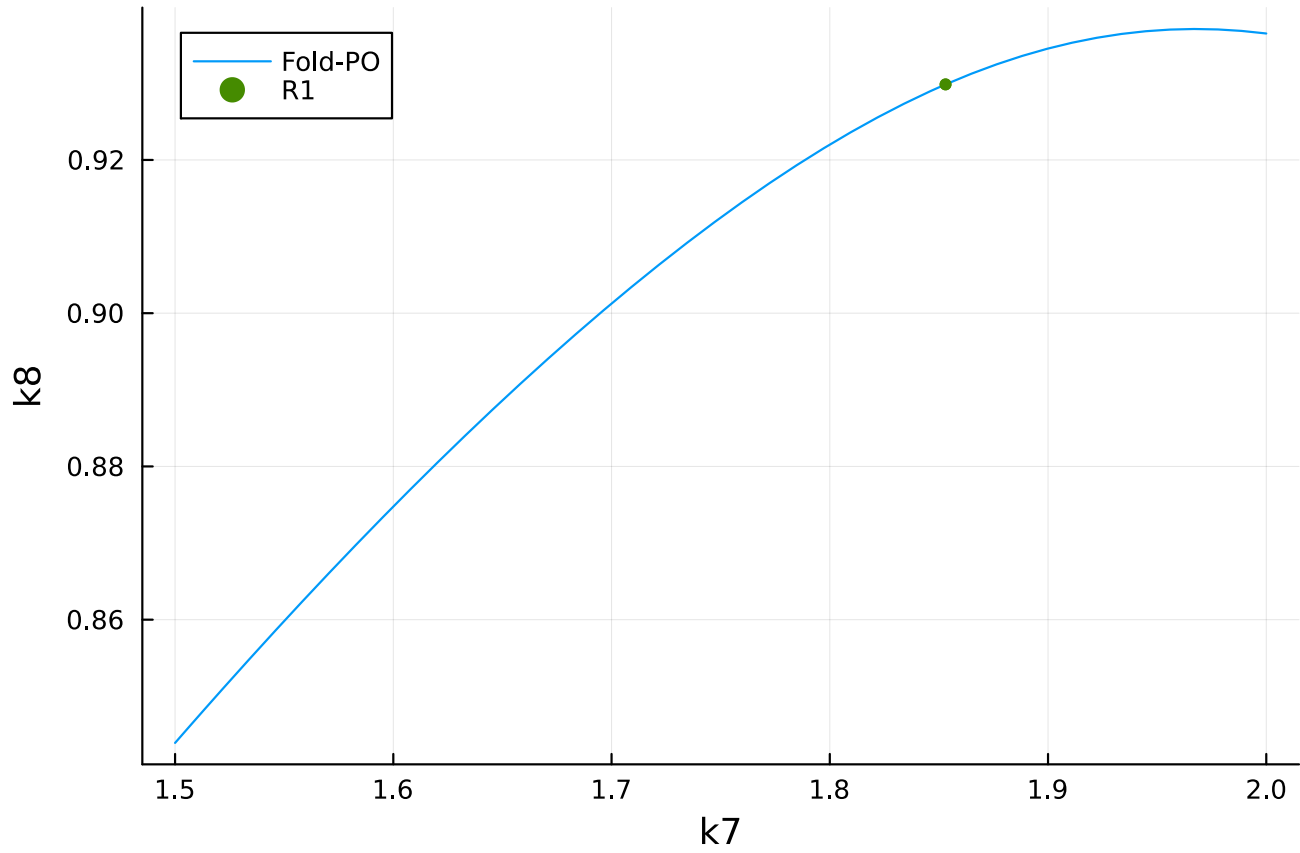
scene = plot(br_coll)
```



### Curve of Fold points of periodic orbits

```
opts_pocl_fold = BK.ContinuationPar(br_coll.contparams, detect_bifurcation = 0, plot_every_step = 10, dsr
@reset opts_pocl_fold.newton_options.verbose = false
@reset opts_pocl_fold.newton_options.tol = 1e-11
fold_po_cl = BK.continuation(deepcopy(br_coll), 2, (@optic _ .k7), opts_pocl_fold;
    # verbosity = 3, # plot = true,
    detect_codim2_bifurcation = 2,
    usehessian = true,
    jacobian_ma = BK.MinAug(),
    normC = norminf,
    callback_newton = BK.cbMaxNorm(1e1),
)

plot(fold_po_cl, branchlabel = "Fold-PO")
```



### Curve of NS points of periodic orbits

```
opts_pocl_ns = BK.ContinuationPar(br_coll.contparams, detect_bifurcation = 1, dsmax = 4e-2, max_steps = 4
ns_po_cl = BK.continuation(deepcopy(br_coll), 1, (@optic _ .k7), opts_pocl_ns;
    # verbosity = 3, plot = true,
    detect_codim2_bifurcation = 2,
    jacobian_ma = BK.MinAugMatrixBased(),
```



GitHub

Curve type: **NSPeriodicOrbitCont**

Number of points: 41

Type of vectors: **Vector{Float64}**

Parameters (:k8, :k7)

Parameter **k7** starts at 1.5, ends at 1.8553381611582482

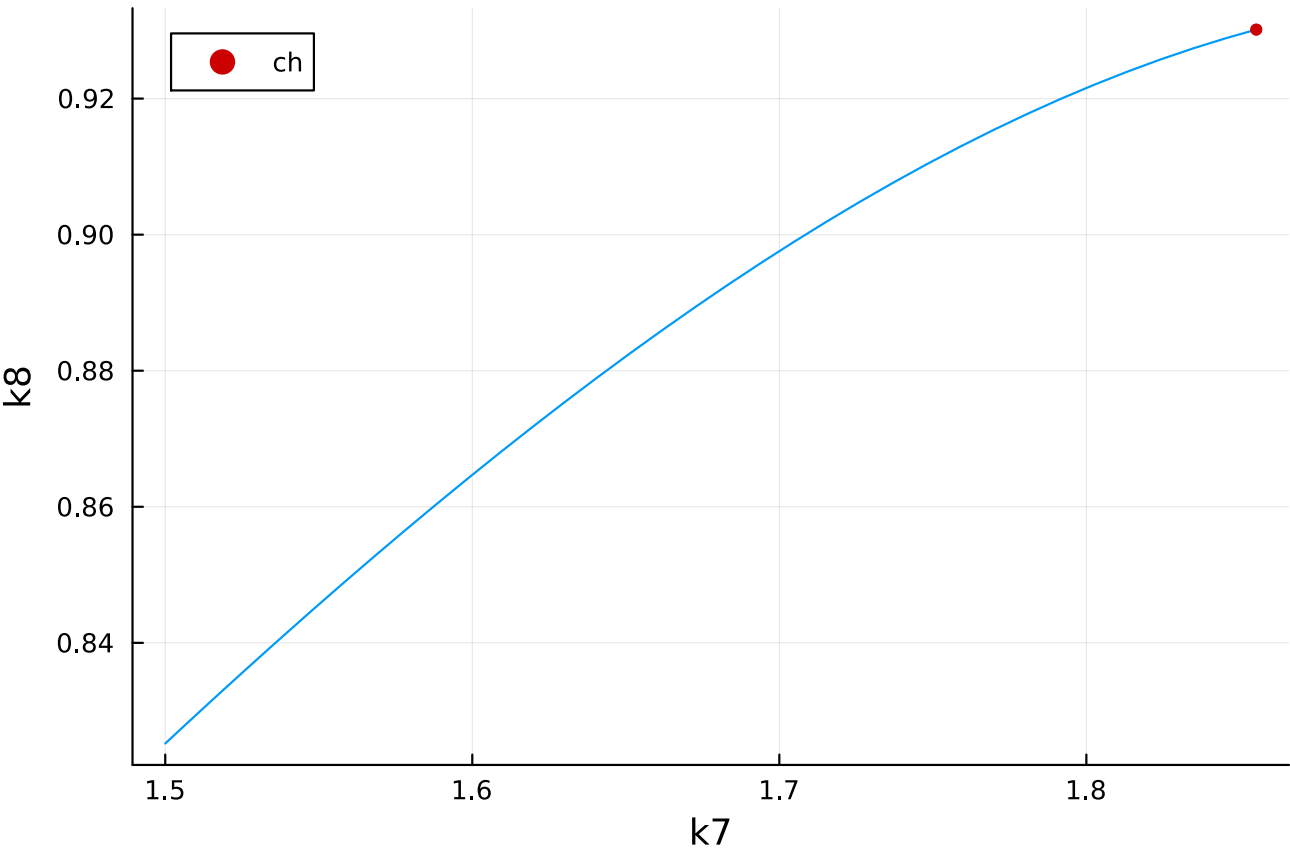
Algo: **PALC [Secant]**

Special points:

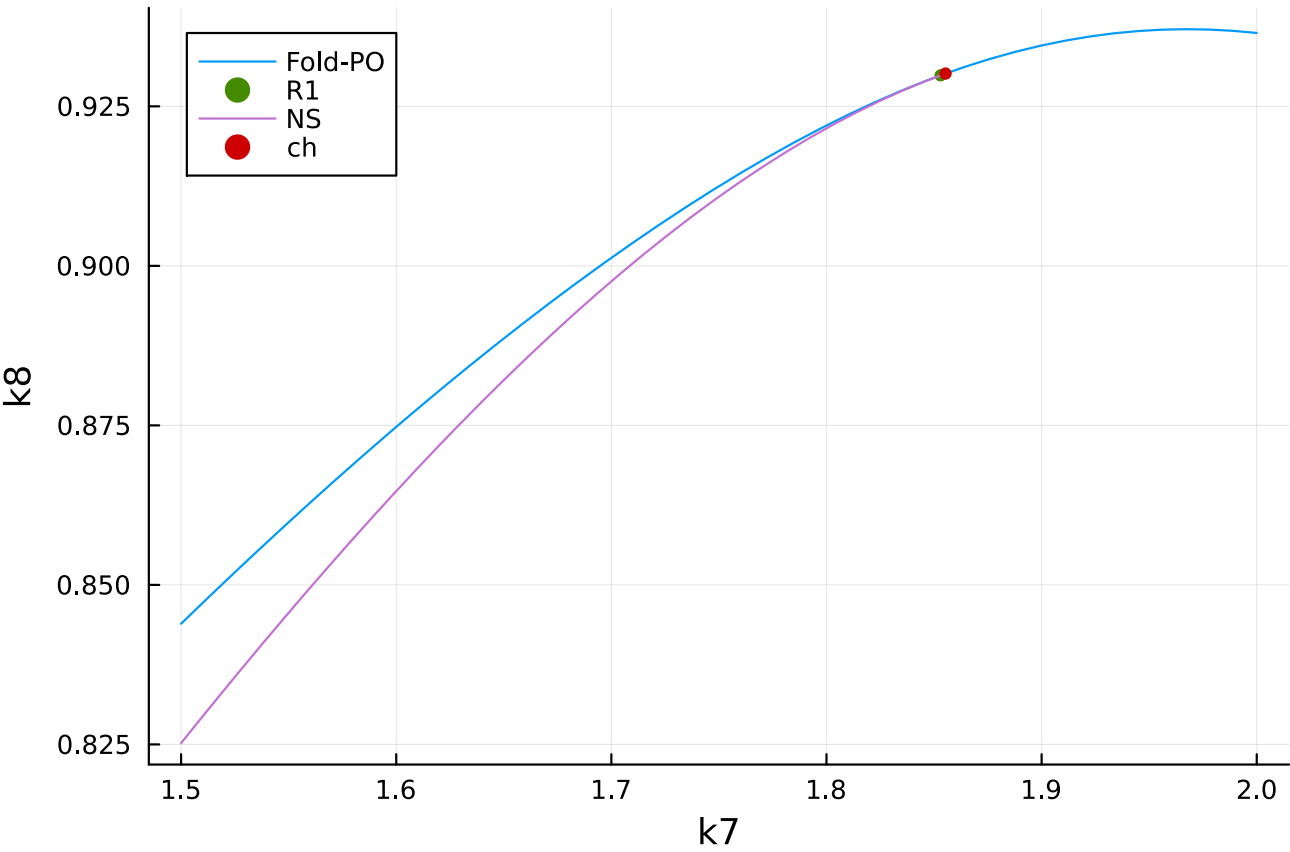
- # 1, **ch** at  $k7 \approx +1.85533816 \in (+1.85533751, +1.85533816)$ ,  $|\delta p|=7e-07$ , **[converged]**,  $\delta = (0, 0)$

- # 2, **endpoint** at  $k7 \approx +1.86711387$ ,

```
scene = plot(ns_po_cl; vars = (:k7, :k8))
```



```
scene = plot(fold_po_cl, branchlabel = "Fold-PO")
plot!(ns_po_cl; vars = (:k7, :k8), branchlabel = "NS")
scene
```



---

[GitHub](#)



# Educational introduction to bifurcation analysis (contributed by G. Datseris)

- Educational introduction to bifurcation analysis (contributed by G. Datseris)
  - Newton's method for finding fixed points
  - Continuation of a bifurcation curve
    - The secant predictor
    - The corrector
  - Code: Newton's algorithm in mixed space
  - Code: Corrector function
  - Code: Predictor function
  - Code: Continuation function
  - Running an example

This page is an educational introduction to bifurcation analysis and creation of bifurcation diagrams. It will show a completely self-contained approach, and its goal is to serve as an introduction to users not yet fully familiar with bifurcation analysis.

The text here is based on Chapter 4 of [Nonlinear Dynamics: a concise introduction interlaced with code](#) and is presented in a shortened form. It is focusing on using the simplest possible method for bifurcation continuation (the secant method) and can only address fixed points (the standard version of Newton's algorithm). Of course, this does not reflect the full features and power of BifurcationKit.jl, but it shows the basic guiding principles.

## Newton's method for finding fixed points

Let  $\mathbf{x}$  be a  $D$ -dimensional vector participating in the continuous dynamical system  $\dot{\mathbf{x}} = f(\mathbf{x}, p)$  which also depends on some arbitrary parameter  $p \in \mathbb{R}$ . To identify fixed points, we want to find the roots  $\mathbf{x}^*$  of  $f$ . Notice that everything that follows also applies to discrete dynamical systems where  $\mathbf{x}_{n+1} = g(\mathbf{x}_n)$ . The difference is that we simply define  $f = g - \mathbf{Id}$  and find the roots of  $f$  again.

To find the roots we can use Newton's method. Starting for some point  $\mathbf{x}_0$ , the following sequence will (typically) converge to a root:

$$\mathbf{x}_{j+1} = \mathbf{x}_j - \delta J_f^{-1}(\mathbf{x}_j) f(\mathbf{x}_j)$$

for  $0 \leq \delta \leq 1$  and with  $J_f$  the  $D \times D$  Jacobian matrix of  $f$ .

## Continuation of a bifurcation curve

A bifurcation curve (in the simplified context of this page) is the location of fixed points versus a parameter  $p$ . The simplest, but also most brute force, way to compute such a curve would be to scan the parameter axis, and for each parameter value apply Newton's method to find the fixed point. Because there could be more than one fixed points at a given parameter, one needs several initial conditions plugged into Newton's method, to ensure all fixed points are found.

A bit better approach would be to true and *continue* the *curve* of the fixed point  $\mathbf{x}^*(p)$ . To do this one needs two ingredients: (1) a predictor and (2) a corrector. The first estimates a new "seed" for Newton's method that attempts to continue the existing points we have found on the curve. The second corrects this prediction to a point in the state-parameter space that is on the bifurcation curve.

## The secant predictor

GitHub

The corrector

The prediction  $\tilde{\mathbf{z}}$  of the secant needs to be corrected not only in the variable space  $\mathbf{x}$ , which is what Newton's method currently does, but also on parameter space. To do this, we need to extend the function  $f$  to have one more element (because otherwise we would have 1 too many unknowns give the amount of equations we have to estimate the zero of  $f(\mathbf{z})$ ). We will extend  $f$  so that its  $D + 1$  entry enforces the  $k$ -th element of the root to be the one suggested by the predictor. I.e., we define the function  $h$

$$h(\mathbf{x}) = (f_1(\mathbf{z}), \dots, f_D(\mathbf{z}), \mathbf{z}[k] - \tilde{\mathbf{z}}[k])$$

and will try to find the zeros of  $h$  now via Newton's method. Notice that now the Jacobian  $J_h$  is  $D + 1$  dimensional, with the last column being the derivatives of  $f$  towards the parameter  $p$ .

## Code: Newton's algorithm in mixed space

Let's first write the code that will be performing Newton's algorithm for the function  $h$  in the mixed space of  $\mathbf{z}$ . For convenience, we would expect the user to only provide the functions  $f, J_f$  as functions  $f(\mathbf{x}, p), J(\mathbf{x}, p)$  with  $\mathbf{x}$  a vector and  $p$  a number. We can do everything else using automatic differentiation.

(Notice: for simplicity, and to be in style with BifurcationKit.jl we will make everything allocate and return new Vector instances. Performance-wise, the code written in this page is as bad as it can possibly get)

First, define a function that given  $f, J$ , the mixed state  $\mathbf{z}$  and the index  $k$  it returns the mixed Jacobian  $J_h$

```
using ForwardDiff # for auto-differentiation
using LinearAlgebra

function mixed_jacobian(z, k, f, J)
    x = z[1:end-1]; p = z[end]
    # start creating the mixed space jacobian
    j = J(x, p)
    # to the state space jacobian add one more column, derivative towards p
    pder = ForwardDiff.derivative(p -> f(x, p), p)
    Jmixed = hcat(j, pder)
    # add the last row, which is 1 for the `k` entry, 0 everywhere else
    last_row = zeros(length(z)); last_row[k] = 1.0
    Jfinal = vcat(Jmixed, last_row')
    return Jfinal
end
```

```
mixed_jacobian (generic function with 1 method)
```

Then we write a function that takes one step of the Newton's algorithm:

```
using LinearAlgebra

function newton_step!(zj, zpred, i, f, J, δ)
    Jfinal = mixed_jacobian(zj, i, f, J)
    xj = zj[1:end-1]; pj = zj[end]
    g = f(xj, pj)
    gz = vcat(g, zj[i] - zpred[i])
    zj+1 = zj - Jfinal \ gz
    return zj+1
end
```

```
newton_step! (generic function with 1 method)
```

And with this, we are ready to compose our corrector function, that takes a guess `zpred` and brings it to a point that is on the bifurcation curve. The keyword arguments help us give a convergence criterion to Newton's algorithm and also catch problematic cases where convergence never happens in time.

```
function corrector(zpred, f, J; δ = 0.9, max_steps = 200, ε = 1e-6, k = 1)
    c = 0
    zj = zpred
    zj+1 = newton_step!(zj, zpred, k, f, J, δ)
    while norm(zj+1 - zj) > ε
        zj = zj+1
        zj+1 = newton_step!(zj, zpred, k, f, J, δ)
        c += 1
        if c > max_steps
            @warn("Newton did not converge.")
            return (zj+1, false)
        end
    end
    return zj+1, true
end
```

corrector (generic function with 1 method)

## Code: Predictor function

Coding the predictor is trivial. If we have no previous entries we start from the initial seed given by the user, otherwise we take do the linear extrapolation discussed above. A user also needs to provide an initial direction  $d\mathbf{z}$  to go towards.

```
function predictor(zs, dz0)
    if length(zs) == 1
        return zs[end]
    elseif length(zs) == 2 # 1 entry is z0, 2nd entry is 1st found fixed point
        return zs[end] .+ dz0
    else
        return 2zs[end] .- zs[end-1]
    end
end
```

predictor (generic function with 1 method)

## Code: Continuation function

Alright, now we can put it all together into the a single "continuation" function.

```
function continuation!(zs, f, J; dz0, pmin, pmax)
    zpred = predictor(zs, dz0)
    (pmin ≤ zpred[end] ≤ pmax) || return false
    z^x, success = corrector(zpred, f, J)
    push!(zs, z^x)
    return success
end

using LinearAlgebra: eigvals

# Continuation loop: do continuation for a given amount of steps
function continuation(f, J, x0, p0; pmin, pmax, dp0, dx0, N = 1000)

    z0 = vcat(x0, p0); zs = [z0]; dz0 = vcat(dx0, dp0)

    ps = [p0]
```

GitHub

```

    success = continuation!(zs, f, J; dz0, pmin, pmax)
    # Stop iteration if we exceed given parameter margins
    success || break
    # Detect stability of found fixed point (needs `Array` coz of StaticArrays.jl)
    eigenvalues = eigvals(J(zs[end][1:end-1], zs[end][end]))
    isstable = maximum(real, eigenvalues) < 0
    push!(stability, isstable)
end
xs = [z[1:end-1] for z in zs]
ps = [z[end] for z in zs]
popfirst!(xs); popfirst!(ps) # remove initial guess
return xs, ps, stability
end
```

continuation (generic function with 1 method)

The code also returns the fixed point stability, although it does assume we have a continuous dynamical system. Adjusting for discrete systems is straightforward.

## Running an example

Let's use the code we have defined to run an example. The following dynamical system has a single saddle node bifurcation, and a fixed point that is "flat" versus the parameter change.

```

function maasch_rule(u, p)
    x, y, z = u
    q, r, s = 1.2, 0.8, 0.8
    dx = -x - y
    dy = -p*z + r*y + s*z^2 - z^2*y
    dz = -q*(x + z)
    return [dx, dy, dz]
end

function maasch_jacob(u, p)
    x, y, z = u
    q, r, s = 1.2, 0.8, 0.8
    return [-1      -1  0;
            0      (r - z^2)  (-p + 2z*s - 2z*y);
            -q      0      -q]
end
```

maasch\_jacob (generic function with 1 method)

We now use it to get the bifurcation curve.

```

pmin = -0.1
pmax = 2
δ = 0.9
p0 = 0.0
x0 = [-1.4, -1.4, -1.4]
dp0 = 0.02
dx0 = [0.01, 0.01, 0.01]

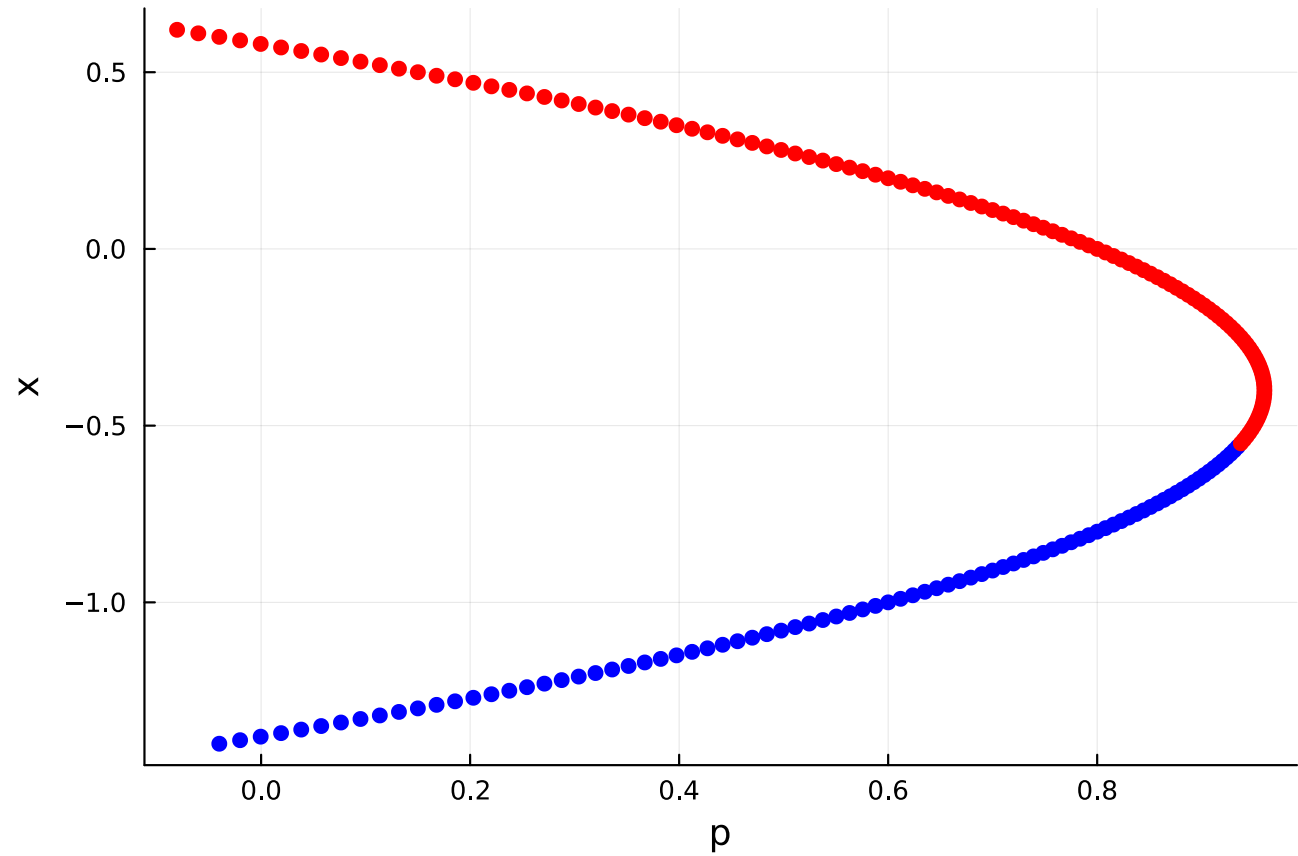
xs, ps, stability = continuation(maasch_rule, maasch_jacob, x0, p0;
    pmin, pmax, dp0, dx0
)
```

(
 [
 [-1.4, 1.4, 1.4],
 [-1.39, 1.39, 1.39],
 [-1.38, 1.38, 1.38],
 [-1.3699999999999999, 1.3699999999999999],
 ...
 ],
 [
 -1.4,
 -1.39,
 -1.38,
 -1.3699999999999999,
 ...
 ],
 [
 0.01,
 0.01,
 0.01,
 0.01,
 ...
 ]
 )

using Plots

color = [s ? :blue : :red for s in stability]

p = scatter(ps, [x[1] for x in xs]; color = colors, markerstrokecolor = colors, xlabel = "p", ylabel = "x")



# Overview of BifurcationKit.jl

- Overview of BifurcationKit.jl
  - Defining Problems
  - Continuing from the initial guess
  - Analyzing the branch

The general workflow for using the package is as follows:

- Define a problem
- Solve the problem
- Analyze the output

## Defining Problems

Problems are specified via a type interface. The problem types are designed to contain the necessary information to fully define their associated continuation method. For example, a bifurcation problem is defined by

$$F(u, pars) = 0$$

with some parameters `pars`, some initial guess `u0`, and scalar parameter axis `lens` contained in `pars`. Therefore the `BifurcationProblem` is defined by those components:

```
prob = BifurcationProblem(F, u0, pars, lens)
```

Note that the number types in the solution will match the types you designate in the problem. However complex types are not advised as they mess up the detection of bifurcation points.

## Continuing from the initial guess

Each type of bifurcation problem has its own problem type which allow the solvers to dispatch to the right methods. The common interface for calling the solvers is:

```
br = continuation(prob, alg; kwargs)
```

Into the command, one passes the bifurcation problem that was defined `prob`, choose an algorithm `alg`, and change the properties of the solver using keyword arguments. The solver returns a branch object `br` which holds all the details for the branch.

## Analyzing the branch

The solution type has a common interface, which makes handling the solution similar between the different types of bifurcation problems. Tools such as interpolations are seamlessly built into the solution interface to make analysis easy. This interface is described in the `ContResult`.

Plotting functionality is provided by a recipe to `Plots.jl`. To use plot branches, simply call the `plot(br)` and the plotter will generate appropriate plots. Plots can be customized using all the keyword arguments provided by `Plots.jl`. Please see `Plots.jl`'s documentation for more information.



---

[GitHub](#)

# Plotting

- [Plotting](#)
  - [Standard plots using the plot recipe from Plots.jl](#)
    - [Specific plotting keyword arguments](#)
    - [Choosing Variables](#)
    - [Plotting bifurcation diagrams](#)
  - [Plotting without the plot recipe](#)
  - [Standard plots using Makie.jl](#)
    - [Example](#)
  - [Plotting eigenvalues](#)

## Standard plots using the plot recipe from Plots.jl

Plotting is provided by calling recipes to `Plots.jl`. It means that to plot a branch `br`, you just need to call

```
# ] add Plots # You need to install Plots.jl before your first time using it!
using Plots
plot(br)
```

where `br` is a branch computed after a call to `br = continuation(...)`. Plots can be customized using [all the keyword arguments provided by Plots.jl](#). For example, we can change the plotting backend to the GR package and put a title on the plot by doing:

```
gr()
plot!(br, title = "I have a branch!")
```

or you can use a scatter plot

```
scatter(br)
```

Then to save the plot, use `savefig`, for example:

```
savefig("myplot.png")
```

## Specific plotting keyword arguments

The available arguments specific to our plotting methods are

- `plotfold = true`: plot the fold points with black dots
- `putspecialptlegend = true`: display the legend corresponding to the bifurcation points
- `vars = nothing`: see below
- `plotstability = true`: display the stability of the branch
- `plotspecialpoints = true`: plot the special (bifurcation) points on the branch
- `branchlabel = "fold branch"`: assign label to a branch which is printed in the legend
- `linewidthunstable`: set the linewidth for the unstable part of the branch
- `linewidthstable`: set the linewidth for the stable part of the branch
- `plotcirclesbif = false` use circles to plot bifurcation points
- `applytoX = identity` apply transformation `applytoX` to x-axis



for making plots:

- `dash_unstable_style = true` use dashed line for plotting the unstable part of the branch

If you have several branches `br1`, `br2`, you can plot them in the same figure by doing

```
plot(br1, br2)
```

in place of

```
plot(br1)
plot!(br2)
```

**! Plot of bifurcation points**

The bifurcation points for which the bisection was successful are indicated with circles and with squares otherwise.

Note that the plot recipes use the parameter axis as `xlabel`, and the passed variable as `ylabel`.

## Choosing Variables

You can select which variables to plot using the keyword argument `vars`, for example:

```
plot(br, vars = (:param, :x))
```

The available symbols are `:x`, `:param`, `:itnewton`, `:itlinear`, `:ds`, `:θ`, `:n_unstable`, `:n_imag`, `:stable`, `:step`,... and:

- `x` if `record_from_solution` (see [continuation](#)) returns a `Number`.
- `x1`, `x2`,... if `record_from_solution` returns a `Tuple`.
- the keys of the `NamedTuple` returned by `record_from_solution`.

The available symbols are provided by calling `propertynames(br.branch)`.

## Plotting bifurcation diagrams

To do this, you just need to call

```
plot(diagram)
```

where `diagram` is a branch computed after a call to `diagram = bifurcationdiagram(...)`. You can use the keywords provided by `Plots.jl` and the different backends. You can thus call `scatter(diagram)`. In addition to the options for plotting branches (see above), there are specific arguments available for bifurcation diagrams

- `code` specify the part of the bifurcation diagram to plot. For example `code = (1,1,)` plots the part after the first branch of the first branch of the root branch.
- `level = (-Inf, Inf)` restrict the branching level for plotting.

## Plotting without the plot recipe

What if you don't want to use `Plots.jl`? You can define your own plotting functions using the internal fields of `br` which is of type `ContResult`. For example, in `PyPlot`, `Gadfly`, `GR`, etc., you can do the following to plot the branch (like the plot recipe `plot(br, vars = (:param, :x))`):

```
plot(br.branch.param, br.branch.x)
```

GitHub

```
col = br.stb ? :green : :red for stb in br.stable]
plot(br.param, br.x, color=col)
```

You can also plot the spectrum at a specific continuation `step::Int` by calling

```
# get the eigenvalues
eigvals = br.eig[step].eigenvals

# plot them in the complex plane
scatter(real.(eigvals), imag.(eigvals))
```

## Standard plots using Makie.jl

Plotting is also provided by calling recipes to `Makie.jl`. It means that to plot a branch `br`, you just need to call

```
# ] add GLMakie # You need to install GLMakie.jl before your first time using it!
using GLMakie
BifurcationKit.plot(br)
```

The keyword arguments to `BifurcationKit.plot` are the same as decribed above in the page. You can also combine diagrams with `BifurcationKit.plot(br1, br2)` or use `BifurcationKit.plot!(ax, br)` to add a branch to an existing plot.

You can also load `CairoMakie.jl` if you prefer

## Example

```
using CairoMakie, BifurcationKit
k = 2
F(x, p) = (@. p + x - x^(k+1)/(k+1))
prob = ODEBifProblem(F, [0.8], 1., (@optic _); record_from_solution = (x,p; k...) -> x[1])
opts = ContinuationPar(dsmax = 0.1, dsmin = 1e-3, ds = -0.001, p_min = -1., p_max = 1.)
br = continuation(prob, PALC(), opts)
f, ax = BifurcationKit.plot(br, dash_unstable_style = true)
f
```

## Plotting eigenvalues

After the computation of a branch `br`, it can be insightful to plot the real part of the eigenvalues along the branch. This can be done using

```
BifurcationKit.plot_eigenvals(br)
```

# Bifurcation problems

- Bifurcation problems
  - Generic bifurcation problem
    - Example
  - Problem modification
    - Example
  - ⚠️⚠️⚠️ Automatic option setting (work in progress) ⚠️⚠️⚠️

The idea behind `BifurcationKit` is to compute bifurcation diagrams in memory limited environments where the device can barely hold the current continuation state. We thus disable by default saving all solutions along the branch and all eigenvectors (see `ContinuationPar` to change this behavior). Still, one needs to save a few solution indicators, like for plotting. This is the reason for the function `record_from_solution` (see below).

## Generic bifurcation problem

`BifurcationProblem` is the basic / generic structure for encoding a bifurcation problem ; it holds the following fields:

- the vector field
- an initial guess
- a set of parameters
- a parameter axis

as well as user defined functions for

- plotting, `plot_solution`
- recording (`record_from_solution`) indicators about the solution when this one is too large to be saved at every continuation step.
- update function `update!`

## Example

```
f(x,p) = @. sin(x * p.a)
u0 = zeros(100_000_000)
params = (a = 1.0, b = 2.0)

# record a few components / indicators about x
myRecord(x,p;k...) = (x1 = x[1], max = maximum(x), nrm = norm(x, Inf))

prob = BifurcationProblem(f, u0, p, (@optic _.a);
    record_from_solution = myRecord
)
```

## Problem modification

In case you want to modify an existing problem, you should use the following method

▼ `BifurcationKit.re_make` — Method

```
re_make(
    prob;
    u0,
```

re\_make(prob; u0 = new\_u0),  
plot\_solution,  
J,  
Jt,  
d2F,  
d3F  
)

This function changes the fields of a problem `::AbstractBifurcationProblem`. For example, you can change the initial condition by doing

re\_make(prob; u0 = new\_u0)

## Example

using BifurcationKit  
F(x,p) = @. p.a + x^2  
# parameters  
par = (a = 0., b = 2)  
prob = BifurcationProblem(F, zeros(3), par, (@optic \_.a))  
# change u0  
prob2 = BifurcationKit.re\_make(prob, u0 = rand(3))

└ Bifurcation problem with uType Vector{Float64}  
└ Inplace: false  
└ Dimension: 3  
└ Symmetric: false  
└ Parameter: a

## ⚠️ Automatic option setting (work in progress) ⚠️

Setting the continuation options can be difficult for new comers. In the case of small ODE, we suggest to use `ODEBifProblem` instead of `BifurcationProblem`. Indeed, in this case, most continuation options will be set up automatically so that very good performance is achieved.

In the case of `BifurcationProblem`, the user has to set up these options manually. This can be useful for large scale problems where the specificities of the problem have to be used.

# Periodic orbits computation

- Periodic orbits computation
  - Trapezoid method
  - Collocation method
  - Shooting method
- Important notes
  - 1. Accessing the periodic orbit
  - 2. Finaliser
  - 3. Record from solution
  - 4. Plot solution
  - 5. Most precise method for Floquet coefficients
  - 6. Misc

Consider the Cauchy problem

$$\frac{du}{dt} = F(u, p).$$

A periodic solution with period  $T$  satisfies

$$\frac{du}{dt} = F(u, p)$$
$$u(0) = u(T).$$

(1)  
(2)

We provide 4 methods for computing periodic orbits (PO):

- one (Trapezoid) based on finite differences to discretize a Cauchy problem,
- one (Collocation) based on orthogonal collocation to discretize a Cauchy problem,
- two (Shooting) based on the flow associated to a Cauchy problem.

It is important to understand the pros and cons of each method to compute PO in large dimensions.

In small dimensions, the collocation method is the preferred choice. It is the most precise, the fastest with dedicated linear solvers, time adaptive and feature full.

## Trapezoid method

The Trapezoid method (or the Collocation one) is usually faster than the ones based on Shooting but it requires more memory as it saves the whole orbit. However the main drawback of this method is that the associated linear solver is not "nice", being composed of a cyclic matrix for which no generic Matrix-free preconditioner is known. Hence, the Trapezoid method is **often used with an ILU preconditioner** which is severely constrained by memory. Also, when the period of the cycle is large, finer time discretization (or mesh adaptation which is not yet implemented) must be employed which is also a limiting factor both in term of memory and preconditioning.

## Collocation method

The Collocation method is (for now) the slowest of the three methods implemented for computing periodic orbits in large dimensions. However, it is by far the most precise one. Additionally, the mesh can be automatically adapted during the continuation. The implementation will be improved for large dimensional systems like the Trapezoid method one.

The methods based on shooting do not share the same drawbacks because the associated linear system is usually well conditioned, at least in the simple shooting case. They are thus often used **without preconditioner at all**. Even in the case of multiple shooting, this can be alleviated by a simple generic preconditioner based on deflation of eigenvalues (see [Linear solvers \(LS\)](#)). Also, the time stepper will automatically adapt to the stiffness of the problem, putting more time points where needed unlike the method based on finite differences which requires an adaptive (time) meshing to provide a similar property. Finally, we can use parallel Shooting to greatly decrease the computation time.

The main drawback of the method is to find a fast time stepper, at least to compete with the method based on finite differences. The other drawback is the precision of the method which cannot compete with the collocation one.

## Important notes

We regroup here some important notes which are valid for all methods above.

### 1. Accessing the periodic orbit

In `record_from_solution`, `plot_solution` or after the computation of a branch of periodic orbits, how do I obtain the periodic orbit, for plotting purposes for example? If `x` is the solution from `newton` for the parameter `p`, you can obtain the periodic orbit as follows

```
x_tt = get_periodic_orbit(x, p)
```

where `x_tt.t` contains the time mesh and `x_tt[:, :]` contains the different components. Note that for Trapezoid and collocation methods, calling `get_periodic_orbit` is essentially free as it is a reshape of `x`. However, in the case of Shooting methods, this requires recomputing the periodic orbit which can be costly for large scale problems.

### 2. Finaliser

If you pass a `finalise_solution` function to [continuation](#), the following occurs:

- 1. If the newton solve was successful, we update the phase condition every `update_section_every_step`
- 2. we call the user defined finalizer `finalise_solution`

### 3. Record from solution

You can pass your own function to [continuation](#). In the particular case of periodic orbits, the method is called like `record_from_solution(x, opt; k...)` where `opt.p` is the current value of the continuation parameter and `opt.prob` is the current state of the continuation problem. You can then obtain the current periodic orbit using (see above)

```
x_tt = get_periodic_orbit(x, opt.p)
```

### 4. Plot solution

Similarly to `record_from_solution`, the method is called like `plot_solution(x, opt; k...)` where `opt.p` is the current value of the continuation parameter and `opt.prob` is the current state of the continuation problem.

### 5. Most precise method for Floquet coefficients

The state of the art method is based on a Periodic Schur decomposition. It is available through the package [PeriodicSchurBifurcationKit.jl](#). For more information, have a look at `FloquetPQZ`.

### 6. Misc

`set_params_po` `generate_ci_problem` `generate_solution`





# Periodic orbits based on Trapezoidal rule

- Periodic orbits based on Trapezoidal rule
  - Encoding of the functional
  - Preconditioning
  - Linear solvers
    - 1. FullLU
    - 2. FullSparseInplace
    - 3. Dense
    - 4. FullMatrixFree
    - 5. BorderedLU
    - 6. BorderedSparseInplace
    - 7. BorderedMatrixFree
  - Floquet multipliers computation
    - Default method
    - Most precise method
  - Computation with `newton`
  - Computation with `newton` and deflation
  - Continuation
  - References

The Trapezoid method allows to compute periodic orbits by discretizing time using Finite Differences based on a trapezoidal rule . The method is implemented in the structure `Trapeze`. The general method is very well exposed in [\[Uecker\]](#), [\[Lust\]](#), [\[Keller\]](#) and we adopt the notations of the first reference.

We look for periodic orbits as solutions  $(x(0), T)$  of

$$M_a \dot{x} = T \cdot F(x), \; x(0) = x(1) \in \mathbb{R}^n \tag{1}$$

where  $M_a$  is a mass matrix (default is the identity one).

In order to have a unique solution, we need to remove the phase freedom. This is done by imposing a *phase* condition

$$\frac{1}{T} \int_0^T \langle x(s) - x_\pi(s), \phi(s) \rangle ds \approx \frac{1}{m} \sum_{i=1}^m \langle x_i - x_{\pi,i}, \phi_i \rangle = 0$$

for some  $x_\pi, \phi$  which are chosen (wisely).

We note  $m$  the number of time slices of the periodic orbit. By discretizing the above problem, we obtain

$$\begin{aligned} 0 &= M_a (x_j - x_{j-1}) - \frac{h}{2} (F(x_j) + F(x_{j-1})) \equiv G_j(x), \quad j = 1, \dots, m-1 \\ x_0 &= x_{m-1} \\ 0 &= x_m - x_1 \equiv G_m(x) \\ 0 &= \sum_{i=1}^m \langle x_i - x_{\pi,i}, \phi_i \rangle = 0 \end{aligned}$$

where  $x_0 = x_m$  and  $h = T/m$ . In view of the Newton method, we study the jacobian of the above system. The Jacobian w.r.t.  $(x_0, T)$  is given by



where [GitHub](#)

$$A_\gamma := \begin{pmatrix} M_1 & 0 & 0 & 0 & \cdots & -H_1 & 0 \\ -H_2 & M_2 & 0 & 0 & \cdots & 0 & 0 \\ 0 & -H_3 & M_3 & 0 & \cdots & 0 & 0 \\ \vdots & \cdots & \ddots & \ddots & \ddots & \vdots & \vdots \\ 0 & \cdots & \cdots & \ddots & \ddots & 0 & 0 \\ 0 & \cdots & \cdots & 0 & -H_{m-1} & M_{m-1} & 0 \\ -\gamma I & 0 & \cdots & \cdots & \cdots & 0 & I \end{pmatrix}$$

with  $M_i := M_a - \frac{h}{2}dF(x_i)$  and  $H_i := M_a + \frac{h}{2}dF(x_{i-1})$ .

We solve the linear equation  $\mathcal{J} \cdot sol = rhs$  with a bordering strategy (i.e. the linear solver is a subtype of `<`: `AbstractBorderedLinearSolver`) which in turn requires to solve  $A_\gamma z = b$  where  $z = (x, x_m)$ . We also solve this equation with a bordering strategy but this time, it can be simplified as follows. If we write  $b = (f, g)$ , one gets  $J_c x = f$  and  $x_m = g + \gamma x_1$  where  $x_1$  is the first time slice of  $x$  and  $J_c$  is the following **cyclic matrix**:

$$J_c := \begin{pmatrix} M_1 & 0 & 0 & 0 & \cdots & -H_1 \\ -H_2 & M_2 & 0 & 0 & \cdots & 0 \\ 0 & -H_3 & M_3 & 0 & \cdots & 0 \\ \vdots & \cdots & \ddots & \ddots & \ddots & \vdots \\ 0 & \cdots & \cdots & \ddots & \ddots & 0 \\ 0 & \cdots & \cdots & 0 & -H_{m-1} & M_{m-1} \end{pmatrix}$$

Our code thus provides methods to invert  $J_c$  and  $A_\gamma$  using a sparse solver or a Matrix-Free one. A preconditioner can be used.

## Encoding of the functional

The functional is encoded in the composite type [Trapeze](#). See the link for more information, in particular on how to access the underlying functional, its jacobian and other matrices related to it like  $A_\gamma, J_c$ ...

## Preconditioning

We strongly advise you to use a preconditioner to deal with the above linear problem. See [2d Ginzburg-Landau equation \(finite differences, codim 2, Hopf aBS\)](#) for an example.

## Linear solvers

We provide many different linear solvers to take advantage of the formulations. These solvers are available through the argument `jacobian` in the constructor of `Trapeze`. For example, you can pass `jacobian = FullLU()`. Note that all the internal solvers and Jacobians are set up automatically, you don't need to do anything. However, for the sake of explanation, we detail how this works.

### 1. FullLU

When using `jacobianP0 = FullLU()`, this triggers the computation of  $\mathcal{J}$  as in (2) at each step of newton/continuation. The Jacobian matrix  $\mathcal{J}$  is stored a `SparseArray`. This can be quite costly flow large  $n$  (see (1)). This Jacobian is often used with the the linear solver `DefaultLS()`.

### 2. FullSparseInplace

Same as `FullLU()` but the Jacobian is allocated only once and updated inplace. This is much faster than `FullLU()` but the sparsity pattern of `dF` must be constant.

Same as `FullSparseInplace()` above but the matrix `dG` is dense. It is also updated inplace. This is useful to study ODE of small dimension.

## 4. FullMatrixFree

A matrix free linear solver is used for  $\mathcal{J}$ : note that a preconditioner is very likely required here because of the cyclic shape of  $\mathcal{J}$  which affects negatively the convergence properties of iterative solvers. Note that  $\mathcal{J}$  is never formed in this case.

## 5. BorderedLU

For `BorderedLU()`, we take advantage of the bordered shape of the linear solver and use a LU decomposition to invert `dG` using a bordered linear solver. More precisely, the bordered structure of  $\mathcal{J}$  is stored using the internal structure `POTrapJacobianBordered`. Then,  $\mathcal{J}$  is inverted using the custom bordered linear solver `PeriodicOrbitTrapBLS` which is based on the bordering strategy (see [Bordered linear solvers \(BLS\)](#)). This particular solver is based on an explicit formula which only requires to invert  $A_\gamma$ : this is done by the linear solver `AyLinearSolver`. In a nutshell, we have:

```
PeriodicOrbitTrapBLS = BorderingBLS(solver = AyLinearSolver(), check_precision = false)
```

## 6. BorderedSparseInplace

Same as `BorderedLU()` but the Jacobian is allocated only once and updated inplace. This is much faster than `BorderedLU()` but the sparsity pattern of `dF` must be constant.

## 7. BorderedMatrixFree

A matrix free linear solver is used but for  $\mathcal{J}_c$  only: it means that `options.linsolver` is used to invert  $\mathcal{J}_c$ .

 **Matrix-Free**

These two Matrix-Free options, `MatrixFree()` and `BorderedMatrixFree()`, thus expose different part of the Jacobian  $\mathcal{J}$  in order to use specific preconditioners. For example, an ILU preconditioner on  $\mathcal{J}_c$  could remove the constraints in  $\mathcal{J}$  and lead to poor convergence. Of course, for these last two methods, a preconditioner is likely be required.

# Floquet multipliers computation

## Default method

A **not very precise** algorithm for computing the Floquet multipliers is provided in the package. The method, dubbed Quick and Dirty (QaD), is not numerically very precise for large / small Floquet exponents because it relies on constructing the monodromy matrix.

Note that the computation of the eigenvalues can be iterative or direct based on the eigensolver passed in arguments.

It amounts to computing the eigenvalues of the monodromy matrix

$$\mathcal{M} = M_1^{-1}H_1M_{m-1}^{-1}H_{m-1}\cdots M_2^{-1}H_2.$$

The method allows, nevertheless, to detect bifurcations of periodic orbits. It seems to work reasonably well for the tutorials considered here. For more information, have a look at [FloquetQaD](#).

## Most precise method

The state of the art method is based on a Periodic Schur decomposition. It is available through the package [PeriodicSchurBifurcationKit.jl](#). For more information, have a look at [FloquetPQZ](#).

We provide a simplified call to `newton` to locate the periodic orbits. Compared to the regular `newton` function, there is an additional option `linearalgo` to select one of the many ways to deal with the above linear problem. The default solver `linearalgo` is ``BorderedLU()`.

Have a look at the [Periodic orbits based on Trapezoidal rule](#) example for the Brusselator for a basic example and at [2d Ginzburg-Landau equation](#) for a more advanced one.

The docs for this specific `newton` are located at [newton](#).

## Computation with `newton` and deflation

We also provide a simplified call to `newton` to locate the periodic orbit with a deflation operator.

### ▼ `BifurcationKit.newton` — Method

```
newton(  
    trap::Trapeze,  
    orbitguess,  
    defOp::DeflationOperator{Tp, Tdot, T, vectype},  
    options::NewtonPar;  
    kwargs...  
) -> NonLinearSolution{_  
A, Tprob, Tres} where {_  
A, Tprob<:(DeflatedProblem{Tprob, Tp, _  
A, T, _B, Val{
```

This function is similar to `newton(probP0, orbitguess, options, jacobianP0; kwargs...)` except that it uses deflation in order to find periodic orbits different from the ones stored in `defOp`. We refer to the mentioned method for a full description of the arguments. The current method can be used in the vicinity of a Hopf bifurcation to prevent the Newton-Krylov algorithm from converging to the equilibrium point.

## Continuation

Have a look at the [Periodic orbits based on Trapezoidal rule](#) example for the Brusselator. We refer to [continuation](#) for more information regarding the arguments.

### ▼ `BifurcationKit.continuation` — Method

```
continuation(  
    trap::Trapeze,  
    orbitguess,  
    alg::BifurcationKit.AbstractContinuationAlgorithm,  
    _contParams::ContinuationPar;  
    959,  
    record_from_solution,  
    linear_algo,  
    kwargs...  
) -> Any
```

This is the continuation routine for computing a periodic orbit using a functional `G` based on finite differences and a Trapezoidal rule.

#### Arguments

- `prob::Trapeze` encodes the functional `G`.
- `orbitguess` a guess for the periodic orbit. See [Trapeze](#) for more details.
- `alg` continuation algorithm.
- `contParams` same as for the regular [continuation](#) method.

#### Keyword arguments

GitHub

specify the choice of the jacobian (and linear algorithm), jacobian must belong to `(BifurcationKit.Dense(), BifurcationKit.AutoDiffDense(), BifurcationKit.FullLU(), BifurcationKit.FullMatrixFree(), BifurcationKit.BorderedLU(), BifurcationKit.BorderedMatrixFree(), BifurcationKit.FullSparseInplace(), BifurcationKit.BorderedSparseInplace(), BifurcationKit.AutoDiffMF())`. This is used to select a way of inverting the jacobian `dG` of the functional `G`.

- For `jacobian = FullLU()`, we use the default linear solver based on a sparse matrix representation of `dG`. This matrix is assembled at each newton iteration. This is the default algorithm. Can be used when the sparsity can change.
- For `jacobian = FullSparseInplace()`, this is the same as for `FullLU()` but the sparse matrix `dG` is updated inplace. This method allocates much less. In some cases, this is significantly faster than using `FullLU()`. Note that this method can only be used if the sparsity pattern of the jacobian is always the same.
- For `jacobian = Dense()`, same as above but the matrix `dG` is dense. It is also updated inplace. This option is useful to study ODE of small dimension.
- For `jacobian = AutoDiffDense()`, evaluate the jacobian using `ForwardDiff`
- For `jacobian = BorderedLU()`, we take advantage of the bordered shape of the linear solver and use a LU decomposition to invert `dG` using a bordered linear solver.
- For `jacobian = BorderedSparseInplace()`, this is the same as for `BorderedLU()` but the cyclic matrix `dG` is updated inplace. This method allocates much less. In some cases, this is significantly faster than using `:BorderedLU`. Note that this method can only be used if the sparsity pattern of the jacobian is always the same.
- For `jacobian = FullMatrixFree()`, a matrix free linear solver is used for `dG`: note that a preconditioner is very likely required here because of the cyclic shape of `dG` which affects negatively the convergence properties of GMRES.
- For `jacobian = BorderedMatrixFree()`, a matrix free linear solver is used but for `Jc` only (see docs): it means that `options.linsolver` is used to invert `Jc`. These two Matrix-Free options thus expose different part of the jacobian `dG` in order to use specific preconditioners. For example, an ILU preconditioner on `Jc` could remove the constraints in `dG` and lead to poor convergence. Of course, for these last two methods, a preconditioner is likely to be required.
- For `jacobian = AutoDiffMF()`, the evaluation map of the differential is derived using automatic differentiation. Thus, unlike the previous two cases, the user does not need to pass a Matrix-Free differential.

Note that by default, the method prints the period of the periodic orbit as function of the parameter. This can be changed by providing your `record_from_solution` argument.

## References

- Uecker
- Lust
- Keller

Uecker, Hannes. Hopf Bifurcation and Time Periodic Orbits with Pde2path – Algorithms and Applications. Communications in Computational Physics 25, no. 3 (2019)

Lust, Kurt, Numerical Bifurcation Analysis of Periodic Solutions of Partial Differential Equations, PhD thesis, 1997.

Keller, Herbert B. “Accurate Difference Methods for Nonlinear Two-Point Boundary Value Problems.” SIAM Journal on Numerical Analysis 11, no. 2 (April 1974): 305–20. <https://doi.org/10.1137/0711028>.

# Periodic orbits based on orthogonal collocation

- [Periodic orbits based on orthogonal collocation](#)
  - [Number of unknowns](#)
  - [Phase condition](#)
  - [Precision of the scheme](#)
  - [Discretization of the BVP and jacobian](#)
  - [Interpolation](#)
  - [Mesh adaptation](#)
  - [Encoding of the functional](#)
  - [Jacobians](#)
  - [Linear solvers](#)
  - [Floquet multipliers computation](#)
  - [Computation with `newton`](#)
  - [Continuation](#)
  - [Examples of optimized use](#)
    - [Inplace jacobian](#)
    - [Optimized linear solver](#)
    - [Optimized Floquet computation](#)
  - [References](#)

We compute `Ntst` time slices of a periodic orbit using orthogonal collocation. This is implemented in the structure `Collocation`.

## ! Large scale

The current implementation is optimized for ODE and for large scale problems for which the jacobian is sparse.

## ! small scale

The current implementation is very optimized for ODE, see inplace `BifurcationProblem` and `COPBLS()`.

The general method is very well exposed in [\[Dankowicz\]](#), [\[Doedel\]](#) and we adopt the notations of [\[Dankowicz\]](#). However our implementation is based on [\[Doedel\]](#) because it is more economical (less equations) when it enforces the continuity of the solution.

We look for periodic orbits as solutions  $(x(0), T)$  of

$$\dot{x} = T \cdot F(x), \ x(0) = x(1) \in \mathbb{R}^n.$$

We focus on the differential equality and consider a partition of the time domain

$$0 = \tau_1 < \cdots < \tau_j < \cdots < \tau_{N_{tst}+1} = 1$$

where the points are referred to as **mesh points**. On each mesh interval  $[\tau_j, \tau_{j+1}]$  for  $j = 1, \cdots, N_{tst}$ , we define the affine transformation

$$\tau = \tau^{(j)}(\sigma) := \tau_j + \frac{(1 + \sigma)}{2} (\tau_{j+1} - \tau_j), \sigma \in [-1, 1].$$

GitHub

$$\dot{x}^{(j)} = T \frac{\tau_{j+1} - \tau_j}{2} \cdot F(x^{(j)})$$

$(E_j)$

with the continuity equation  $x^{(j+1)}(-1) = x^{(j)}(1)$ .

We now aim at solving  $(E_j)$  by using an approximation with a polynomial of degree  $m$ . Following [\[Dankowicz\]](#), we define a (uniform) partition:

$$-1 = \sigma_1 < \dots < \sigma_i < \dots < \sigma_{m+1} = 1.$$

The points  $\tau_{i,j} = \tau^{(i)}(\sigma_j)$  are called the **base points**: they serve as collocation points.

The associated  $m + 1$  Lagrange polynomials of degree  $m$  are:

$$\mathcal{L}_i(\sigma) := \prod_{k=1, k \neq i}^{m+1} \frac{\sigma - \sigma_k}{\sigma_i - \sigma_k}, i = 1, \dots, m + 1.$$

We then introduce the approximation  $p_j$  of  $x^{(j)}$ :

$$p_j(\sigma) \equiv \sum_{k=1}^{m+1} \mathcal{L}_k(\sigma) x_{j,k}$$

and the problem to be solved at the **collocation nodes**  $z_l, l = 1, \dots, m$ :

$$\forall 1 \leq l \leq m, \quad 1 \leq j \leq N_{tst}, \quad \dot{p}_j(z_l) = T \frac{\tau_{j+1} - \tau_j}{2} \cdot F(p_j(z_l)).$$

$(E_j^2)$

The **nodes**  $(z_l)$  are associated with a Gauss–Legendre quadrature.

In order to have a unique solution, we need to remove the phase freedom. This is done by imposing a *phase* condition.

We have  $p_j(\sigma_k) = x_{j,k}$

## Number of unknowns

Putting the period unknown aside, we have to find the  $x_{j,k}$  which gives  $n \times N_{tst} \times (m + 1)$  unknowns.

The equations  $E_j^2$  provides  $n \times N_{tst} \times m$  plus the  $(N_{tst} - 1) \times n$  equations for the continuity equations. This makes a total of  $(N_{tst} - 1) \times m \times n + n \times N_{tst} \times m = n[N_{tst}(m + 1) - 1]$  equations to which we add the  $n$  equations for the periodic boundary condition. In total, we have

$$n \times N_{tst} \times (m + 1)$$

equations which matches the number of unknowns.

## Phase condition

To ensure uniqueness of the solution to the functional, we use the following phase condition

$$\frac{1}{T} \int_0^T \langle x(s), \dot{x}_0(s) \rangle ds = 0$$

During continuation at step  $k$ , we use  $\frac{1}{T} \int_0^T \langle x(s), \dot{x}_{k-1}(s) \rangle ds$



It is known that the error between the true periodic orbit  $\gamma^*$  and the approximated one  $\gamma$  scales as

GitHub

$$\sup_i \|\gamma^*(t_i) - \gamma(t_i)\| = O(h^{2m+1}).$$

## Discretization of the BVP and jacobian

We only focus on the differential part. Summing up, we obtained the following equations for the  $x_{j,l} \in \mathbb{R}^n$ :

$$\sum_{k=1}^{m+1} \mathcal{L}'_k(z_l)x_{j,k} = F\left(\sum_{k=1}^{m+1} \mathcal{L}_k(z_l)x_{j,k}\right)$$

The jacobian in the case  $m = 2$  is given by:

$$\begin{pmatrix} x_{0,0} & x_{0,1} & x_{1,0} & x_{1,1} & x_{2,0} & x_{2,1} & x_{3,0} & \mathbf{T} \\ H_{0,0}^0 & H_{0,1}^0 & H_{1,0}^0 & & & & & * \\ H_{0,0}^1 & H_{0,1}^1 & H_{1,0}^1 & & & & & * \\ & & H_{1,0}^0 & H_{1,1}^0 & H_{2,0}^0 & & & * \\ & & H_{1,0}^1 & H_{1,1}^1 & H_{2,0}^1 & & & * \\ & & & & H_{2,0}^0 & H_{2,1}^0 & H_{3,0}^0 & * \\ & & & & H_{2,0}^1 & H_{2,1}^1 & H_{3,0}^1 & * \\ & & & & & & & * \\ -I & & & & & & I & * \\ * & * & * & * & * & * & * & * \end{pmatrix}$$

where

$$H_{k,l}^{l_2} = \mathcal{L}'_{l_2,l} \cdot I_n - T \frac{\tau_{j+1} - \tau_j}{2} \cdot \mathcal{L}_{l_2,l} \cdot dF(x_{k,l}) \in \mathbb{R}^n.$$

## Interpolation

! Missing docstring.

Missing docstring for BifurcationKit.POSolution. Check Documenter's build log for details.

## Mesh adaptation

The goal of this method<sup>[Russell]</sup> is to adapt the mesh  $\tau_i$  in order to minimize the error. It is particularly helpful near homoclinic solutions where the period diverges. It can also be useful in order to use a smaller  $N_{tst}$ .

## Encoding of the functional

The functional is encoded in the composite type `Collocation`. See the link for more information, in particular on how to access the underlying functional, its jacobian...

## Jacobians

We provide many different jacobians to take advantage of the problem or the dimensionality. These jacobians are available through the argument `jacobian` in the constructor of `Collocation`. For example, you can pass `jacobian = FullSparse()`. Note that all the internal linear solvers and jacobians are set up automatically so you don't need to do anything. However, for the sake of explanation, we detail how this works.

1. DenseAnalytical()

### 1. DenseMatrixTriedInplace()

[GitHub](#)  
Same as 1. but caches more information to limit allocations. (Meant to become the default jacobian in the future).

1. AutoDiffDense()

The jacobian is computed with automatic differentiation, it works for dense matrices. Can be used for debugging.

1. FullSparse()

The jacobian is computed with an analytical formula, it works for sparse matrices.

1. FullSparseInplace()

The sparse jacobian is computed in place, limiting memory allocations, with an analytical formula when the sparsity of the jacobian of the vector field is constant. This is much faster than FullSparse().

## Linear solvers

When the jacobian is dense and dimension is small  $\sim O(10)$ , you should use condensation of parameters COPBLS for performance.

1. DefaultLS() can be used for most jacobians
2. COPLS() or COPBLS() which is the method of **condensation of parameters** (COP) implemented in Auto-07p [\[Govaerts\]](#). For this to be most efficient, the vector field must be written in non-allocating form. In this case, this is almost 10x faster than using DefaultLS() for low dimensional ODEs (say  $\text{dim} = O(10)$ ). The performances are closing on to that of Auto-07p.
3. You can use bordered linear solvers in large dimensions to take advantage of the specific shape of the jacobian. See also Trapezoid method for additional information.

## Floquet multipliers computation

We provide three methods to compute the Floquet coefficients.

1. The algorithm (Default) FloquetColl is based on the method of condensation of parameters (COP) described in [\[Doedel\]](#). It is the fastest method. It can also be used for sparse jacobians.
2. The algorithm FloquetCollGEV is a simplified version of the procedure described in [\[Fairgrieve\]](#). It boils down to solving a large generalized eigenvalue problem. There is clearly room for improvements here but this can be used to check the results of the previous method.

These methods allow to detect bifurcations of periodic orbits. It seems to work reasonably well for the tutorials considered here. However they may be imprecise [\[Lust\]](#).

1. The state of the art method is based on a Periodic Schur decomposition. It is available through the package [PeriodicSchurBifurcationKit.jl](#). For more information, have a look at FloquetPQZ.

## Computation with newton

▼ [BifurcationKit.newton](#) — Method

```
newton(  
    coll::Collocation,  
    orbitguess,  
    options::NewtonPar;  
    kwargs...  
) -> NonLinearSolution{_A, Tprob, Tres} where {_A, Tprob<:(BifurcationKit.PeriodicOrbitFunctionalColl
```

This is the Newton solver for computing a periodic orbit using orthogonal collocation method. Note that the linear solver has to be appropriately set up in options.



Similar to `newton` except that `prob` is a `Collocation`.

- GitHub
- `prob` a problem of type `<: Collocation` encoding the shooting functional `G`.
  - `orbitguess` a guess for the periodic orbit.
  - `options` same as for the regular `newton` method.

Optional argument

- `jacobian` Specify the choice of the linear algorithm, which must belong to `(AutoDiffDense(), )`. This is used to select a way of inverting the jacobian `dG`
  - For `AutoDiffDense()`. The jacobian is formed as a dense `Matrix`. You can use a direct solver or an iterative one using `options`. The jacobian is formed inplace.
  - For `DenseAnalytical()` Same as for `AutoDiffDense` but the jacobian is formed using a mix of AD and analytical formula.

We provide a simplified call to `newton` to locate the periodic orbits. `newton` will look for `prob.jacobian` in order to select the requested way to compute the jacobian.

The docs for this specific `newton` are located at [newton](#).

## Continuation

We refer to [continuation](#) for more information regarding the arguments. `continuation` will look for `prob.jacobian` in order to select the requested way to compute the jacobian.

## Examples of optimized use

We show in this part how to optimize the use of collocation for small systems. We consider a small dynamical system which has a Hopf bifurcation and we compute the branch of periodic orbits from the Hopf bifurcation.

The basic code is as follows (see Tutorials).

```
using BifurcationKit, BenchmarkTools

@inbounds function auto_cat!(dz, z, p, t = 0)
    (;κ, μ, σ, δ) = p
    a,b,c = z
    ab2 = a*b^2
    dz[1] = μ *(κ+c) - ab2 - a
    dz[2] = (ab2 + a - b)/σ
    dz[3] = (b-c)/δ
    dz
end

prob = ODEBifProblem(auto_cat!, [1.,0,1], (κ = 65., μ = 0.01, σ = 5e-3, δ = 2e-2 ) , (@optic _..μ);
    record_from_solution = (x, p; k...) -> (E = x[1], x = x[2], u = x[3]),
    )

opts_br = ContinuationPar(p_min = -0.0, p_max = .19, ds = 0.04, dsmax = 0.1, n_inversion = 8, detect_bifurcation = true)
br = continuation(prob, PALC(tangent=Bordered()), opts_br; plot = false, normC = norminf, bothside = true)
```

- Curve type: **EquilibriumCont**
- Number of points: 95
- Type of vectors: **Vector{Float64}**
- Parameter **μ** starts at -0.0, ends at 0.19
- Algo: **PALC [Bordered]**
- Special points:

```
...
- # 4, endpoint at  $\mu \approx +0.19000000$ ,
GitHub
```

```
# collocation method to discretize the periodic orbit
po_method = Collocation(80, 4,
    jacobian = BifurcationKit.DenseAnalyticalInplace(),
);

opts_po_cont = ContinuationPar(dsmax = 0.03,
    ds= -0.001,
    dsmin = 1e-4,
    p_max = 0.8,
    p_min=-5.,
    max_steps = 280,

    newton_options = NewtonPar(tol = 1e-9, max_iterations = 4),
    nev = 3,
    tol_stability = 1e-3,
    detect_bifurcation = 3,
    plot_every_step = 20,
    save_sol_every_step = 1,
    detect_fold = false,
    n_inversion = 6)

br_po = @benchmark continuation(
    br, 3, opts_po_cont,
    po_method,
    alg = PALC(),
     $\delta p$  = 0.001,
    normC = norminf,
    # under-optimized Floquet computation. ~Slow but general.
    eigsolver = BifurcationKit.FloquetColl(small_n = false),
)
```

```
BenchmarkTools.Trial: 1 sample with 1 evaluation per sample.
Single result which took 9.078 s (1.89% GC) to evaluate,
with a memory estimate of 14.40 GiB, over 2727983 allocations.
```

The slow running time comes from `\` used to invert the collocation jacobian.

## Inplace jacobian

The first optimization is to pass an inplace jacobian of the vector field. Right now, `ForwardDiff.jl` provides this in `BifurcationProblem` but it is not very optimized.

```
@inbounds function auto_cat_jac!(J, z, p, t = 0)
    (; $\kappa$ ,  $\mu$ ,  $\sigma$ ,  $\delta$ ) = p
    a,b,c = z
    ab = 2*a*b
    J[1,1] = -1-(b^2)
    J[1,2] = -ab
    J[1,3] =  $\mu$ 
    J[2,1] = (1+b^2) /  $\sigma$ 
    J[2,2] = (-1+ab) /  $\sigma$ 
    # J[2,3] = 0
    # J[3,1] = 0
    J[3,2] = 1 /  $\delta$ 
    J[3,3] = -1 /  $\delta$ 
    J
end
```

GitHub

```
    J! = auto_cat_jac!,
    )

br = continuation(prob, PALC(tangent=Bordered()), opts_br; plot = false, normC = norminf, bothside = true)

br_po = @benchmark continuation(
    br, 3, opts_po_cont,
    po_method,
    alg = PALC(),
    δp = 0.001,
    normC = norminf,
    # under-optimized Floquet computation. ~Slow but general.
    eigsolver = BifurcationKit.FloquetColl(small_n = false),
    )
```

BenchmarkTools.Trial: 1 sample with 1 evaluation per sample.  
Single result which took 8.990 s (1.70% GC) to evaluate,  
with a memory estimate of 14.19 GiB, over 167663 allocations.

This is not a large gain.

## Optimized linear solver

The next optimization is to used the specific linear solver based on condensation of parameters:

```
br_po = @benchmark continuation(
    br, 3, opts_po_cont,
    po_method,
    alg = PALC(),
    δp = 0.001,
    linear_algo = BifurcationKit.COPBLS(),
    eigsolver = BifurcationKit.FloquetColl(small_n = false),
    normC = norminf,
    )
```

BenchmarkTools.Trial: 8 samples with 1 evaluation per sample.  
Range (min ... max): 570.233 ms ... 1.174 s | GC (min ... max): 2.88% ... 46.74%  
Time (median): 578.114 ms | GC (median): 3.65%  
Time (mean ± σ): 655.598 ms ± 209.907 ms | GC (mean ± σ): 13.09% ± 15.34%



Memory estimate: 2.26 GiB, allocs estimate: 165977.

This is massive gain, almost 10x faster!

## Optimized Floquet computation

```
br_po = @benchmark continuation(
    br, 3, opts_po_cont,
    po_method,
    alg = PALC(),
    δp = 0.001,
    # verbosity = 3,
    linear_algo = BifurcationKit.COPBLS(),
    eigsolver = BifurcationKit.FloquetColl(small_n = true),
    )
```



# Continuation methods: introduction

- Continuation methods: introduction
  - Natural continuation
  - Step size control
  - References

The goal of these methods<sup>[Kuz] [Govaerts] [Rabinowitz] [Mei] [Keller]</sup> is to find solutions  $x \in \mathbb{R}^n$  to nonlinear equations

$$\mathbb{R}^n \ni F(x, p) = 0 \tag{E}$$

as function of a real parameter  $p$ . Given a known solution  $(x_0, p_0)$ , we can, under reasonable assumptions, continue it by computing a 1d curve of solutions  $\gamma = (x(s), p(s))_{s \in I}$  passing through  $(x_0, p_0)$ .

For the sequel, it is convenient to use the following formalism<sup>[Kuz]</sup>

- prediction of the next point
- correction
- step size control.

## Natural continuation

More information is available at [Predictors - Correctors](#)

We just use this simple continuation method to give a **trivial** example of the formalism. Knowing  $(x_0, p_0)$ , we form the predictor  $(x_0, p_0 + ds)$  for some  $ds$  and use it as a guess for a Newton corrector applied to  $x \rightarrow F(x, p_0 + ds)$ . The corrector is thus the **Newton** algorithm.

This continuation method is `continuation(prob, Natural(), options)`.

### Usage

You should almost never use this predictor for computations. It fails at turning points, is not adaptive, ...

## Step size control

Each time the corrector phase failed, the step size  $ds$  is halved. This has the disadvantage of having **unnecessary** Newton iterations (which costs time) and imposing small steps (which can be slow as well). To prevent this, the step size can be controlled internally with the idea of having a constant number of Newton iterations per point. This is in part controlled by the aggressiveness factor `a` in `ContinuationPar`.

## References

- Kuz

Kuznetsov, Elements of Applied Bifurcation Theory.
- Govaerts

Govaerts, Numerical Methods for Bifurcations of Dynamical Equilibria; Allgower and Georg, Numerical Continuation Methods

Krasovskii, Applications of Bifurcation Theory, Danilowicz and Schmelzer, Recipes for Continuation

- [GitHub](#)

Mei, Numerical Bifurcation Analysis for Reaction-Diffusion Equations

- [Keller](#)

Keller, Lectures on Numerical Methods in Bifurcation Problems



# Predictors - Correctors

- Predictors - Correctors
  - 1. Natural, zeroth order predictor
  - 2. First order predictor
    - 2a. Secant predictor
    - 2b. Bordered predictor
  - 3. Polynomial predictor
  - 4. Multiple predictor (aka pmcont in pde2path)
  - References

The continuation method works with the following pattern (see [\[Allgower1990\]](#)):

1. compute **tangent**
2. call **predictor** (based on tangent, mostly)
3. call **corrector**

There are several couples predictor-tangent/corrector which can be used in **BifurcationKit.jl** as we now explain. The tangent computation is formally included in the predictor whereas it is a distinct function in the code.

 **Corrector**

Note that setting the predictor also sets the corresponding corrector: it selects the couple predictor-corrector. You don't have (in fact cannot) set them independently.

## 1. Natural, zeroth order predictor

This is the dumbest predictor based on the formula  $(x_1, p_1) = (x_0, p_0 + ds)$  with Newton corrector ; it fails at Turning points. This is set by the algorithm `Natural()` in [continuation](#). For matrix based jacobian, it is not faster than the pseudo-arclength predictor because the factorisation of the jacobian is cached. For Matrix-free methods, this predictor can be faster than the following ones until it hits a Turning point.

## 2. First order predictor

This predictor is based on a computation of the tangent  $\tau = (dx, dp)$  to the curve of solutions, it is given by  $(x_1, p_1) = (x_0, p_0) + ds \cdot \tau$ . This predictor passes Turning points when used with PALC Newton corrector. **BifurcationKit.jl** provides two ways to compute the tangent  $(dx, dp)$ .

### 2a. Secant predictor

This predictor is called **secant** and is parametrized by the algorithm `PALC(tangent = Secant())` in [continuation](#) with [Secant](#) . It is computed with  $\tau = (x_1, p_1) - (x_0, p_0)$  and normalized with the norm  $\|(x, p)\|_\theta^2 := \frac{\theta}{length(x)} \langle x, x \rangle + (1 - \theta) \cdot p^2$  for some  $0 < \theta < 1$ .

 **Parameter `θ`**

The parameter  $\theta$  in the norm above (see also the struct `ContinuationPar`) is very important. It should be tuned for the continuation to work properly especially in the case of large problems where the  $\langle x - x_0, dx_0 \rangle$  component in the [Pseudo arclength continuation



GitHub

## 2b. Bordered predictor

This predictor departs from the previous one in the way the tangent  $\tau$  is estimated. It computes  $\tau := (dx, dp)$  by solving solving the bordered linear system 
$$\begin{bmatrix} F_x & F_p \\ \frac{\theta}{length(x)}dx_0 & (1-\theta)dp_0 \end{bmatrix} \begin{bmatrix} dx \\ dp \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$
 where  $\tau_0 := (dx_0, dp_0)$  is the tangent at the previous continuation step.

The predictor is set by the option `PALC(tangent = Bordered())` in `continuation` with `Bordered`. The linear solver for the linear problem in  $(dx, dp)$  is set by the option `bls` in `PALC`: it is one of `Bordered linear solvers (BLS)`.

## 3. Polynomial predictor

The polynomial predictor is based on a fit (least square regression) of an  $n$ th-order polynomial  $P$  on the last  $k$  solution vectors, where  $n < k$ . The arclength  $s$  is used for the polynomial which then fits the solution  $(x_i, p_i, s_i)$  as  $P(s_i) \approx (x_i, p_i)$ . To keep  $s$  in suitable range (see [\[Waugh\]](#)), we rescale it as  $s \rightarrow \frac{s-\bar{s}}{\sigma}$  where  $\sigma$  is the standard deviation of the  $s_i$ .

This algorithm is parametrized by `alg = Polynomial(Fred, n, k, v0)` where `pred::AbstractTangentComputation` is the tangent predictor used only for the first  $k$  solutions before the polynomial predictor is operational and `v0` is an example of guess. More information is available in `Polynomial`.

## 4. Multiple predictor (aka pmcont in pde2path)

The predictor is designed [\[Uecker2014\]](#) to avoid spurious branch switching and pass singular points especially in PDE where branch point density can be quite high. It is based on the use of many predictors with increasing "jumps"  $(x_i, p_i) = (x_0, p_0) + i \cdot ds \cdot \tau$ ,  $i \leq nb$  and use a corrector (PALC Newton) with the following twist. The criterion is that in each Newton step, the residual has to decrease by a factor  $0 < \alpha < 1$ :

$$\|F(u_n, p_n)\| \leq \alpha \|F(u_{n-1}, p_{n-1})\|$$

otherwise the corrector fails. The solution that is returned is the one for the highest  $i$ . We refer to [\[Uecker2014\]](#) for an exposition of the step size adaption strategy.

This algorithm is parametrized by `alg = Multiple(pred, x0, α, nb)` where  $\tau$  is an initial tangent vector (used to set the types) and `pred::PALC` is a predictor. The default value is `pred = PALC()`. More information is available in `Multiple`.

## References

- Allgower1990

Allgower and Georg, Numerical Continuation Methods, 1990

- Uecker2014

1.Uecker, H. pde2path - A Matlab Package for Continuation and Bifurcation in 2D Elliptic Systems. NMTMA 7, 58–106 (2014).

- Waugh

Waugh, Illingworth, and Juniper, "Matrix-Free Continuation of Limit Cycles for Bifurcation Analysis of Large Thermoacoustic Systems."



# Pseudo arclength continuation

- [Pseudo arclength continuation](#)
  - [Predictor](#)
  - [Corrector](#)
  - [Linear Algebra](#)
    - [Norm](#)
    - [Dot product](#)
    - [Linear problem](#)
  - [Step size control](#)
  - [Flow chart](#)
  - [References](#)

This is one of the continuation methods implemented in the package. It is set by the option `PALC(tangent = Bordered())` or `PALC(tangent = Secant())` in [continuation](#). See also [PALC](#) for more information.

For solving

$$\mathbb{R}^n \ni F(x, p) = 0 \tag{E}$$

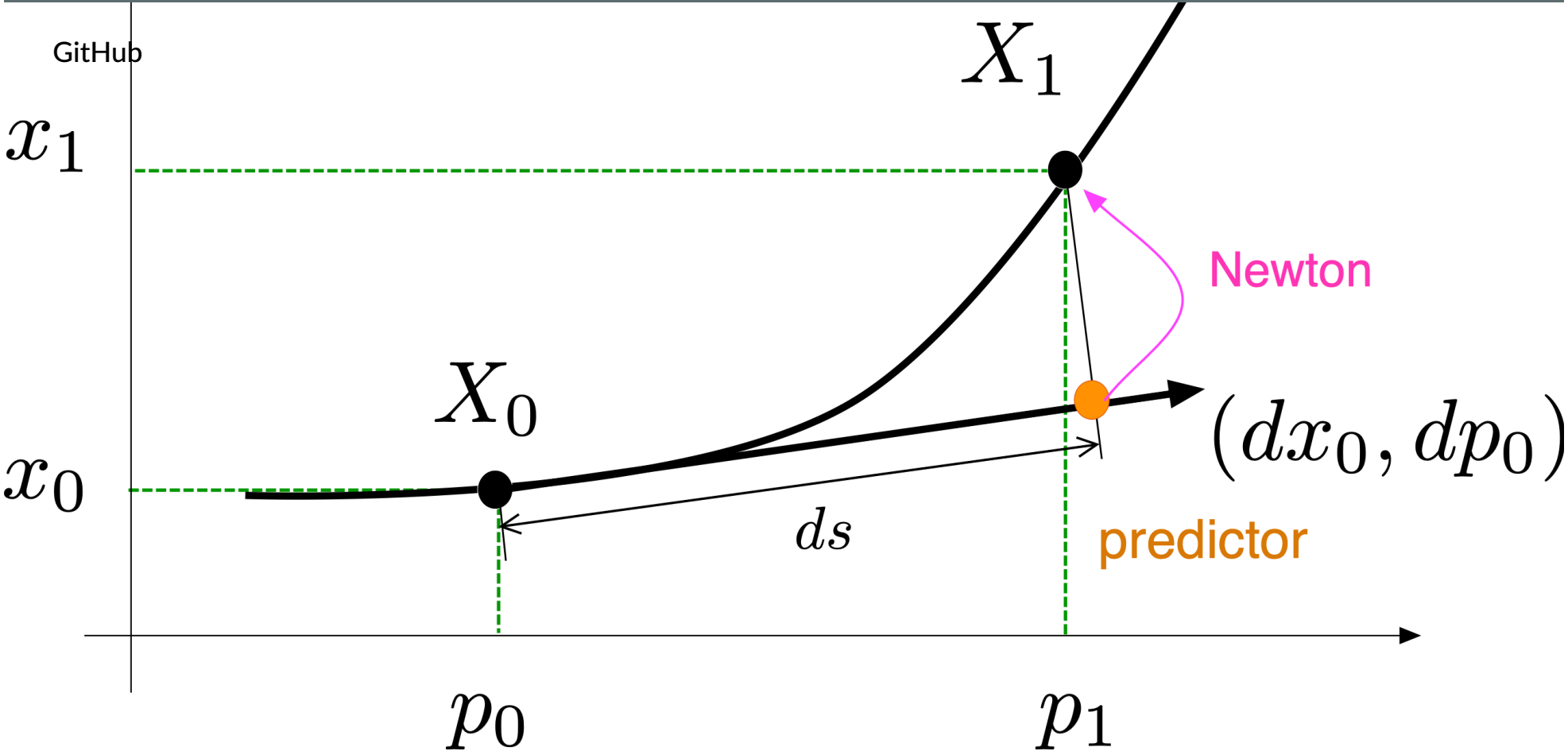
using a Newton algorithm, we miss an equation. The simplest way is to select an hyperplane in the space  $\mathbb{R}^n \times \mathbb{R}$  passing through  $(x_0, p_0)$ :

$$N(x, p) := \frac{\theta}{n} \langle x - x_0, dx_0 \rangle + (1 - \theta) \cdot (p - p_0) \cdot dp_0 - ds = 0 \tag{N}$$

with  $\theta \in [0, 1]$  and where  $ds$  is the pseudo arclength (see [\[Keller\]](#)).

 **Parameter `θ`**

The parameter  $\theta$  in `ContinuationPar` is very important. It should be tuned for continuation to work properly especially in the case of large problems where the  $\langle x - x_0, dx_0 \rangle$  component in the constraint might be favored too much. Also, large  $\theta$ s favour  $p$  as the corresponding term in the constraint  $N$  involves the term  $1 - \theta$ .



## Predictor

The possible predictors are listed in [Predictors - Correctors](#).

## Corrector

The corrector is the newton algorithm for finding the roots  $(x, p)$  of

$$\begin{bmatrix} F(x, p) \\ N(x, p) \end{bmatrix} = 0 \tag{PALC}$$

## Linear Algebra

### Norm

First, the option `normC` [continuation](#) specifies the norm used to evaluate the residual in the following way:

$$\max(\text{normC}(F(x, p)), |N(x, p)|) < \text{tol}.$$

It is thus used as a stopping criterion for the corrector. The dot product (resp. norm) used in  $N$  and in the (iterative) linear solvers is `LinearAlgebra.dot` (resp. `LinearAlgebra.norm`). It can be changed by importing these functions and redefining it. Note that by default, the  $\mathcal{L}^2$  norm is used.

These details are important because the constraint  $N$  incorporates the factor `length`. For some custom type implementing a Vector space, the dot product could already incorporates the `length` factor in which case you should either redefine the dot product or change  $\theta$ .

### Dot product

In the constraint  $N$  above, the scalar product is in fact saved in `BifurcationKit.jl` as `dotp(x,y) -> dot(x,y)/length(y)`. This is used in the bordered linear solvers associated to PALC. If you want to use your own dot product, you can pass

```
dotPALC = BK.DotTheta(mydot),
```

GitHub

Linear problem

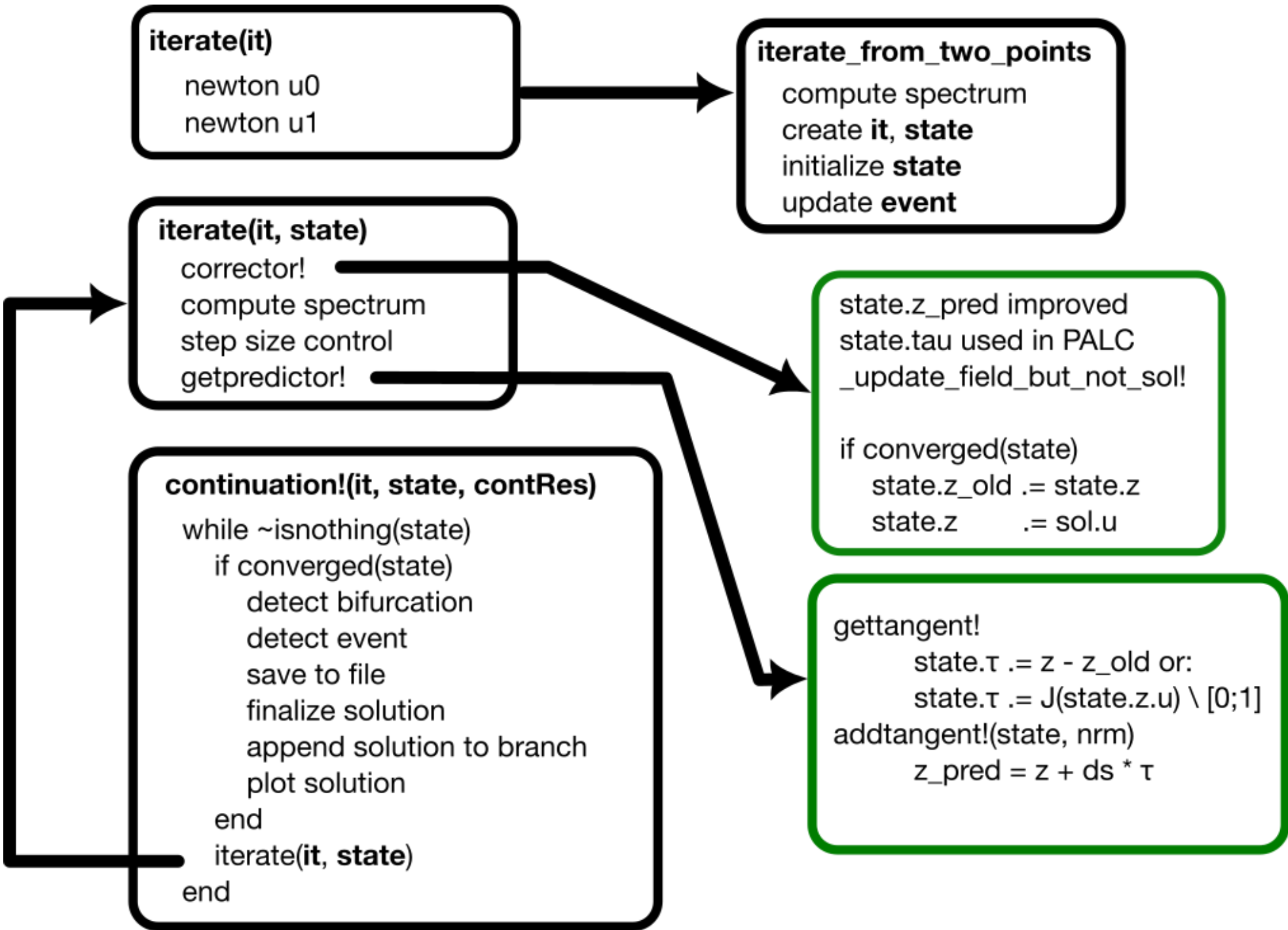
Pseudo arclength continuation is based on a newton solver applied to the enlarged problem (PALC). We thus need to solve a linear system of size  $n + 1$  whereas the user passed a linear solver (in `ContinuationPar().newton_options`) for a system of size  $n$ .

The linear solver for the linear problem associated to (PALC) is set by the option `linear_algo` in `continuation`: it is one of [Bordered linear solvers \(BLS\)](#).

Step size control

Each time the corrector fails, the step size  $ds$  is halved. This has the disadvantage of having lost Newton iterations (which costs time) and imposing small steps (which can be slow as well). To prevent this, the step size is controlled internally with the idea of having a constant number of Newton iterations per point. This is in part controlled by the aggressiveness factor `a` in `ContinuationPar`.

Flow chart



References

- Keller

Keller, Herbert B. Lectures on Numerical Methods in Bifurcation Problems. Springer, 1988

---

[GitHub](#)

# Moore-Penrose continuation

- Moore-Penrose continuation
  - Predictor
  - Corrector
  - Linear Algebra
    - Norm
    - Linear problem
  - Algorithm for solving (MS)
    - Direct case
    - Iterative case
  - Step size control
  - References

This is one of the various continuation methods implemented in `BifurcationKit.jl`. It is set by the option `alg = AutoSwitch()` in `continuation`. See also `AutoSwitch` for more information.

For solving

$$\mathbb{R}^n \ni F(x, p) = 0 \tag{E}$$

using a Newton algorithm, we miss an equation. Hence, we proceed as follows [\[Meijer\]](#). Starting from a predictor  $(x_1, p_1)$ , we look for the solution to (E) that is closest to  $(x_1, p_1)$ . Hence, we optimise

$$\min_{(x,p)} \{ \| (x, p) - (x_1, p_1) \| \text{ such that } F(x, p) = 0 \} \tag{MS}$$

It can be interpreted as a PALC in which the hyperplane is adapted at every step.

## Predictor

The possible predictors `tangent::AbstractTangentPredictor` are listed in [Predictors - Correctors](#). They can be used to create a Moore-Penrose algorithm like `MoorePenrose(tangent = PALC())`

## Corrector

The corrector is the Gauss Newton algorithm applied to (MS).

## Linear Algebra

### Norm

The option `normC` `continuation` specifies the norm used to evaluate the distance in (MS). The dot product (resp. norm) used in the (iterative) linear solvers is `LinearAlgebra.dot` (resp. `LinearAlgebra.norm`). It can be changed by importing these functions and redefining it. Note that by default, the  $L^2$  norm is used.

### Linear problem

The linear solver for the linear problem associated to (MS) is set by the option `linear_algo` in `continuation`: it is one of [Bordered linear solvers \(BLS\)](#).

Let us write  $y \equiv (x, p) \in \mathbb{R}^{n+1}$ . In order to solve for the argmin, we apply the newton algorithm with jacobian belonging to  $\mathbb{R}^{N \times N}$ . [GitHub](#).

$$y^{k+1} = y^k - d_y F(y^k)^+ F(y^k)$$

where the superscript  $^+$  indicates the Moore-Penrose pseudoinverse of rank  $N$ .

## Direct case

In this case, triggered by the option `MoorePenrose(method = BifurcationKit.direct)`, the pseudoinverse is computed with `\`.

For the option `MoorePenrose(method = BifurcationKit.pInv)`, the pseudoinverse is computed with `pinv`.

## Iterative case

In this case, triggered by the option `MoorePenrose(method = BifurcationKit.iterative)`, the pseudoinverse is computed with an iterative method described in [\[Meijer\]](#):

$$\begin{cases} y_1^{j+1} = y_1^j - \left( \begin{matrix} F_y(y_1^j) \\ (\phi_1^j)^\top \end{matrix} \right)^{-1} \left( \begin{matrix} F(y_1^j) \\ 0 \end{matrix} \right) \\ \phi_1^{j+1} = \left( \begin{matrix} F_y(y_1^{j+1}) \\ (\phi_1^j)^\top \end{matrix} \right)^{-1} \left( \begin{matrix} 0 \\ 1 \end{matrix} \right), \quad j = 0, 1, 2, \dots \end{cases}$$

We initialise  $\phi_1^0$  with the tangent.

## Step size control

Each time the corrector fails, the step size  $ds$  is halved. This has the disadvantage of having lost Newton iterations (which costs time) and imposing small steps (which can be slow as well). To prevent this, the step size is controlled internally with the idea of having a constant number of Newton iterations per point. This is in part controlled by the aggressiveness factor `a` in `ContinuationPar`.

## References

- Meijer

Meijer, Dercole, and Oldeman, “Numerical Bifurcation Analysis.”



# Automatic switching between Natural and PALC

This is one of the continuation methods implemented in the package. It is set by the option `AutoSwitch()` in `continuation`. See also `AutoSwitch` for more information.

This algorithm uses a `Natural` continuation when the branch is "horizontal" and switches to PALC (or other which cab be specified) otherwise.

## Example

We provide an example of use. We define a `BifurcationProblem` as usual and pass the continuation algorithm `AutoSwitch`.

```
using Plots
using BifurcationKit
const BK = BifurcationKit

function F(x, p)
    (;α) = p
    f = similar(x)

    f[1] = (-2x[1]+x[2]) + α * exp(x[1])
    f[2] = ( x[1]-2x[2]) + α * exp(x[2])

    return f
end

sol0 = zeros(2)
par = (α = 0.0, )
prob = ODEBifProblem(F, sol0, par, (@optic _ .α))
```

```
└─ Bifurcation problem with uType Vector{Float64}
└─ Inplace: false
└─ Dimension: 2
└─ Symmetric: false
└─ Parameter: α
```

```
opts = ContinuationPar(dsmax = 0.15, max_steps = 40, )

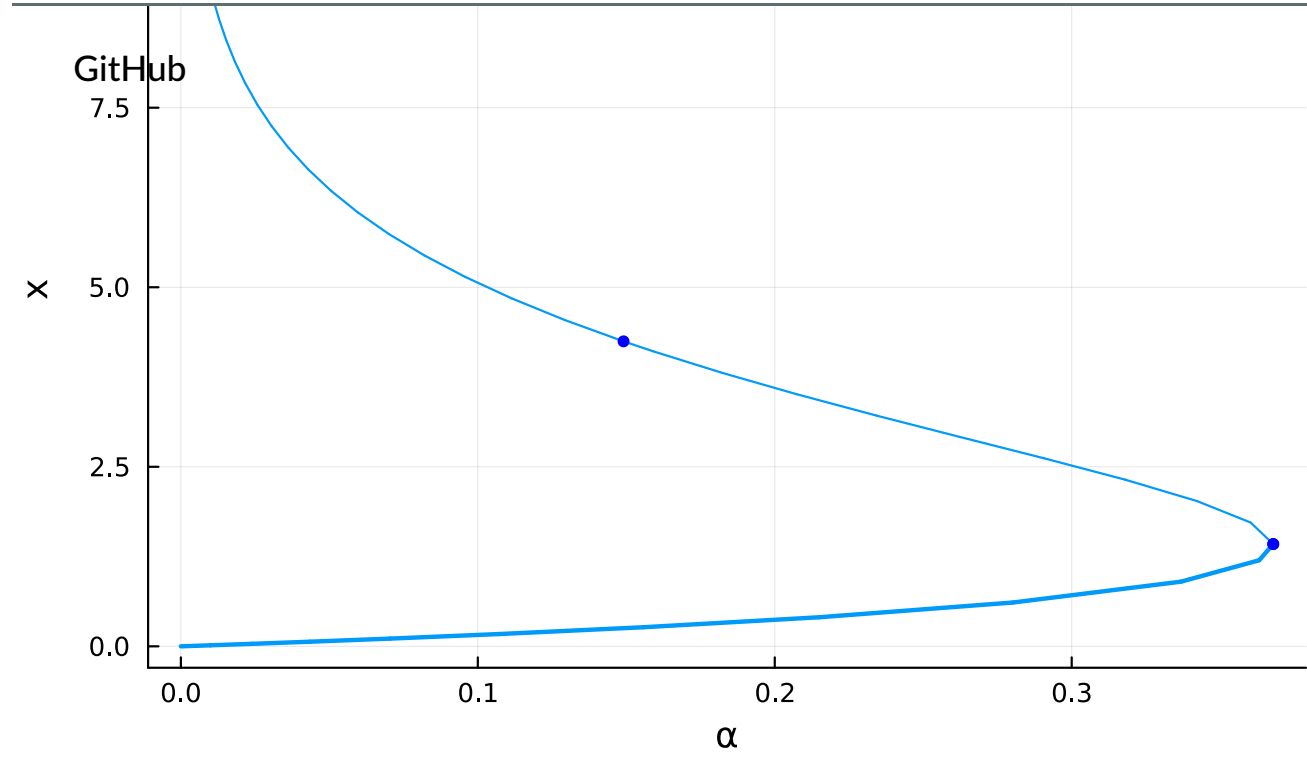
br = continuation(prob, AutoSwitch(), opts, normC = norminf)
```

```
└─ Curve type: EquilibriumCont
└─ Number of points: 41
└─ Type of vectors: Vector{Float64}
└─ Parameter α starts at 0.0, ends at 0.006224449009227574
└─ Algo: AutoSwitch
└─ Special points:

- # 1,      bp at α ≈ +0.36787092 ∈ (+0.36747428, +0.36787092), |δp|=4e-04, [converged], δ = ( 1,  0),
- # 2,      bp at α ≈ +0.14907908 ∈ (+0.14907908, +0.14973607), |δp|=7e-04, [converged], δ = ( 1,  0),
- # 3, endpoint at α ≈ +0.00518664,
```

You can plot the result as usual:

```
plot(br)
```





# Asymptotic numerical method (ANM)

! Dimensions

This is a method for small dimensions, less than several thousands.

To access this algorithm, you have to use the package [AsymptoticNumericalMethod.jl](#)

The method [\[Rubbert\]](#), [\[Charpentier\]](#) seeks a Taylor approximation of the solutions of

$$\mathbf{F}(X,p) = 0 \in \mathbb{R}^n$$

where  $X \in \mathbb{R}^n, p \in \mathbb{R}$ . The solution is found by solving

$$F(x(s),p(s)) = 0$$

$$\langle x(s) - x(0), \frac{\partial u}{\partial s}(0) \rangle + (p(s) - p(0)) \frac{\partial p}{\partial s}(0) = s$$

where

$$x(s) = \sum_{k=0}^K x_k s^k, \quad p(s) = \sum_{k=0}^K p_k s^k$$

for some user passed  $K > 0$ . It gives

$$F(x(s),p(s)) = \sum_{k=0}^K F_k s^k + h.o.t.$$

from which we deduce the equations  $F_k = 0$ . We then find:

$$F_{1,x_1=Id,p_1=1}(x_k,p_k) = -F_{k,x_k=0,p_k=0}.$$

The validity range of the solution can be estimated by

$$r_K = \left( \frac{\epsilon}{||F_K||} \right)^{1/K}$$

This allows to iterate and find a sequence of series which spans the parameter range.

## Implementation

The method is based on the package [TaylorSeries.jl](#) which makes it easy to manipulate Taylor series based on Automatic Differentiation.

## Method

See [AsymptoticNumericalMethod.ANM](#) for more information.

We provide an example of use. We define a `BifurcationProblem` as usual and pass the continuation algorithm `ANM`.

# GitHub

```
using AsymptoticNumericalMethod, Plots
using BifurcationKit
const BK = BifurcationKit

function F(x, p)
    (;α) = p
    f = similar(x)

    f[1] = (-2x[1]+x[2]) + α * exp(x[1])
    f[2] = ( x[1]-2x[2]) + α * exp(x[2])

    return f
end

sol0 = zeros(2)
par = (α = 0.0, )
prob = ODEBifProblem(F, sol0, par, (@optic _ .α); record_from_solution = (x,p;k...) -> norminf(x))
```

- Bifurcation problem with uType `Vector{Float64}`
- Inplace: `false`
- Dimension: `2`
- Symmetric: `false`
- Parameter:  `$\alpha$`

```
optanm = ContinuationPar(dsmin = 0.01, dsmax = 0.15, detect_bifurcation = 3, ds= 0.01, newton_options = M  
branm = continuation(prob, ANM(20, 1e-8), optanm, normC = norminf, verbosity = 2)
```

```

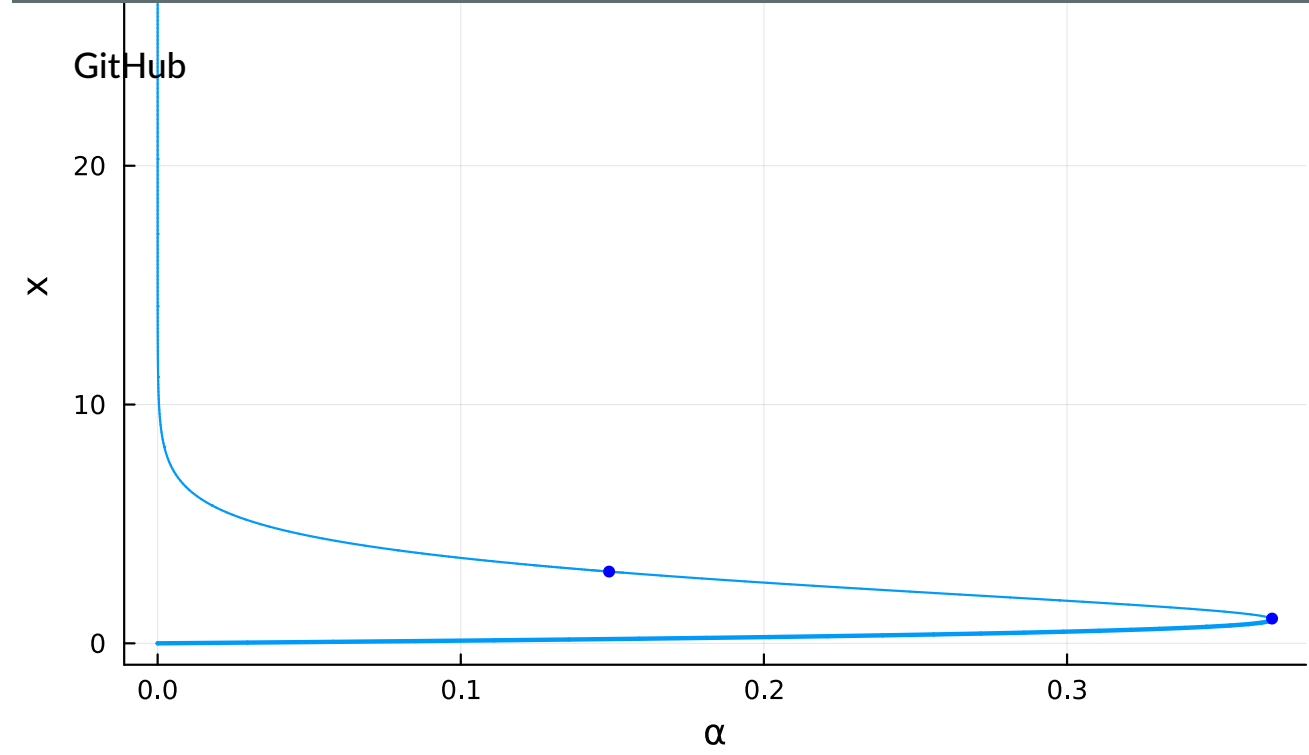
- Curve type: EquilibriumCont
- 15 series of degree 20, tol = 1.0e-8
- Number of points: 301
- Type of vectors: Vector{Float64}
- Parameter  $\alpha$  starts at 6.666666666666667e-5, ends at 3.098597914187087e-12
- Algo: ANM
- Special points:

- # 1,      bp at  $\alpha \approx +0.36763519 \in (+0.36787944, +0.36787944)$ ,  $|\delta p|=2e-11$ , [converged],  $\delta = (1, 0)$ ,
- # 2,      bp at  $\alpha \approx +0.14892964 \in (+0.14936111, +0.14936164)$ ,  $|\delta p|=5e-07$ , [converged],  $\delta = (1, 0)$ ,

```

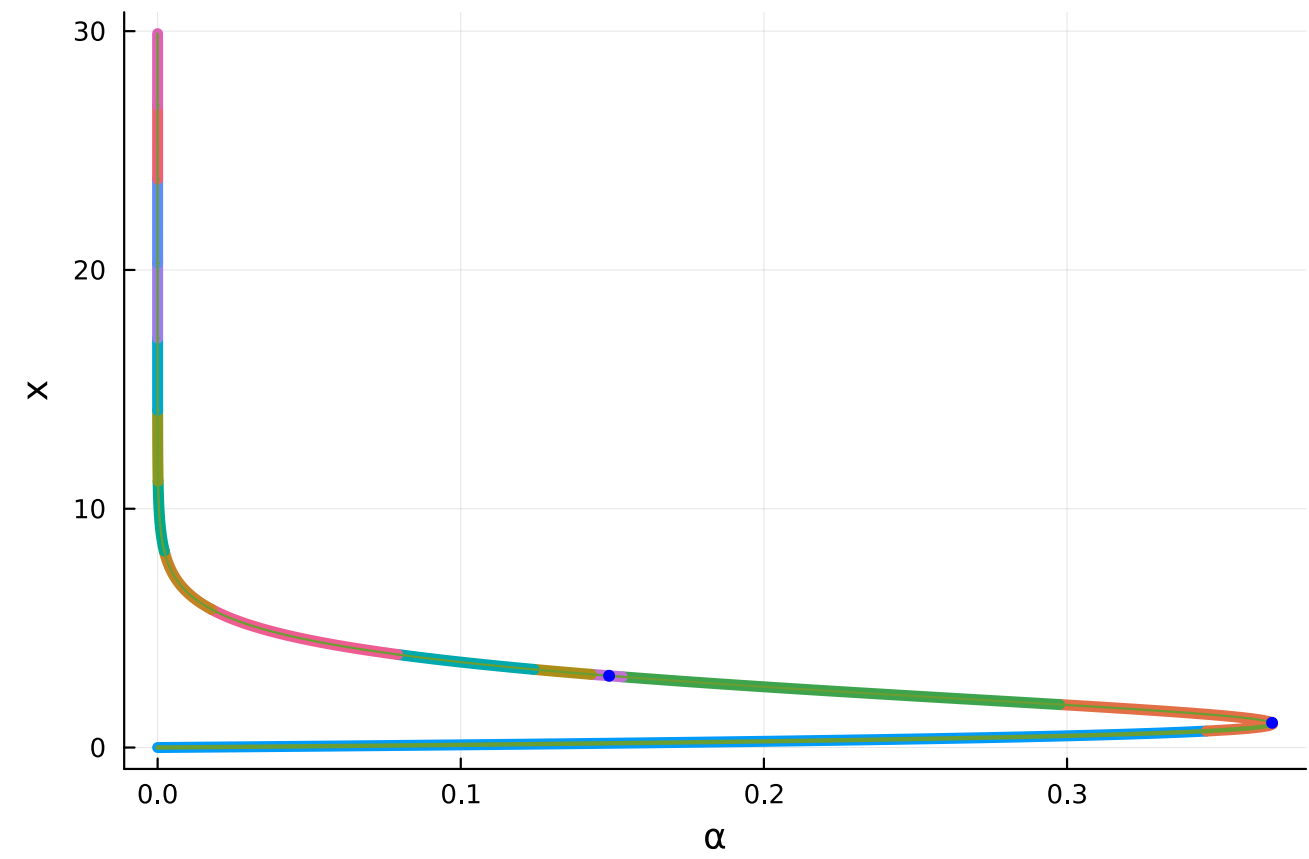
You can plot the result as usual:

```
plot(branm)
```



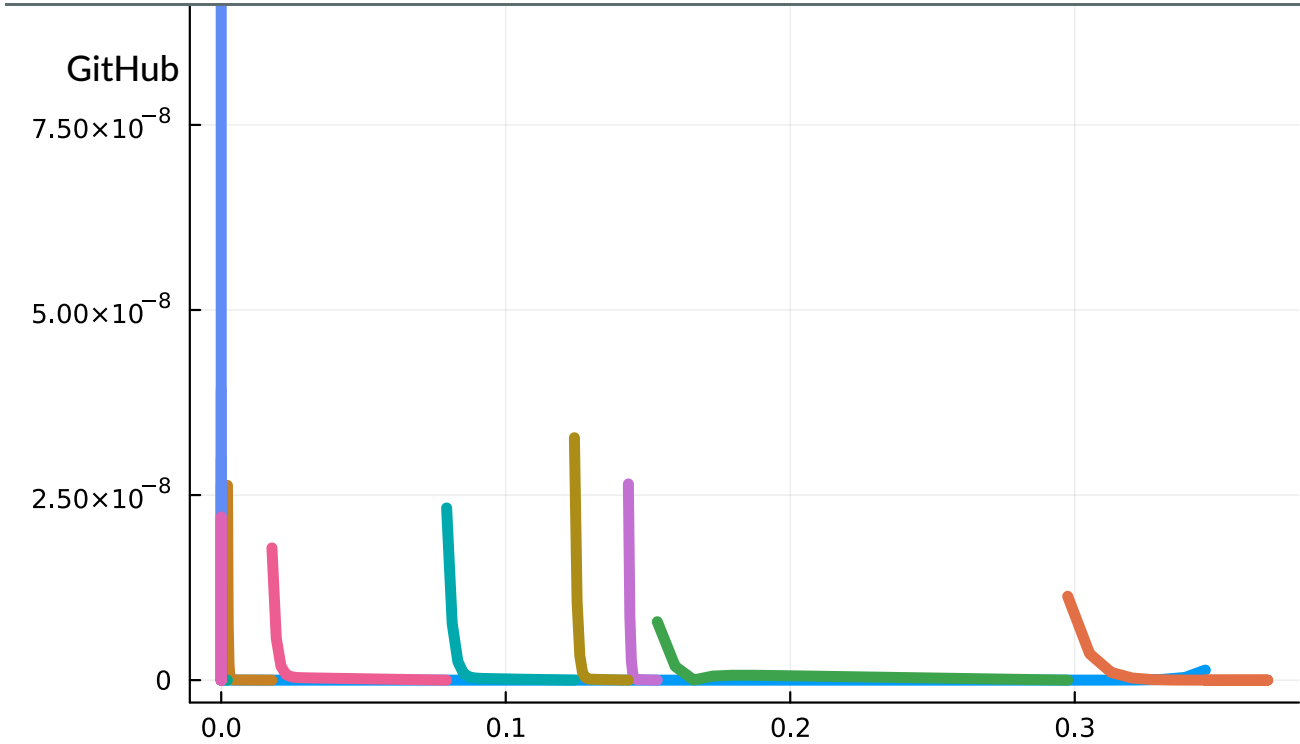
You can also show the radius of convergence of each series:

```
plot(branm, plotseries = true)
```



Finally, for each series, we ca evaluate the residual norm:

```
plot()
for ii in eachindex(branm.polU)
    s = LinRange(-0*branm.radius[ii], branm.radius[ii], 20)
    plot!([branm.polp[ii].(s)], [norminf(F(branm.polU[ii])(_s), BK.setparam(prob,branm.polp[ii])(_s)))])
end
title!("")
```



## References

- Charpentier

Charpentier, Isabelle, Bruno Cochelin, and Komlanvi Lampoh. “Diamanlab - An Interactive Taylor-Based Continuation Tool in MATLAB,” n.d., 12.

- Rubbert

Rubbert, Lennart, Isabelle Charpentier, Simon Henein, and Pierre Renaud. “Higher-Order Continuation Method for the Rigid-Body Kinematic Design of Compliant Mechanisms”, n.d., 18.

# Deflated Continuation

- Deflated Continuation
  - Algorithm
  - Tips
  - Basic example

References

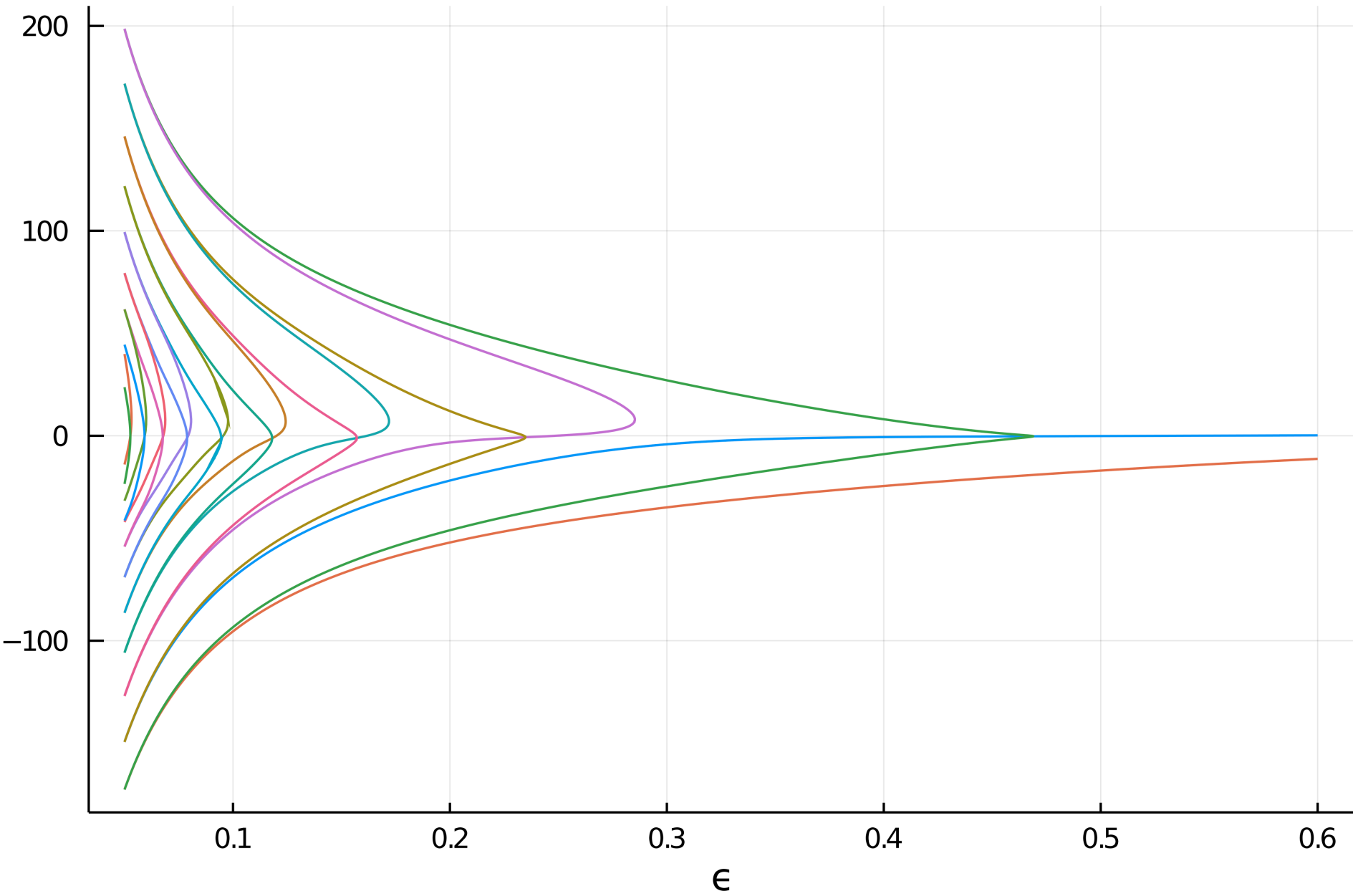
Farrell, Patrick E., Casper H. L. Beentjes, and Ásgeir Birkisson. **The Computation of Disconnected Bifurcation Diagrams.** ArXiv:1603.00809 [Math], March 2, 2016. <http://arxiv.org/abs/1603.00809>.

Deflated continuation allows to compute branches of solutions to the equation  $F(x, p) = 0$ . It is based on the Deflated Newton (see [Deflated problems](#)) algorithm.

See [DefCont](#) for more information.

However, unlike the regular continuation method, deflated continuation allows to compute **disconnected** bifurcation diagrams, something that is impossible for our [Automatic Bifurcation diagram computation](#) which is limited to the connected component of the initial point.

You can find an example of use of the method in [Carrier Problem](#). We reproduce below the result of the computation which shows various disconnected components arising form Fold bifurcations that are found seemingly by the method.



## Algorithm

Input: Nonlinear residual  $f(u,\lambda)$ .  
Input: Deflation operator  $M(u; u^*)$ .  
Input: Initial solutions  $S(\lambda_{\min})$  to  $f(\cdot, \lambda_{\min})$ .  
 $\lambda \leftarrow \lambda_{\min}$   
**while**  $\lambda < \lambda_{\max}$  **do**  
     $F(\cdot) \leftarrow f(\cdot, \lambda + \Delta\lambda)$  # Fix the value of  $\lambda$  to solve for  
     $S(\lambda + \Delta\lambda) \leftarrow \emptyset$   
    **for**  $u_0 \in S(\lambda)$  **do** # Continue known branches.  
        apply Newton's method to  $F$  from initial guess  $u_0$ .  
        **if** solution  $u^*$  found **then**  
             $S(\lambda + \Delta\lambda) \leftarrow S(\lambda + \Delta\lambda) \cup \{u^*\}$  # Record success  
             $F(\cdot) \leftarrow M(\cdot; u^*)F(\cdot)$  # Deflate solution  
  
    **for**  $u_0 \in S(\lambda)$  **do** # Seek new branches.  
        success  $\leftarrow$  true  
        **while** success **do**  
            apply Newton's method to  $F$  from initial guess  $u_0$ .  
            **if** solution  $u^*$  found **then** # New branch found  
                 $S(\lambda + \Delta\lambda) \leftarrow S(\lambda + \Delta\lambda) \cup \{u^*\}$  # Record success  
                 $F(\cdot) \leftarrow M(\cdot; u^*)F(\cdot)$  # Deflate solution  
            **else**  
                success  $\leftarrow$  false  
     $\lambda \leftarrow \lambda + \Delta\lambda$   
**return**  $S$

## Tips

The following piece of information is valuable in order to get the algorithm working in various conditions (see also [here](#)) especially for small systems (e.g.  $\dim < 20$ ):

- newton is quite good and it is convenient to limit it otherwise it will be able to bypass the deflation. For example, you can use `max_iterations = 10` in `NewtonPar`
- try to limit the newton residual by using the argument `callback_newton = BifurcationKit.cbMaxNorm(1e7)`. This will likely remove the occurrence of `␣ Error: Same solution found for identical parameter value!!`
- finally, you can try some aggressive shift (here `0.01` in the deflation operator, like `DeflationOperator(2, dot, 0.01, [sol])`) but use it wisely.

## Basic example

We show a quick and simple example of use. Note in particular that the algorithm is able to find the disconnected branch. The starting points are marked with crosses

```
using BifurcationKit, LinearAlgebra, Plots
const BK = BifurcationKit

k = 2
N = 1
F(x, p) = @. p * x + x^(k+1)/(k+1) + 0.01
Jac_m(x, p) = diagm(0 => p .+ x.^k)

# bifurcation problem
prob = ODEBifProblem(F, [0.], 0.5, (@optic _), J = Jac_m)

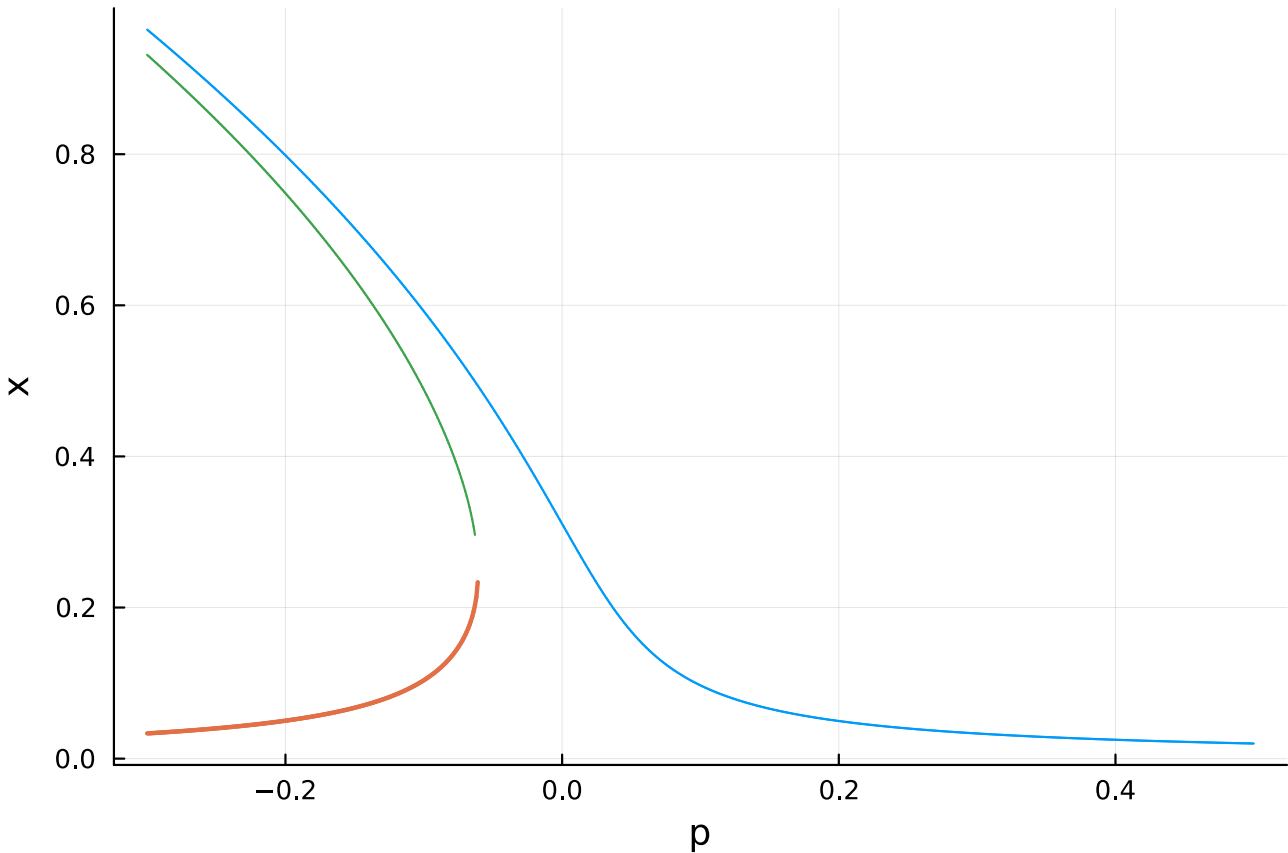
# continuation options
opts = BK.ContinuationPar(ds=0.001, max_steps = 140, newton_options = NewtonPar(tol = 1e-8))

# algorithm
alg = DefCont(deflation_operator = DeflationOperator(2, .001, [[0.]]), perturb_solution = (x,p,id) -> (x
```



GitHub

```
plot=false,  
callback_newton = BK.cbMaxNorm(1e3) # reject newton step if residual too large  
)  
plot(brdc)
```



# Detection of bifurcation points of periodic orbits

- Detection of bifurcation points of periodic orbits
  - Precise detection of bifurcation points using bisection
  - Large scale computations
  - List of detected bifurcation points
  - Eigensolver
  - Generic bifurcation
  - Fold bifurcation
  - Neimark-Sacker bifurcation
  - Period-doubling bifurcation

The bifurcations are detected during a call to `br = continuation(prob, alg, contParams::ContinuationPar;kwargs...)` by turning on the following flags:

- `contParams.detect_bifurcation = 2`

The bifurcation points are located by looking at the spectrum **e.g.** by monitoring the unstable eigenvalues. The Floquet exponent  $\lambda$  is declared unstable if `real( $\lambda$ ) > contParams.tol_stability`. The located bifurcation points are then returned in `br.specialpoint`.

## Precise detection of bifurcation points using bisection

Note that the bifurcation points detected when `detect_bifurcation = 2` can be rather *crude* localization of the true bifurcation points. Indeed, we only signal that, in between two continuation steps *which can be large*, a (several) bifurcation has been detected. Hence, we only have a rough idea of where the bifurcation is located, unless your `dsmax` is very small... This can be improved as follows.

If you choose `detect_bifurcation = 3`, a **bisection algorithm** is used to locate the bifurcation points more precisely. It means that we recursively track down the change in stability. Some options in `ContinuationPar` control this behavior:

- `n_inversion`: number of sign inversions in the bisection algorithm
- `max_bisection_steps` maximum number of bisection steps
- `tol_bisection_eigenvalue` tolerance on real part of Floquet exponent to detect bifurcation points in the bisection steps

### ! Bisection mode

During the bisection, the eigensolvers are called like `eil(J, nev; bisection = true)` in order to be able to adapt the solver precision.

## Large scale computations

The user must specify the number of eigenvalues to be computed (like `nev = 10`) in the parameters `::ContinuationPar` passed to `continuation`. Note that `nev` is automatically incremented whenever a bifurcation point is detected <sup>[1]</sup>. Also, there is an option in `::ContinuationPar` to save (or not) the eigenvectors. This can be useful in memory limited environments (like on GPUs).

## List of detected bifurcation points

|  | Bifurcation | index used |
|--|-------------|------------|
|--|-------------|------------|



| BifurcationKit | MultiParamContinuation | DDEBifurcationKit | HclinicBifurcationKit | Tutorials | Citing |
|----------------|------------------------|-------------------|-----------------------|-----------|--------|
| GitHub         | Neimark-Sacker         |                   | ns                    |           |        |
|                | Period doubling        |                   | pd                    |           |        |
|                | Not documented         |                   | nd                    |           |        |

## Eigsolver

The user must provide an eigsolver by setting `newton_options.eigsolver` where `newton_options` is located in the parameter `::ContinuationPar` passed to `continuation`. See [NewtonPar](#) and [ContinuationPar](#) for more information on the composite type of the options passed to `newton` and `continuation`.

The eigsolver is highly problem dependent and this is why the user should implement / parametrize its own eigsolver through the abstract type `AbstractEigenSolver` or select one among [List of implemented eigen solvers](#).

 **Floquet multipliers computation**

The computation of Floquet multipliers is necessary for the detection of bifurcations of periodic orbits (which is done by analyzing the Floquet exponents obtained from the Floquet multipliers). Hence, the eigsolver needs to compute the eigenvalues with largest modulus (and not with largest real part which is their default behavior). This can be done by changing the option `which = :LM` of the eigsolver. Nevertheless, note that for most implemented eigsolvers in `BifurcationKit`, the proper option is automatically set.

## Generic bifurcation

By this we mean a change in the dimension of the Jacobian kernel. The detection of Branch point is done by analysis of the spectrum of the Jacobian.

The detection is triggered by setting `detect_bifurcation > 1` in the parameter `::ContinuationPar` passed to `continuation`.

## Fold bifurcation

The detection of **Fold** point is done by monitoring the monotonicity of the parameter.

The detection is triggered by setting `detect_fold = true` in the parameter `::ContinuationPar` passed to `continuation`. When a **Fold** is detected on a branch `br`, a point is added to `br.foldpoint` allowing for later refinement using the function `newton_fold`.

## Neimark-Sacker bifurcation

The detection of Neimark-Sacker point is done by analysis of the spectrum of the Jacobian.

The detection is triggered by setting `detect_bifurcation > 1` in the parameter `::ContinuationPar` passed to `continuation`. When a **Neimark-Sacker point** is detected, a point is added to `br.specialpoint`.

## Period-doubling bifurcation

The detection of Period-doubling point is done by analysis of the spectrum of the Jacobian.

The detection is triggered by setting `detect_bifurcation > 1` in the parameter `::ContinuationPar` passed to `continuation`. When a **Period-doubling point** is detected, a point is added to `br.specialpoint`.

- 1 In this case, the Krylov dimension is not increased because the eigsolver could be a direct solver. You might want to increase this dimension using the callbacks in [continuation](#).



# Continuation of Fold of periodic orbits

- Continuation of Fold of periodic orbits
  - Fold continuation
  - Setting the jacobian
  - Fold points continuation
  - References

In this page, we explain how to perform continuation of Fold points and detect the associated codim 2 bifurcations.

!

Large scale problems

The continuation of Fold bifurcations is not optimized for Shooting Matrix-Free problems (e.g. Monodromy) yet.

## List of detected codim 2 bifurcation points

|                                  | Bifurcation           | symbol used | Multipliers               |
|----------------------------------|-----------------------|-------------|---------------------------|
|                                  | Cusp                  | cuspid      | {1,1}                     |
| Strong resonance 1:1 bifurcation |                       | R1          | {1,1,1}                   |
|                                  | Fold / Flip           | foldPD      | {1,1,-1}                  |
|                                  | Fold / Neimark-Sacker | foldNS      | {1,1, $e^{\pm i\alpha}$ } |

In a nutshell, all you have to do (see below) is to call `continuation(br, ind_bif, lens2)` to continue the bifurcation point stored in `br.specialpoint[ind_bif]` and set proper options.

## Fold continuation

The continuation of Fold bifurcation points is based on a **Minimally Augmented**<sup>[Govaerts]</sup> formulation which is an efficient way to detect singularities (see [Fold / Hopf Continuation](#)). All the methods (see [Periodic orbits computation](#)) for computing periodic orbits are compatible with this algorithm. In particular, you can perform these computations in large dimensions.

## Detection of codim 2 bifurcation points

You can detect the following codim 2 bifurcation points by using the keyword argument `detect_codim2_bifurcation` in the method `continuation`

- the detection of Cusp (Cusp) is performed by the detection of Fold bifurcation points along the curve of Folds by monitoring the parameter component of the tangent.
- the detection of the R1 bifurcation point is performed using the test function  $\psi_{R1}(p) = 1 + \cos \omega$ .
- the detection the above bifurcation points is done by monitoring the number of eigenvalues  $\lambda$  such that  $\Re \lambda > \min_{\nu \in \Sigma(dF)} |\Re \nu|$  and  $\Im \lambda > \epsilon$  where  $\epsilon$  is the Newton tolerance.

## Setting the jacobian

In order to apply the newton algorithm to the Fold functional, one needs to invert the jacobian. This is not completely trivial as one must compute this jacobian and then invert it. You can select the following jacobians for your computations (see below):

(say < 100)

- [GitHub](#) `jacobian_ma = FiniteDifferences()`: same as `jacobian_ma = AutoDiff()` but the jacobian is computed using finite differences.
- `jacobian_ma = MinAug()`: a specific procedure for evaluating the jacobian and inverting it (without forming the jacobian!) is used. This is well suited for large dimensions and for matrix-free version.
- `jacobian_ma = MinAugMatrixBased()` the jacobian matrix is evaluated using analytical formula. This is faster than `AutoDiff()`.

## Fold points continuation

To compute the codim 2 curve of Fold points of periodic orbits, one can call `continuation` with the following options

### ▼ `BifurcationKit.continuation` — Function

```
continuation(  
    br::BifurcationKit.AbstractBranchResult,  
    ind_bif,  
    lens2::Union{typeof(identity), IndexLens, PropertyLens, ComposedFunction};  
    ...  
) -> Any  
continuation(  
    br::BifurcationKit.AbstractBranchResult,  
    ind_bif,  
    lens2::Union{typeof(identity), IndexLens, PropertyLens, ComposedFunction},  
    options_cont::ContinuationPar;  
    prob,  
    start_with_eigen,  
    detect_codim2_bifurcation,  
    update_minaug_every_step,  
    kwargs...  
) -> Any
```

Codimension 2 continuation of Fold / Hopf points. This function turns an initial guess for a Fold / Hopf point into a curve of Fold / Hopf points based on a Minimally Augmented formulation. The arguments are as follows

- `br` results returned after a call to `continuation`
- `ind_bif` bifurcation index in `br`
- `lens2` second parameter used for the continuation, the first one is the one used to compute `br`, e.g. `getlens(br)`
- `options_cont = br.contparams` arguments to be passed to the regular `continuation`

### Optional arguments:

- `linsolve_adjoint` solver for  $(J+i\omega)^* \cdot \text{sol} = \text{rhs}$  or  $J^t \cdot \text{sol} = \text{rhs}$
- `bdlinsolver` bordered linear solver for the constraint equation
- `bdlinsolver_adjoint` bordered linear solver for the constraint equation with top-left block  $(J-i\omega)^*$  or  $J^t$ . Required in the linear solver for the Minimally Augmented Fold/Hopf functional. This option can be used to pass a dedicated linear solver for example with specific preconditioner.
- `update_minaug_every_step` update vectors `a`, `b` in Minimally Formulation every `update_minaug_every_step` steps
- `detect_codim2_bifurcation`  $\in \{0,1,2\}$  whether to detect Bogdanov-Takens, Bautin and Cusp. If equals 1 non precise detection is used. If equals 2, a bisection method is used to locate the bifurcations. Default value = 2.
- `start_with_eigen = false` whether to start the Minimally Augmented problem with information from eigen elements. If `start_with_eigen = false`, then:
  - `a::Nothing` estimate of null vector of  $J$  (resp.  $J-i\omega$ ) for Fold (resp. Hopf). If nothing is passed, a random vector is used. In case you do not rely on `AbstractArray`, you should probably pass this.
  - `b::Nothing` estimate of null vector of  $J^t$  (resp.  $(J-i\omega)^*$ ) for Fold (resp. Hopf). If nothing is passed, a random vector is used. In case you do not rely on `AbstractArray`, you should probably pass this.

where the parameters are as above except that you have to pass the branch `br` from the result of a call to `continuation` with detection of bifurcations enabled and `index` is the index of Hopf point in `br` you want to refine.

! **ODE problems**

For ODE problems, it is more efficient to use the Matrix based Bordered Linear Solver passing the option `bdlsolver = MatrixBLS()`

! **start\_with\_eigen**

It is recommended that you use the option `start_with_eigen = true`

where `br` is a branch of periodic orbits and the options are as above except with have an additional parameter `axis lens2` which is used to locate the bifurcation points.

## References

- Govaerts

Govaerts, Willy J. F. Numerical Methods for Bifurcations of Dynamical Equilibria. Philadelphia, Pa: Society for Industrial and Applied Mathematics, 2000.

## Continuation of Period-doubling (PD) bifurcations of periodic orbits

- Continuation of Period-doubling (PD) bifurcations of periodic orbits
  - PD continuation
  - Detection of codim 2 bifurcation points
  - Setting the jacobian
  - PD points continuation
  - Algorithmic details
  - References

## ! Large scale problems

The continuation of PD bifurcations is not optimized for Shooting Matrix-Free problems (e.g. Monodromy) yet.

In this page, we explain how to perform continuation of PD points of periodic orbits and detect the following codim 2 bifurcations.

## List of detected codim 2 bifurcation points

|  | Bifurcation                      | symbol used | Multipliers               |
|--|----------------------------------|-------------|---------------------------|
|  | Strong resonance 1:2 bifurcation | R2          | $\{1, -1, -1\}$           |
|  | Fold / Flip                      | foldFlip    | $\{1, 1, -1\}$            |
|  | Period-Doubling / Neimark-Sacker | pdNS        | $\{-1, e^{\pm i\alpha}\}$ |
|  | Generalized Period-Doubling      | gpd         | $\{1, -1\}$               |

In a nutshell, all you have to do (see below) is to call `continuation(br, ind_bif, lens2)` to continue the bifurcation point stored in `br.specialpoint[ind_bif]` and set proper options.

## PD continuation

The continuation of PD bifurcation points is based on a **Minimally Augmented**<sup>[Govaerts]</sup> formulation which is an efficient way to detect singularities (see [Fold / Hopf Continuation](#)). All the methods (see [Periodic orbits computation](#)), except the Trapezoid one, for computing periodic orbits are compatible with this algorithm. In particular, you can perform these computations in large dimensions.

## Detection of codim 2 bifurcation points

You can detect the following codim 2 bifurcation points by using the option `detect_codim2_bifurcation` in the method `continuation`

- the detection of Generalized Period-Doubling bifurcation is done by computing the PD normal form
- the detection of the R2 bifurcation is performed using the test function  $\psi_{R2}(p) = \langle w(p), v(p) \rangle$
- the detection the other above bifurcation points is done by monitoring the number of eigenvalues  $\lambda$  such that  $\Re \lambda > \min_{\nu \in \Sigma(dF)} |\Re \nu|$  and  $\Im \lambda > \epsilon$  where  $\epsilon$  is the Newton tolerance.



In order to apply the newton algorithm to the PD functional, one needs to invert the jacobian. This is not completely trivial as one [has to](#) compute this jacobian and then invert it. You can select the following jacobians for your computations (see below):

- `jacobian_ma = AutoDiff()` [Default]: automatic differentiation is applied to the PD functional and the matrix is then inverted using the provided linear solver. In particular, the jacobian is formed. This is very well suited for small dimensions (say < 100)
- `jacobian_ma = FiniteDifferences()`: same as `jacobian_ma = AutoDiff()` but the jacobian is computed using finite differences.
- `jacobian_ma = MinAug()`: a specific procedure for evaluating the jacobian and inverting it (without forming the jacobian!) is used. This is well suited for large dimensions and for matrix-free version.
- `jacobian_ma = MinAugMatrixBased()` the jacobian matrix is evaluated using analytical formula. This is faster than `AutoDiff()`.

For the case `jacobian_ma = MinAug()`, when the shooting method is employed, the adjoint of the flow is required. This can usually be computed with `ReverseDiff.jl`.

## PD points continuation

To compute the codim 2 curve of PD points of periodic orbits, one can call `continuation` with the following options (here with collocation but shooting works too)

❗ **Missing docstring.**

Missing docstring for continuation(`br::BifurcationKit.AbstractResult{Tkind, Tprob}`, `ind_bif::Int64`, `lens2::BifurcationKit.AllOpticTypes`, `options_cont::ContinuationPar = br.contparams` ; `detect_codim2_bifurcation::Int = 0`, `update_minaug_every_step = 1`, `kwargs...`) where `{Tkind <: BifurcationKit.PeriodicOrbitCont, Tprob <: BifurcationKit.WrapPOColl}`. Check Documenter's build log for details.

where `br` is a branch of periodic orbits and the options are as above except with have an additional parameter axis `lens2` which is used to locate the bifurcation points.

## Algorithmic details

The continuation of PD points is based on the formulation

$$G(u, p, \omega) = (F_{po}(u, p), \sigma(u, p)) \in \mathbb{R}^{n+1} \quad (\mathcal{F}_{pd})$$

where  $F_{po}$  is the functional for locating periodic orbits and the test function  $\sigma$  is solution of

$$\begin{bmatrix} N(u, p) & w \\ v^\top & 0 \end{bmatrix} \begin{bmatrix} r \\ \sigma(u, p) \end{bmatrix} = \begin{bmatrix} 0_n \\ 1 \end{bmatrix}.$$

The jacobian of the PD functional to use for the Newton algorithm

$$\begin{bmatrix} \partial_u F_{po} & \partial_p F_{po} \\ \partial_u \sigma & \partial_p \sigma \end{bmatrix}.$$

## Shooting

In the case of Multiple Standard Shooting, the matrix  $N$  is based on the monodromy  $M(x_i, T_i)$

GitHub

$$N := \begin{pmatrix} \vdots & & \ddots & \ddots & \vdots \\ 0 & \dots & \dots & \ddots & -I \\ I & \dots & \dots & 0 & M_m \end{pmatrix}.$$

## Orthogonal collocation

In the case of orthogonal collocation, the matrix  $N$  is the jacobian of the periodic orbit functional stripped of the phase condition ( $m = 2$ ):

$$\begin{pmatrix} H_{0,0}^0 & H_{0,1}^0 & H_{1,0}^0 & & & & & \\ H_{0,0}^1 & H_{0,1}^1 & H_{1,0}^1 & & & & & \\ & & H_{1,0}^0 & H_{1,1}^0 & H_{2,0}^0 & & & \\ & & H_{1,0}^1 & H_{1,1}^1 & H_{2,0}^1 & & & \\ & & & & H_{2,0}^0 & H_{2,1}^0 & H_{3,0}^0 & \\ & & & & H_{2,0}^1 & H_{2,1}^1 & H_{3,0}^1 & \\ & & & & & & & I \\ I & & & & & & & \end{pmatrix}$$

## References

- Govaerts

Govaerts, Willy J. F. Numerical Methods for Bifurcations of Dynamical Equilibria. Philadelphia, Pa: Society for Industrial and Applied Mathematics, 2000.





- the detection of R4 is performed using the test function  $\psi_{R4}(p) = \cos \omega$ .  
GitHub
- the detection the other above bifurcation points is done by monitoring the number of eigenvalues  $\lambda$  such that  $\Re \lambda > \min_{\nu \in \Sigma(dF)} |\Re \nu|$  and  $\Im \lambda > \epsilon$  where  $\epsilon$  is the Newton tolerance.

## Setting the jacobian

In order to apply the newton algorithm to the PD functional, one needs to invert the jacobian. This is not completely trivial as one must compute this jacobian and then invert it. You can select the following jacobians for your computations (see below):

- `jacobian_ma = AutoDiff()` [Default]: automatic differentiation is applied to the NS functional and the matrix is then inverted using the provided linear solver. In particular, the jacobian is formed. This is very well suited for small dimensions (say < 100)
- `jacobian_ma = FiniteDifferences()`: same as `jacobian_ma = AutoDiff()` but the jacobian is computed using finite differences.
- `jacobian_ma = MinAug()`: a specific procedure for evaluating the jacobian and inverting it (without forming the jacobian!) is used. This is well suited for large dimensions and for matrix-free version.
- `jacobian_ma = MinAugMatrixBased()` the jacobian matrix is evaluated using analytical formula. This is faster than `AutoDiff()`.

## NS points continuation

To compute the codim 2 curve of Fold points of periodic orbits, one can call `continuation` with the following options

! Missing docstring.

Missing docstring for continuation(`br::BifurcationKit.AbstractBranchResult`, `ind_bif::Int64`, `lens2::BifurcationKit.AllOpticTypes`, `options_cont::ContinuationPar` = `br.contparams` ; `kwargs...`). Check Documenter's build log for details.

where `br` is a branch of periodic orbits and the options are as above except with have an additional parameter axis `lens2` which is used to locate the bifurcation points.

## Algorithmic details

The continuation of NS points is based on the formulation

$$G(u, p, \omega) = (F_{po}(u, p, \omega), \Re \sigma(u, p, \omega), \Im \sigma(u, p, \omega)) \in \mathbb{R}^{n+2} \quad (\mathcal{F}_{ns})$$

where  $F_{po}$  is the functional for locating periodic orbits and the test function  $\sigma$  is solution of

$$\begin{bmatrix} N(u, p, \omega) & w \\ v^\top & 0 \end{bmatrix} \begin{bmatrix} r \\ \sigma(u, \omega, p) \end{bmatrix} = \begin{bmatrix} 0_n \\ 1 \end{bmatrix}.$$

The jacobian of the NS functional to use for the Newton algorithm

$$\begin{bmatrix} \partial_u F_{po} & \partial_p F_{po} & 0 \\ \partial_u \sigma_r & \partial_p \sigma_r & \partial_\omega \sigma_r \\ \partial_u \sigma_i & \partial_p \sigma_i & \partial_\omega \sigma_i \end{bmatrix}.$$

## Shooting

In the case of Multiple Standard Shooting, the matrix  $N$  is based on the monodromy  $M(x_i, T_i)$

GitHub

$$N := \begin{pmatrix} \vdots & & \ddots & \ddots & \vdots \\ 0 & \dots & \dots & \ddots & -I \\ e^{i\omega} I & \dots & \dots & 0 & M_m \end{pmatrix}.$$

## Orthogonal collocation

In the case of orthogonal collocation, the matrix  $N$  is the jacobian of the periodic orbit functional stripped of the phase condition ( $m = 2$ ):

$$\begin{pmatrix} H_{0,0}^0 & H_{0,1}^0 & H_{1,0}^0 & & & & & \\ H_{0,0}^1 & H_{0,1}^1 & H_{1,0}^1 & & & & & \\ & & H_{1,0}^0 & H_{1,1}^0 & H_{2,0}^0 & & & \\ & & H_{1,0}^1 & H_{1,1}^1 & H_{2,0}^1 & & & \\ & & & & H_{2,0}^0 & H_{2,1}^0 & H_{3,0}^0 & \\ & & & & H_{2,0}^1 & H_{2,1}^1 & H_{3,0}^1 & \\ I & & & & & & & e^{i\omega} I \end{pmatrix}$$

## References

- Govaerts

Govaerts, Willy J. F. Numerical Methods for Bifurcations of Dynamical Equilibria. Philadelphia, Pa: Society for Industrial and Applied Mathematics, 2000.

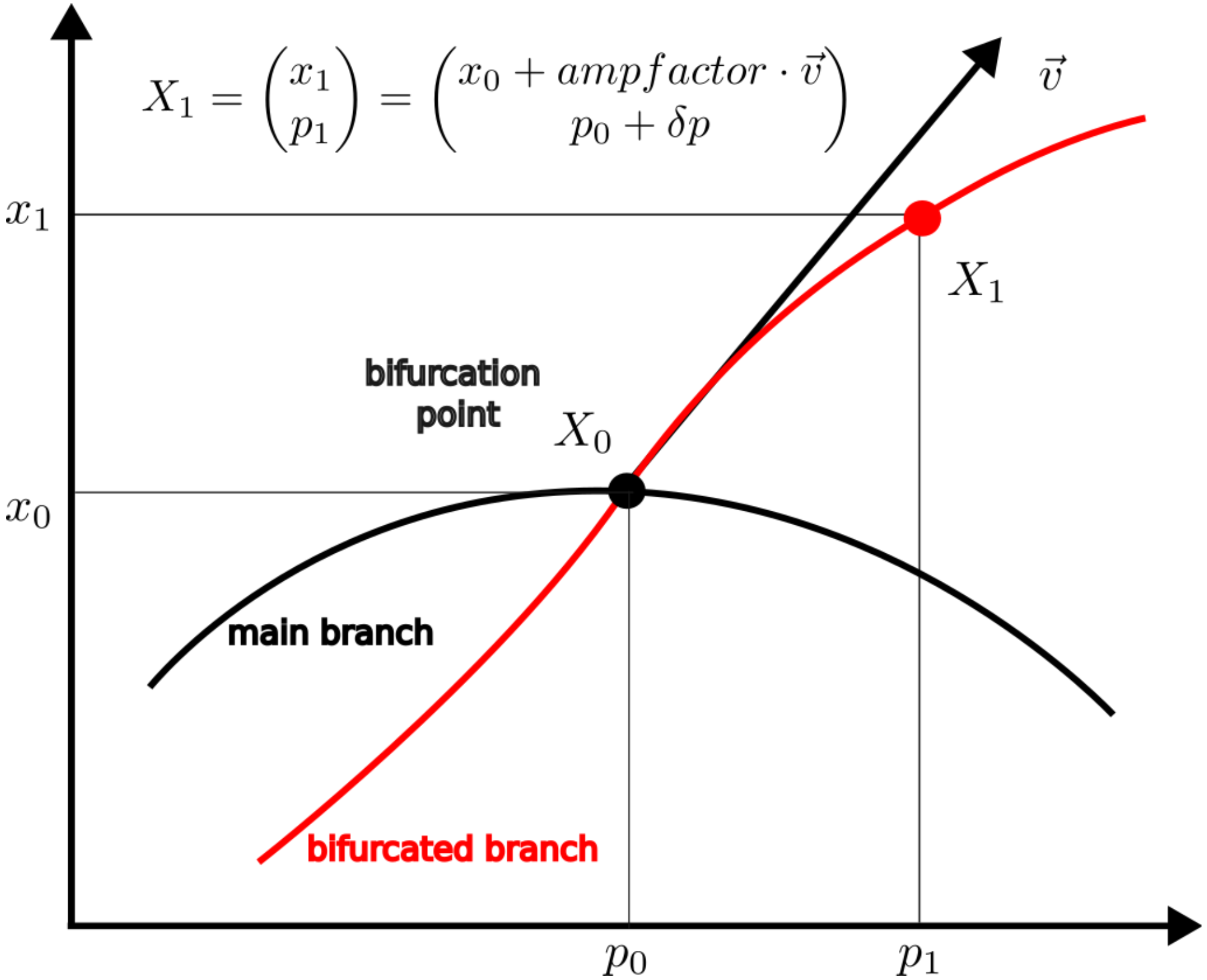
# Branch switching

- [Branch switching](#)
  - [\(Automatic\) branch switching of simple bifurcation points](#)
  - [Branch switching of non-simple bifurcation points](#)
  - [Graph of bifurcations](#)
  - [Summary of branching procedures](#)

The precise definition of the methods is given in [Branch switching \(branch point\)](#) and [Branch switching \(Hopf point\)](#).

## (Automatic) branch switching of simple bifurcation points

Unlike other continuation softwares (Auto07p, MatCont, pde2path), BifurcationKit has methods for automatic branch switching where the user is only asked to provide the parameter distance from a point on the bifurcated branch to the bifurcation point. Let us explain this in more detail using the following figure.



Knowing a bifurcation point  $X_0 = (x_0, p_0)$  on a branch `br`, we want to find the bifurcated (red) curve. To this, we need an initial guess  $X_1$  on the bifurcated curve. Usually, the tangent  $\vec{v}$  can be found which leaves two parameters  $\delta p$ , `ampfactor` to be determined. Note that both parameters needs to be small. For example, the default values are  $\delta p = ds$  (`ds` is in `ContinuationPar`) and `ampfactor` = 0.1.

doubling with Trapezoid method. Thus, the general method is like

GitHub

```
continuation(br, ind_bif, options_cont;
    δp = 0.1, ampfactor = 0.1,
    kwargs...)
```

- In case of **automatic branch switching**, the user is only asked to provide  $\delta p$  while `ampfactor` is internally estimated using the normal form of the bifurcation point. When the user does not pass  $\delta p$ , the value of `ds` in `ContinuationPar` is used. Thus, the general method is like

```
continuation(br, ind_bif, options_cont; kwargs...)
```

When the bifurcation diagram is very stiff, **automatic branch switching** may fail (for example `BifurcationKit` could return `ampfactor=1e6` or `ampfactor = 1e-10`). In this case, one can override **automatic branch switching** and specify  $\delta p$ , `ampfactor` directly as for **manual branch switching** using

```
continuation(br, ind_bif, options_cont;
    δp = 0.1, ampfactor = 0.1,
    use_normal_form = false,
    kwargs...)
```

⚠ Manual branch switching

Manual branch switching is automatically used by `BifurcationKit` when the normal form of the bifurcation point is not implemented (like PD normal form for Trapezoid method). You can force `BifurcationKit` to use it using the `use_normal_form = false` key word.

## Branch switching of non-simple bifurcation points

We refer to [Branch switching](#) for more details.

## Graph of bifurcations

We provide a graph of bifurcations of equilibria and periodic orbits that can be detected in `BifurcationKit`. An arrow from say `Equilibrium` to `Hopf` means that Hopf bifurcations can be detected while continuing equilibria. Each object of codim 0 (resp. 1) can be continued in one (resp. 2) parameters.

## Summary of branching procedures

We collect in the following table the list of automatic branch switching (aBS) functions. Their detailed explanation follows in this page.

|  | function                                                                   | ind-th bif. point       | Type T                       | description                                                                              |
|--|----------------------------------------------------------------------------|-------------------------|------------------------------|------------------------------------------------------------------------------------------|
|  | <code>continuation(br::ContResult{T}, ind::Int; kw...)</code>              | <code>:bp, :nd</code>   | <code>EquilibriumCont</code> | aBS from equilibria to equilibria                                                        |
|  | <code>continuation(br::ContResult{T}, ind::Int, lens2::Lens; kw...)</code> | <code>:bp, :hopf</code> | <code>EquilibriumCont</code> | Fold/Hopf continuation w.r.t. parameters <code>getlens(br)</code> and <code>lens2</code> |

| BifurcationKit                                                                                        | MultiParamContinuation | DDEBifurcationKit | HclinicBifurcationKit | Tutorials                                                                                           | Citing                                       |
|-------------------------------------------------------------------------------------------------------|------------------------|-------------------|-----------------------|-----------------------------------------------------------------------------------------------------|----------------------------------------------|
| GitHub                                                                                                |                        |                   |                       |                                                                                                     | Hopf/Fold w.r.t. parameters of codim 2<br>br |
| continuation(br::ContResult{T}, ind_hopf::Int, ::ContinuationPar, prob::AbstractPeriodicOrbitProblem) |                        | :hopf             | EquilibriumCont       | Branch switching from Hopf point to periodic orbits                                                 |                                              |
| continuation(br::ContResult{T}, ind::Int, kw...)                                                      |                        | :bp,:pd           | PeriodicOrbitCont     | Branch switching from Branch / Period-doubling point of periodic orbits to curve of periodic orbits |                                              |
| continuation(br::ContResult{T}, ind::Int, kw...)                                                      |                        | :gh,:zh,:hh       | TwoParamCont          | Branch switching from Bautin / Zero-Hopf/ Hopf-Hopf point to curve of Fold/NS of periodic orbits    |                                              |

« [Neimark-Sacker](#)

[From equilibria to equilibria](#) »

Powered by [Documenter.jl](#) and the [Julia Programming Language](#).



# From Hopf / PD / Branch point to periodic orbits

- From Hopf / PD / Branch point to periodic orbits
  - From Hopf point to periodic orbits
    - Algorithm
    - Example
  - From Period-doubling point to curve of periodic orbits
    - Case of Shooting and Collocation
    - Case of Trapezoid method
    - Algorithm
  - From Branch point to curve of periodic orbits

## From Hopf point to periodic orbits

In order to compute the bifurcated branch of periodic solutions at a Hopf bifurcation point, you need to choose a method to compute periodic orbits among:

- Periodic orbits based on Trapezoidal rule
- Periodic orbits based on orthogonal collocation
- Periodic orbits based on the shooting method

Once you have decided which method to use, you can call the following:

```
continuation(br::ContResult, ind_HOPF::Int, _contParams::ContinuationPar,
    prob::AbstractPeriodicOrbitProblem ;
    δp = nothing, ampfactor = 1, kwargs...)
```

We refer to `continuation` for more information about the arguments. Here, we just say a few words about how we can specify `prob::AbstractPeriodicOrbitProblem`.

- For [Periodic orbits based on Trapezoidal rule](#), you can pass `Trapeze(M = 51)` where `M` is the number of times slices in the periodic orbit.
- For [Periodic orbits based on orthogonal collocation](#), you can pass `Collocation(M, m)` where `M` is the number of times slices in the periodic orbit and `m` is the degree of the collocation polynomials.
- For [Periodic orbits based on the shooting method](#), you need more parameters. For example, you can pass `Shooting(M, odeprob, Euler())` or `PoincareShooting(M, odeprob, Euler())` where `odeprob::ODEProblem` (see [DifferentialEquations.jl](#)) is an ODE problem to specify the Cauchy problem amd `M` is the number of sections.

See [Branch switching \(Hopf point\)](#) for the precise method definition

## Algorithm

The algorithm proceeds as follows. The normal form of the Hopf bifurcation is first computed. Then a predictor for the bifurcated branch of periodic orbits is generated from the normal form. Finally, this predictor is used as a guess for the computation of periodic orbits.

## Example

The simplest example is from the [getting-started section](#) which we repeat partially below. Several examples are provided in [example ODE](#). In the case of PDE, you can have a look at [Brusselator](#) or [2d Ginzburg-Landau equation](#).



```
using BifurcationKit, Plots

GitHub
function Fsl(X, p)
    (;r, μ, ν, c3) = p
    u, v = X
    ua = u^2 + v^2
    [
        r * u - ν * v - ua * (c3 * u - μ * ν)
        r * v + ν * u - ua * (c3 * v + μ * u)
    ]
end

par_sl = (r = 0.1, μ = 0., ν = 1.0, c3 = 1.0)
u0 = zeros(2)
prob = ODEBifProblem(Fsl, u0, par_sl, (@optic _ .r))
opts = ContinuationPar()
br = continuation(prob, PALC(), opts, bothside = true)
```

Curve type: EquilibriumCont

Number of points: 25

Type of vectors: Vector{Float64}

Parameter r starts at -1.0, ends at 1.0

Algo: PALC [Secant]

Special points:

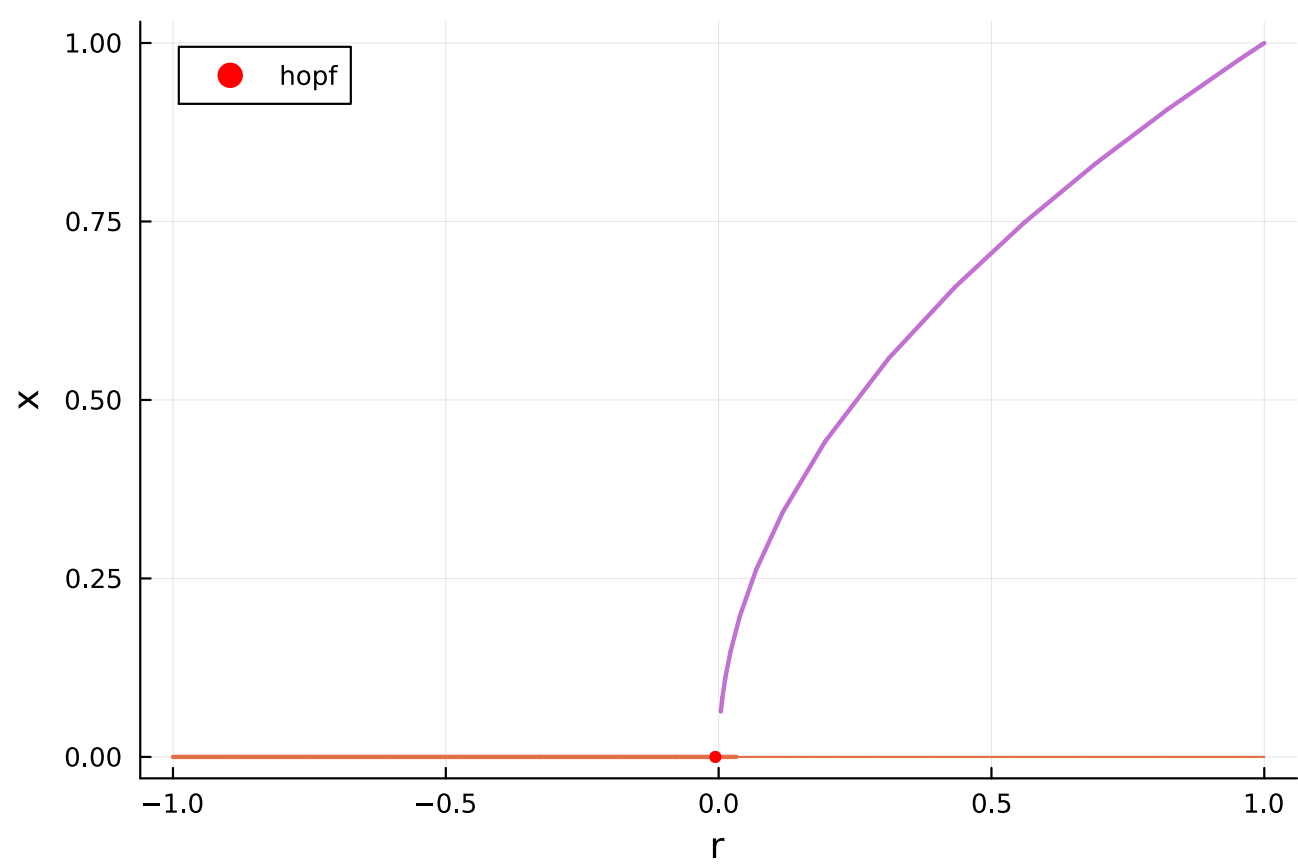
- # 1, endpoint at r ≈ -1.00000000,

- # 2, hopf at r ≈ -0.00595553 ∈ (-0.00595553, +0.00299379), |δp|=9e-03, [converged], δ = ( 2, 2),

- # 3, endpoint at r ≈ +1.00000000,

We then compute the branch of periodic solutions using orthogonal collocation (for example):

```
br_po = continuation(br, 2, opts,
    Collocation(20, 5)
)
plot(br, br_po)
```



## From Period-doubling point to curve of periodic orbits

For all cases, an example of use is provided in [Lur'e problem](#).



we provide an automatic branching procedure in this case. In essence, all you have to do is to call

GitHub

```
continuation(br::ContResult, ind_PD::Int, _contParams::ContinuationPar;  
    prm = Val(true), detailed = Val(true),  
    kwargs...)
```

The option `prm = Val(true)` enforces that the period-doubling normal form is computed using the Poincaré return map ; this is only necessary in case of use of the collocation method. Indeed, in the case of the collocation method, an automatic procedure based on the looss normal form has yet to be implemented.

## Case of Trapezoid method

We do not provide (for now) the automatic branching procedure for these bifurcations of periodic orbits. As a consequence, the user is asked to provide the amplitude of the bifurcated solution.

The call is as follows. Please note that a deflation is included in this method to simplify branch switching.

```
continuation(br::AbstractBranchResult, ind_PD::Int, contParams::ContinuationPar;  
    δp = 0.1, ampfactor = 1,  
    usedeflation = false,  
    kwargs...)
```

## Algorithm

The algorithm proceeds as follows. The normal form of the Period-doubling bifurcation is first computed. Then a predictor for the bifurcated branch of periodic orbits is generated from the normal form. Finally, this predictor is used as a guess for the computation of periodic orbits.

## From Branch point to curve of periodic orbits

We do not provide (for now) the automatic branching procedure for these bifurcations of periodic orbits. As a consequence, the user is asked to provide the amplitude of the bifurcated solution.

We provide the branching method for the following methods to compute periodic orbits: [Trapeze](#), [Collocation](#), [Shooting](#) and [PoincareShooting](#). The call is as follows. Please note that a deflation is included in this method to simplify branch switching.

An example of use is provided in [Lur'e problem](#).

```
continuation(br::AbstractBranchResult, ind_PD::Int, contParams::ContinuationPar;  
    δp = 0.1, ampfactor = 1, usedeflation = false, kwargs...)
```

# Branch point of periodic orbit

- Branch point of periodic orbit
  - Predictor
  - Which method to use?
  - Normal form based on Poincaré return map
  - References

At a Branch point (BP) of a periodic orbit  $\gamma$  (with period  $T$ ) for the Cauchy problem at parameter value  $p_0$ :

$$\frac{du}{dt} = F(u, p), \tag{E}$$

the eigenvalues (Floquet coefficients) of the monodromy operator  $\mathcal{M} = Y(T)$  solution to

$$\frac{dY}{dt} = A(t)Y(t), Y(0) = I_n$$

contain the eigenvalue 1 with algebraic multiplicity 2. There are two ways to compute the normal form of this bifurcation

- using the Poincaré return map [\[Kuznetsov\]](#)
- using the method of [\[looss\]](#), see also [\[Kuz2\]](#)

You can obtain the normal form of a BP bifurcation using

```
pd = get_normal_form(br, ind; prm = Val(false))
```

where `prm` indicates whether you want to use the method based on Poincaré return map (PRM) or the one based on `looss` method.

## Predictor

The predictor for a non trivial guess at distance  $\delta p$  from the bifurcation point is provided by the method:

▼ `BifurcationKit.predictor` — Method

```
predictor(  
  nf::BifurcationKit.BranchPointPO{<:Shooting},  
  δp,  
  ampfactor;  
  override  
) -> NamedTuple{(:orbitguess, :pnew, :prob, :ampfactor, :po), <:Tuple{Any, Any, Shooting, Any, Any}}
```

Compute the predictor for the simple branch point of periodic orbit.

If `override = true`, then the predictor is simply `x0 .+ ampfactor .* e` for the parameter `p0 + δp` and where `e` is a bifurcating eigenvector.

## Which method to use?

Depending on the method used for computing the periodic orbits, you have several possibilities:

- For collocation, you can use PRM and looss methods. Note that the looss method is the most precise. **This is not yet implemented.** [GitHub](#)
- For Trapezoid method, the normal form is **not yet implemented**.

## Normal form based on Poincaré return map

Given a transversal section  $\Sigma$  to  $\gamma$  at  $\gamma(0)$ , the Poincaré return map  $\mathcal{P}$  associates to each point  $x \in \Sigma$  close to  $\gamma(0)$  the first point  $\mathcal{P}(x, p) \in \Sigma$  where the orbit of (E) with initial condition  $x$  intersects again  $\Sigma$  at  $\mathcal{P}(x, p)$ . Hence, the discrete map  $x_{n+1} = \mathcal{P}(x_n, p)$  has normal form

$$x_{n+1} = x_n + a_{10}\delta p + a_{11}\delta p x + a_{02}x^2 + a_{03}x^3$$

 **Large scale problems**

The computation of the normal form is not optimized for Matrix-Free problems (e.g. Monodromy) yet.

## References

- Kuznetsov

Yu. A. Kuznetsov, "Elements of Applied Bifurcation Theory", 2nd ed., 1998.
- Kuz2

Kuznetsov et al., “Numerical Periodic Normalization for Codim 1 Bifurcations of Limit Cycles.”
- looss

looss, "Global Characterization of the Normal Form for a Vector Field near a Closed Orbit.", 1988

# Period-doubling point

- Period-doubling point
  - Which method to use?
  - Predictor
  - Normal form based on Poincaré return map
  - Normal form based on looss method
  - References

At a period-doubling (PD) bifurcation of a periodic orbit  $\gamma$  (with period  $T$ ) for parameter value  $p_0$  for the Cauchy problem

$$\frac{du}{dt} = F(u, p), \tag{E}$$

the eigenvalues (Floquet coefficients) of the monodromy operator  $\mathcal{M} = Y(T)$  solution to

$$\frac{dY}{dt} = A(t)Y(t), Y(0) = I_n$$

contain the simple eigenvalue  $\mu = -1$ .

There are two ways to compute the normal form of this bifurcation

- using the Poincaré return map [\[Kuznetsov\]](#)
- using the method of [\[looss\]](#) see also [\[Kuz2\]](#)

You can obtain the normal form of a PD bifurcation using

```
pd = get_normal_form(br, ind; prm = Val(false))
```

where `prm` indicates whether you want to use the method based on Poincaré return map (PRM) or the one based on looss method.

## Which method to use?

Depending on the method used for computing the periodic orbits, you have several possibilities:

- For shooting, you can only the PRM method. Shooting is the preferred way for large scale systems. Note that the PRM method is not very precise numerically.
- For collocation, you can use PRM and looss methods. Note that the looss method is the most precise.
- For Trapezoid method, PD normal form is **not yet implemented**.

## Predictor

The predictor for a non trivial guess at distance  $\delta p$  from the bifurcation point is provided by the method

▼ [BifurcationKit.predictor](#) — Method

```
predictor(  
  nf::BifurcationKit.PeriodDoublingPO{<:Shooting},  
  δp,  
  ampfactor;
```

Computes the predictor for the period-doubling bifurcation of periodic orbit.

GitHub

If `override = true`, then the predictor is simply `x0 .+ ampfactor .* e` for the parameter `p0 + δp` and where `e` is a bifurcating eigenvector.

## Normal form based on Poincaré return map

Given a transversal section  $\Sigma$  to  $x_0$  at  $x_0(0)$ , the Poincaré return map  $\mathcal{P}$  associates to each point  $x \in \Sigma$  close to  $x_0(0)$  the first point  $\mathcal{P}(x, p) \in \Sigma$  where the orbit of (E) with initial condition  $x$  intersects again  $\Sigma$  at  $\mathcal{P}(x, p)$ . Hence, the discrete map  $x_{n+1} = \mathcal{P}(x_n, p)$  has normal form

$$x_{n+1} = -x_n + cx_n^3 + \dots$$

where [\[Kuz2\]](#)

$$c = \frac{1}{6} \left\langle p^*, \mathcal{C}(p, p, p) + 3\mathcal{B} \left( p, (I_{n-1} - \mathcal{A})^{-1} \mathcal{B}(p, p) \right) \right\rangle$$

where  $\mathcal{C} = d^3\mathcal{P}(x_0(0))$ ,  $\mathcal{B} = d^2\mathcal{P}(x_0(0))$  and  $\mathcal{A} = d\mathcal{P}(x_0(0))$ . Also:

$$\mathcal{A}p = -p, \mathcal{A}^T p^* = -p^*$$

❗

**Large scale problems**

The computation of the normal form is not optimized for Matrix-Free problems (e.g. Monodromy) yet.

## Normal form based on looss method

This is based on [\[looss\]](#), [\[Kuz2\]](#). Suppose that the  $T$  periodic orbit  $x_0(\tau)$  has a Period-Doubling bifurcation for a parameter value  $p_0$ . Locally, the orbits can be represented by  $p - p_0 := \mu$  and

$$x(\tau) = x_0(\tau) + \xi v(\tau) + H(\tau, \xi, \mu)$$

where

$$\begin{cases} \frac{d\tau}{dt} = 1 + a_{01} \cdot (p - p_0) + a_2 \xi^2 + \dots \\ \frac{d\xi}{d\tau} = c_{11} \cdot (p - p_0) \xi + c_3 \xi^3 + \dots \end{cases}$$

with center manifold correction  $H(\tau, \xi, \mu)$  being  $2T$  periodic in  $\tau$  and  $v(\tau)$  is a Floquet eigenvector for the eigenvalue -1.

## References

- Kuznetsov

Yu. A. Kuznetsov, "Elements of Applied Bifurcation Theory", 2nd ed., 1998.
- Kuz2

Kuznetsov et al., “Numerical Periodic Normalization for Codim 1 Bifurcations of Limit Cycles.”
- looss

looss, "Global Characterization of the Normal Form for a Vector Field near a Closed Orbit.", 1988





# Neimark-Sacker point

- Neimark-Sacker point
  - Which method to use?
  - Normal form based on Poincaré return map
  - Normal form based on looss method
  - References

At a Neimark-Sacker (NS) bifurcation of a periodic orbit  $\gamma$  (with period  $T$ ) for parameter value  $p_0$  for the Cauchy problem

$$\frac{du}{dt} = F(u, p), \tag{E}$$

the eigenvalues (Floquet coefficients) of the monodromy operator  $\mathcal{M} = Y(T)$  solution to

$$\frac{dY}{dt} = A(t)Y(t), Y(0) = I_n$$

contain the eigenvalues  $e^{\pm i\theta}$  with  $\theta$  and

$$e^{iq\theta} - 1 \neq 0, \quad q = 1, 2, 3, 4 \text{ (no strong resonances).}$$

There are two ways to compute the normal form of this bifurcation

- using the Poincaré return map [\[Kuznetsov\]](#)
- using the method of [\[looss\]](#), see also [\[Kuz2\]](#)

You can obtain the normal form of a NS bifurcation using

```
ns = get_normal_form(br, ind; prm = Val(false))
```

where `prm` indicates whether you want to use the method based on Poincaré return map (PRM) or the one based on looss method.

## Which method to use?

Depending on the method used for computing the periodic orbits, you have several possibilities:

- For shooting, you can only the PRM method. Shooting is the preferred way for large scale systems. Note that the PRM method is not very precise numerically.
- For collocation, you can use PRM and looss methods. Note that the looss method is **the most precise**.
- For Trapezoid method, NS normal form is **not yet implemented**.

## Normal form based on Poincaré return map

Given a transversal section  $\Sigma$  to  $\gamma$  at  $\gamma(0)$ , the Poincaré return map  $\mathcal{P}$  associates to each point  $x \in \Sigma$  close to  $\gamma(0)$  the first point  $\mathcal{P}(x, p) \in \Sigma$  where the orbit of (E) with initial condition  $x$  intersects again  $\Sigma$  at  $\mathcal{P}(x, p)$ . Hence, the discrete map  $x_{n+1} = \mathcal{P}(x_n, p)$  has normal form

$$z_{n+1} = z_n e^{i\theta} (1 + d|z_n|^2)$$

where [\[Kuz2\]](#)

where  $\mathcal{C} = d^3\mathcal{P}(\gamma(0))$ ,  $\mathcal{B} = d^2\mathcal{P}(\gamma(0))$  and  $\mathcal{A} = d\mathcal{P}(\gamma(0))$ . Also:

$$\mathcal{A}v = e^{i\theta}v, \mathcal{A}^\mathrm{T}v^* = e^{-i\theta}v^*, \text{ and } \langle v^*, v \rangle = 1$$

!

Large scale problems

The computation of the normal form is not optimized for Matrix-Free problems (e.g. Monodromy) yet.

## Normal form based on looss method

This is based on [\[looss\]](#), [\[Kuz2\]](#). Suppose that the  $T$  periodic orbit  $\gamma(\tau)$  has a Neimark-Sacker bifurcation for a parameter value  $p_0$ . We also assume that there are no strong resonances. Locally, the orbits can be represented by

$$x(\tau) = \gamma(\tau) + Q_0(\tau)\xi + \Phi(\tau, \xi)$$

where

$$\begin{cases} \frac{d\tau}{dt} = 1 + a|\xi|^2 + \dots \\ \frac{d\xi}{dt} = \frac{i\theta}{T}\xi + d\xi|\xi|^2 + \dots \end{cases}$$

with center manifold correction  $\Phi(\tau, \xi)$  being  $T$  periodic in  $\tau$  and  $Q_0(\tau)$  is the Floquet operator.

## References

- Kuznetsov
- Kuz2
- looss

Yu. A. Kuznetsov, "Elements of Applied Bifurcation Theory", 2nd ed., 1998.

Kuznetsov et al., “Numerical Periodic Normalization for Codim 1 Bifurcations of Limit Cycles.”

looss, "Global Characterization of the Normal Form for a Vector Field near a Closed Orbit.", 1988



# Linear solvers (LS)

- Linear solvers (LS)
  - List of implemented linear solvers
  - Preconditioner

If you provide your own linear solver, it must be a subtype of `AbstractLinearSolver` otherwise `BifurcationKit.jl` will not recognize it. See example just below.

The linear solvers provide a way of inverting the Jacobian `J` or solving  $J * x = rhs$ . Such linear solver `linsolve` will be called like `sol, success, itnumber = linsolve(J, rhs; kwargs...)` throughout the package.

Here is an example of the simplest one (see `src/LinearSolver.jl` for the true implementation) to give you an idea, the backslash operator:

```
struct DefaultLS <: AbstractLinearSolver end

function (l::DefaultLS)(J, rhs; k...)
    return J \ rhs, true, 1
end
```

Note that for `newton` to work, the linear solver must return 3 arguments. The first one is the result, the second one is whether the computation was successful and the third is the number of iterations required to perform the computation.

You can then call it as follows (and it will be called like this in `newton`)

```
ls = DefaultLS()
J = rand(2, 2) # example of linear operator
ls(J, rand(2))
```

## List of implemented linear solvers

The detailed information for each one of them is located in the [API](#).

- Default `\` solver based on LU or Cholesky depending on the type of the Jacobian. This works for sparse matrices as well. You can create one via `linsolver = DefaultLS()`.
- GMRES from `IterativeSolvers.jl`. You can create one via `linsolver = GMRESIterativeSolvers()` and pass appropriate options.
- GMRES from `KrylovKit.jl`. You can create one via `linsolver = GMRESKrylovKit()` and pass appropriate options.
- All solvers in `Krylov.jl`. You can create one via `linsolver = KrylovLS()` or `KrylovLSInplace()` and pass appropriate options. This is available from `BifurcationKit@0.4.5`

### ! Different linear solvers

By tuning the options of `GMRESKrylovKit`, you can select CG, GMRES... see [KrylovKit.jl](#).

### ! Other solvers

For `KrylovLS`, just pass the required solver, like `cg`.

It is very straightforward to implement the Conjugate Gradients from [IterativeSolvers.jl](#) by copying the interface done for `gmres`. Same goes for `minres`,... Not needing them, I did not implement this.

Preconditioners should be considered when using Matrix Free methods such as GMRES. `GMRESIterativeSolvers` provides a very `Cuthill` interface for using them. For `GMRESKrylovKit`, we implemented a left preconditioner. Note that, for `GMRESKrylovKit`, you are not restricted to use `Vectors` anymore.

A curated list is provided by the package `LinearSolve.jl`.

Finally, here are some packages to use preconditioners:

1. `Krylov.jl` provides some preconditioners.
2. `IncompleteLU.jl` an ILU like preconditioner
3. `AlgebraicMultigrid.jl` Algebraic Multigrid (AMG) preconditioners. This works especially well for symmetric positive definite matrices.
4. `Preconditioners.jl` A convenient interface to conveniently called most of the above preconditioners using a single syntax.
5. We provide a preconditioner based on deflation of eigenvalues (also called preconditioner based on Leading Invariant Subspaces) using a partial Schur decomposition. There are two ways to define one *i.e.* `PrecPartialSchurKrylovKit` and `PrecPartialSchurArnoldiMethod`.

 **Using Preconditioners**

Apart from setting a preconditioner for a linear solver, it can be advantageous to change the preconditioner during computations, *e.g.* during a call to continuation or newton. This can be achieved by taking advantage of the callbacks to these methods. See the example `2d Ginzburg-Landau equation (fnite differences, codim 2, Hopf aBS)`.

# Bordered linear solvers (BLS)

- Bordered linear solvers (BLS)
  - Full matrix `MatrixBLS`
  - Bordering method `BorderingBLS`
  - Full Matrix-Free `MatrixFreeBLS`
  - References

The bordered linear solvers must be subtypes of `AbstractBorderedLinearSolver` `<: AbstractLinearSolver`.

The methods provided here solve bordered linear equations. The detailed information for each one of them is located in the [API](#).

We look for the solution  $u$  of  $J \cdot u = v$  where

$$J = \begin{pmatrix} A & b \\ c^T & d \end{pmatrix} \text{ and } v = \begin{pmatrix} v_1 \\ v_2 \end{pmatrix} \tag{E}$$

Such linear solver `bdlsolve` will be called like `sol, success, itnumber = bdlsolve(A, b, c, d, v1, v2)` throughout the package.

### Complex numbers

In the case where  $c \in \mathbb{C}^N$ , please note that the adjoint operator  $c^T$  involves a conjugate.

## Full matrix `MatrixBLS`

This easiest way to solve  $(E)$  is by forming the matrix  $J$ . In case it is sparse, it should be relatively efficient. You can create such bordered linear solver using `bls = MatrixBLS(ls)` where `ls::AbstractLinearSolver` is a linear solver (which defaults to `\`) used to solve the linear problem associated to  $J$ . This is the default method used in the package.

## Bordering method `BorderingBLS`

The general solution  $u = \begin{pmatrix} u_1 \\ u_2 \end{pmatrix}$  to  $(E)$  when  $A$  is non singular is

$$\begin{aligned} u_2 &= \frac{1}{d - c \cdot x_2} (v_2 - c \cdot x_1) \\ u_1 &= x_1 - u_2 x_2 \end{aligned}$$

where  $x_1 = A^{-1}v_1, x_2 = A^{-1}b$ .

It is very efficient for large scale problems because it is entirely Matrix-Free and one can use preconditioners. You can create such bordered linear solver using `bls = BorderingBLS(ls)` where `ls::AbstractLinearSolver` is a linear solver which defaults to `\`. The intermediate solutions  $x_1 = A^{-1}v_1, x_2 = A^{-1}b$  are formed using `ls`.

- In the case where `ls = DefaultLS()`, the factorisation of `A` is cached so the second linear solve is very fast

There are more options to `BorderingBLS`. First, the residual can be checked using the option `check_precision = true`. If the residual is above a prescribed tolerance, an iterative method is used based on several bordering transformations. This is the *BEC+k* algorithm in [\[Govaerts\]](#).

In cases where  $A$  is singular but  $J$  is not, the bordering method may fail. It can thus be advantageous to form the Matrix-Free version of  $J$  and call a generic linear solver to find the solution to  $(E)$ . You can create such bordered linear solver using `bls = MatrixFreeBLS(ls)` where `ls::AbstractLinearSolver` is a (Matrix Free) linear solver which is used to invert  $J$ .

For now, this linear solver only works with `AbstractArray`

## References

- Govaerts

Govaerts, W. "Stable Solvers and Block Elimination for Bordered Systems." SIAM Journal on Matrix Analysis and Applications 12, no. 3 (July 1, 1991): 469–83. <https://doi.org/10.1137/0612034>.

# Eigen solvers (Eig)

- Eigen solvers (Eig)
  - Methods for computing eigenvalues
  - List of implemented eigen solvers
  - Generalized eigen solvers (GEV)

The eigen solvers must be subtypes of AbstractEigenSolver.

They provide a way of computing the eigen elements of the Jacobian  $J$ . Such eigen solver `eigsolve` will be called like `ev, evects, itnumber = eigsolve(J, nev; kwargs...)` throughout the package, `nev` being the number of requested eigen elements of largest real part and `kwargs` being used to send information about the algorithm (perform bisection,...).

Here is an example of the simplest of them (see `src/EigSolver.jl` for the true implementation) to give you an idea:

```
struct DefaultEig <: AbstractEigenSolver end

function (l::DefaultEig)(J, nev; kwargs...)
    # I put Array so we can call it on small sparse matrices
    F = eigen(Array(J))
    I = sortperm(F.values, by = real, rev = true)
    nev2 = min(nev, length(I))
    return F.values[I[1:nev2]], F.vectors[:, I[1:nev2]], true, 1
end
```

## ⚠ Eigenvalues

The eigenvalues must be ordered by decreasing real part for the detection of bifurcations to work properly.

## ⚠ Eigenvectors

You have to implement the method `geteigenvector(eigsolver, eigenvectors, i::Int)` for `newtonHopf` to work properly.

## Methods for computing eigenvalues

Like for the linear solvers, computing the spectrum of operators  $A$  associated to PDE is a highly non trivial task because of the clustering of eigenvalues. Most methods are based on the so-called [power method](#) but this only yields the eigenvalues with largest modulus. In case of the Laplacian operator, this can be disastrous and it is better to apply the power method to  $(\sigma I - A)^{-1}$  instead.

This method, called **Shift-invert**, is readily available for the solvers `EigArpack` and `EigArnoldiMethod`, see below. It is mostly used to compute interior eigenvalues. For the solver `EigKrylovKit`, one must implement its own shift invert operator, using for example `GMRESKrylovKit`.

In some cases, it may be advantageous to consider the **Cayley transform**  $(\sigma I - A)^{-1}(\tau I + A)$  to focus on a specific part of the spectrum. As it is mathematically equivalent to the Shift-invert method, we did not implement it.

## List of implemented eigen solvers

The detailed information for each one of them is located in the [API](#)

- eig(J, 3).
2. Eigensolver from `Arpack.jl`. You can create one via `eigsolver = EigArpack()` and pass appropriate options (see [Arpack.jl](#)). For example, you can compute eigenvalues using Shift-Invert method with shift  $\sigma$  by using `EigArpack( $\sigma$ , :LR)`. Note that you can specify how the eigenvalues are ordered (by decreasing real part by default). Finally, this method can be used for (sparse) matrix or Matrix-Free formulation. For a matrix `J`, you can compute 3 eigen-elements using `eig(J, 3)`. In the case of a Matrix-Free jacobian `dx -> J(dx)`, you need to tell the eigensolver the dimension of the state space by giving an example of vector: `eig = EigArpack(v0 = zeros(10))`. You can then compute 3 eigen-elements using `eig(dx -> J(dx), 3)`.

3. Eigensolver from `KrylovKit.jl`. You create one via `eig = EigKrylovKit()` and pass appropriate options (see [KrylovKit.jl](#)). This method can be used for (sparse) matrix or Matrix-Free formulation. In the case of a matrix `J`, you can create a solver like this `eig = EigKrylovKit()`. Then, you compute 3 eigen-elements using `eig(J, 3)`. In the case of a Matrix-Free jacobian `dx -> J(dx)`, you need to tell the eigensolver the dimension of the state space by giving an example of vector: `eig = EigKrylovKit(x0 = zeros(10))`. You can then compute 3 eigen-elements using `eig(dx -> J(dx), 3)`.

4. Eigensolver from `ArnoldiMethod.jl`. You can create one via `eig = EigArnoldiMethod()` and pass appropriate options (see [ArnoldiMethod.jl](#)). For example, you can compute eigenvalues using the Shift-Invert method with shift  $\sigma$  by using `EigArnoldiMethod( $\sigma$ , LR())`. Note that you can also specify how the eigenvalues are ordered (by decreasing real part by default). In the case of a matrix `J`, you can create a solver like `eig = EigArnoldiMethod()`. Then, you compute 3 eigen-elements using `eig(J, 3)`. In the case of a Matrix-Free jacobian `dx -> J(dx)`, you need to tell the eigensolver the dimension of the state space by giving an example of vector: `eig = EigArnoldiMethod(x0 = zeros(10))`. You can then compute 3 eigen-elements using `eig(dx -> J(dx), 3)`.

5. A general Shift-Invert strategy is implemented in `ShiftInvert`. See [ShiftInvert](#)

!

Slow computations

This is probably due to iterative refinement conducted by SuiteSparse as explained in this [blog post](#). You can disable this using

```
using SuiteSparse
SuiteSparse.UMFPACK.umf_ctrl[8] = 0
```

## Generalized eigen solvers (GEV)

Associated to an eigensolver `eig` in (`DefaultEig`, `EigArnoldiMethod`, `EigArpack`), a GEV is provided which can be called as

```
gev(eig, A, B, nev; k...)
```

Another way is to rely on `BK.EigenMassMatrix`. See [EigenMassMatrix](#).